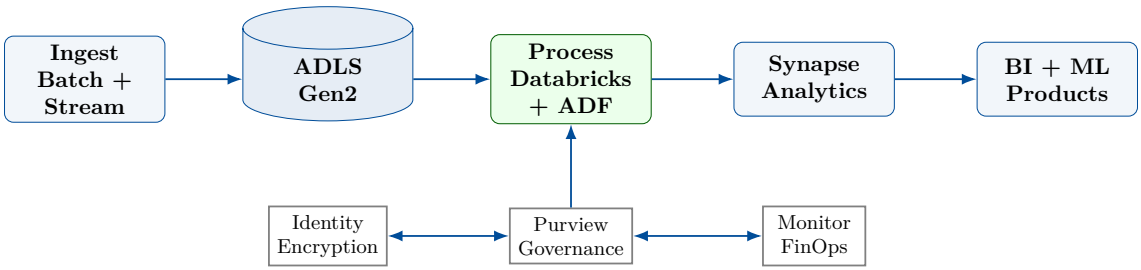
A Comprehensive, Hands-On Academic Guide

Azure Master
Data Engineer



Inside this book

Core Storage | Warehousing | Batch ETL | Streaming | ML + MLOps | Security + Exam

From secure lake storage to governed, observable, production-scale Azure data products.

PhD Aned Esquerra Arguelles

A 24-week hands-on program for Azure data engineering mastery

July 31, 2026

Preface

Welcome to *Azure Master Data Engineer*. This text is designed as an academic, hands-on, progressive program for building cloud-scale data platforms on Microsoft Azure. By working through the architecture discussions, command-line labs, and design exercises, you will move from storage foundations to governed lakehouse, warehouse, streaming, and machine-learning data products.

This book is the theory-and-architecture companion to a six-milestone Azure Data Engineering curriculum. Where the practice workspace asks you to build weekly deliverables, this book explains why each Azure service exists, what operational failure mode it addresses, and how a data engineer reasons about reliability, cost, compliance, and maintainability.

Who This Book Is For

This book is written for data engineers, analytics engineers, cloud engineers, database professionals, and technically minded analysts preparing for Azure data-platform work. It assumes basic SQL, Python, and command-line familiarity. Prior Azure experience is helpful but not required: the early chapters introduce storage accounts, identity, naming, redundancy, and lifecycle controls before the program moves into Synapse, Data Factory, Databricks, Event Hubs, Cosmos DB, Azure Machine Learning, and governance.

How This Book Is Organized

The book follows the training roadmap as **6 Parts / 6 Milestones** and **24 Chapters / 24 Weeks**, followed by appendices:

- **Part I: Core Data Infrastructure and Storage** establishes ADLS Gen2, secure access, file formats, medallion zones, and governance foundations.
- **Part II: Data Warehousing and Analytics** builds analytical serving layers with Azure Synapse Analytics, serverless SQL, dedicated SQL pools, and Power BI semantic models.
- **Part III: Batch Processing and ETL Orchestration** develops repeatable pipelines with Azure Data Factory, integration runtimes, Databricks, Spark, and Delta Lake.
- **Part IV: Streaming and Real-Time Analytics** introduces Event Hubs, IoT ingestion, Stream Analytics, real-time Spark, and operational analytics patterns with Cosmos DB.
- **Part V: Machine Learning and MLOps** connects curated data products to Azure Machine Learning, feature engineering, pipeline automation, model monitoring, and GenAI-ready data services.
- **Part VI: Security, Governance, and Exam Prep** integrates Zero Trust, observability, FinOps, compliance, DP-203-style exam review, and the enterprise capstone.

Each chapter opens with **Learning Objectives** and closes with **Further Reading**, **Key Takeaways**, and **Exercises**. Hands-on chapters include Azure CLI, JSON, SQL, or Python listings that are meant to be adapted in a real subscription.

Reading Paths

- **New to Azure data engineering:** read Part I in order, perform every hands-on storage exercise, then continue to Synapse and Data Factory.
- **Experienced SQL or BI practitioner:** skim Chapter 1 for storage vocabulary, then read Parts II and III while returning to governance notes as needed.
- **Spark or Databricks practitioner:** read Chapters 1–4 for Azure storage and governance assumptions, then focus on Databricks, Delta Lake, streaming, and MLOps.
- **Platform architect:** prioritize architecture diagrams, pitfall boxes, Well-Architected trade-offs, and the security/governance chapters.
- **Certification candidate:** complete the conceptual exercises, review the command-line examples, and use Part VI to consolidate DP-203 objectives.

Conventions

Code identifiers, resource names, and file paths appear in `monospace`. Azure CLI snippets use the shared listing style and assume PowerShell on Windows unless noted. Callout boxes flag **objectives**, **key terms**, **definitions**, **notes**, **tips**, and **pitfalls**. Citations refer to the bibliography at the back of the book. Service names use their current Microsoft branding, while design guidance emphasizes transferable principles: least privilege, explicit recovery objectives, lifecycle economics, reproducible infrastructure, observability, and governed data contracts.

Contents

List of Figures

List of Tables

Chapter 0

Getting Started: Your Azure Free Account and Environment

Learning Objectives

- Create an Azure free account, understand the $200 credit window, and distinguish free credit, 12-month free services, and always-free tiers.
- Explain the Azure tenant, subscription, resource group, and resource hierarchy used throughout this book.
- Secure the lab identity with Microsoft Entra ID, MFA, Azure RBAC, least privilege, and separated administrative accounts.
- Install and authenticate Azure CLI on Windows PowerShell, and recognize Azure PowerShell and Azure Cloud Shell alternatives.
- Configure cost guardrails with Microsoft Cost Management budgets, alerts, tags, and cleanup routines before deploying labs.
- Apply the book's default region, naming, and tagging conventions to make every exercise reproducible and easy to delete.
- Verify the environment by creating, listing, and deleting a resource group and storage account from PowerShell.

Key Terms

Tenant: a Microsoft Entra ID directory that stores identities, groups, applications, and security policies. **Subscription**: an Azure billing and management boundary that contains resource groups and resources. **Resource group**: a logical lifecycle container for Azure resources that are deployed, tagged, authorized, and deleted together. **Azure RBAC**: the role-based authorization system that grants actions over Azure scopes. **Budget**: a Microsoft Cost Management rule that tracks spending and sends alerts when thresholds are crossed.

0.1 Why Chapter 0 Exists

A data engineering book should not begin with an expensive Spark cluster or a production data lake. It should begin with a controlled account, a known command-line environment, a least-privilege identity, a budget, and a cleanup habit. Most cloud accidents in early learning projects are not caused by advanced distributed-systems failures. They are caused by using the wrong subscription, leaving a resource group running, granting a broad role to make an error disappear, or forgetting that globally unique services continue to bill after the tutorial window

closes.

This chapter establishes the operating baseline for every Azure exercise in the book. The environment is intentionally Windows plus PowerShell because that is the assumed local shell for this training repository. Azure CLI commands are shown in PowerShell syntax when variables or line continuation are needed. Azure Cloud Shell remains useful when a local machine cannot install software, but the preferred workflow for repeatable labs is a local terminal, a named subscription, a standard resource group prefix, and explicit cleanup.

Definition

A **preliminary cloud environment** is the smallest secure setup that lets you deploy, inspect, measure, and delete learning resources without relying on production credentials or guesswork.

Note

Azure product names change over time. Azure Active Directory is now **Microsoft Entra ID**. Older documentation, screenshots, and exam material may still say Azure AD, but this book uses Microsoft Entra ID except when quoting legacy names.

0.2 Creating an Azure Free Account

0.2.1 What the Free Account Includes

Microsoft's Azure free account is designed for learning, evaluation, and small experiments. At sign-up time, Microsoft commonly offers three distinct benefits: a $200 credit that can be used during the first 30 days, selected popular services free for 12 months, and always-free service quantities that continue as long as the account remains eligible [14]. These benefits are related but not identical. The $200 credit is a short introductory balance. The 12-month services are service-specific allowances. Always-free tiers are ongoing limits for selected services, often with monthly quotas.

For this book, the free account is sufficient for many foundational labs if you create only the requested resources, choose modest SKUs, and delete resource groups after each exercise. It is not a blank check: every create command should have a matching cleanup step.

Free-account item	What it means	Engineering caution
$200 credit	Introductory credit for eligible Azure charges during the first 30 days	Credit can be exhausted quickly by large compute, premium storage, or forgotten services
12-month free services	Popular services have limited free quantities for 12 months	Only the listed quantity and SKU are free; exceeding limits bills normally
Always-free tiers	Selected services have ongoing free monthly allowances	Quotas reset by service rules and may not cover realistic data workloads
Spending limit	Some free or credit subscriptions prevent charges beyond credit until upgraded	Upgrading or changing offer type can remove the safety net

Table 1: The Azure free account combines time-limited credit, service-specific allowances, and ongoing free quotas.

0.2.2 Sign-up Walkthrough

Begin at the Azure free account page and choose *Start free*. You need a Microsoft account, such as an Outlook account or another email address registered as a Microsoft account. Microsoft asks for contact information, phone verification, agreement to terms, and a payment card. The card is used for identity verification and fraud prevention. For a free account, Microsoft states that you are not automatically charged unless you explicitly upgrade or exceed an offer that permits billing [14].

A practical sign-up sequence is:

1. Open the Azure free account page in a private browser window so an existing work or school login does not accidentally select the wrong tenant.
2. Sign in with the Microsoft account that will own the learning environment. If you already have an organizational Azure account, keep it separate from this book's lab account unless your instructor says otherwise.
3. Complete identity verification with phone and card. Use a card you are authorized to use, and record who owns the account in your lab notebook.
4. Open the Azure Portal and confirm that a subscription exists under *Subscriptions*. Copy the subscription name and subscription ID.
5. Immediately create a budget, enable MFA, and decide a naming prefix before deploying the first technical resource.

Common Pitfall

Using the wrong tenant during sign-up. Many learners have a personal Microsoft account and a work or school account with the same email address. If the portal asks whether to use a personal or work account, pause and verify which identity should own the labs. Deploying into an employer tenant can create policy, billing, and cleanup problems.

0.2.3 The Azure Resource Hierarchy

Azure resources live in a hierarchy. At the identity layer, Microsoft Entra ID provides the tenant that contains users, groups, applications, and service principals. At the Azure management layer, management groups can organize multiple subscriptions. A subscription contains resource groups. A resource group contains resources such as storage accounts, key vaults, factories, workspaces, and virtual networks [31].

For this book, the most important scopes are subscription, resource group, and resource. The subscription is the billing and policy boundary for the labs. The resource group is the disposable unit for each chapter or week. The resource is the actual service instance. If you deploy every chapter into a clearly named resource group, cleanup is normally one command: delete that group.

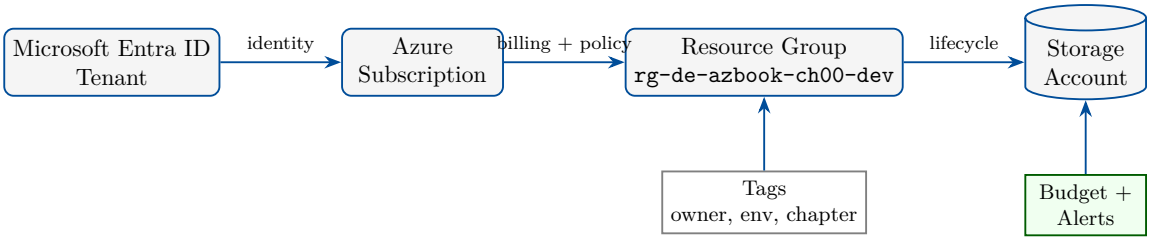


Figure 1: Chapter 0 setup flow from identity and subscription selection to a tagged, budgeted lab resource group.

Tip

Treat the resource group as the learning lab's blast-radius boundary. A predictable resource group name lets you review all resources, export a deployment record, and delete the complete lab without hunting through the portal.

0.3 Securing Identity Before Deploying Resources

0.3.1 Microsoft Entra ID and Administrative Separation

Microsoft Entra ID is Azure's identity platform for users, groups, enterprise applications, service principals, conditional access, and MFA [47]. In a new personal learning tenant, you may be the only user. In an organization, the tenant may already have administrators, policies, and approved groups. Either way, do not normalize daily work as a highly privileged administrator.

The **Global Administrator** role is a tenant-wide Microsoft Entra role. It can manage identity settings and many administrative functions across the tenant. It is not the right role for running day-to-day data engineering commands. Daily work should use a normal user account with only the Azure RBAC permissions needed in the lab subscription or resource group. Administrative accounts should be reserved for administrative actions, protected with MFA, and monitored.

Definition

Least privilege means granting an identity only the minimum actions, data access, and scope required to complete a task, for only as long as the task requires.

0.3.2 Azure RBAC Roles for Labs

Azure RBAC controls management-plane access at scopes such as subscription, resource group, or resource [44]. The common built-in roles are:

- **Owner**: can manage resources and assign access to others. Use sparingly because it can delegate privileges.
- **Contributor**: can create and manage resources but cannot grant access. This is often sufficient for a learner working inside a dedicated lab resource group.
- **Reader**: can view resources but cannot change them. Use for reviewers, auditors, and safe inspection.

Data services often have separate data-plane roles. For example, managing a storage account is different from reading blobs inside it. Later chapters distinguish roles such as `Storage Blob Data Reader` and `Storage Blob Data Contributor`. Chapter 0 focuses on management-plane setup so that later labs can explain data-plane permissions in context.

Common Pitfall

Solving every error with Owner. Granting Owner at the subscription level may make a command succeed, but it hides the real boundary: missing RBAC, missing data-plane role, missing ACL, wrong subscription, or blocked network path. Broad permissions are hard to audit and easy to forget.

0.3.3 Create a Least-Privileged User or Service Principal

For an individual learning account, the first identity goal is separation. Use one account for emergency administration and a different daily account for labs when your tenant license and permissions allow it. In an organization, ask for a dedicated lab subscription or resource group and a role assignment scoped to that boundary.

A service principal is an application identity used by automation. It is useful for CI/CD and non-interactive scripts, but it must be protected like any credential. For early manual labs, prefer interactive user login with MFA. Use a service principal only when a lab requires automation and store secrets in the approved secret store, not in source files or screenshots.

```
# Run only after creating the resource group and confirming the scope.
$subscriptionId = "00000000-0000-0000-0000-000000000000"
$rg = "rg-de-azbook-ch00-dev"
$scope = "/subscriptions/$subscriptionId/resourceGroups/$rg"

az ad sp create-for-rbac `
  --name "sp-de-azbook-ch00-dev" `
  --role Contributor `
  --scopes $scope

# Store the output securely. Do not commit appId, password, or tenant.
```

Listing 1: Creating a resource-group scoped service principal with Azure CLI from PowerShell.

The command returns credentials once. If you lose the secret, create a new one rather than searching terminal history or files. Later chapters prefer managed identity where Azure compute supports it.

0.3.4 Enable MFA

Enable MFA before you deploy resources. MFA reduces the risk that a stolen password becomes a working cloud credential. In a personal Microsoft account, configure two-step verification or the Microsoft Authenticator app through account security settings. In a Microsoft Entra tenant, use the Entra admin center or Security Defaults when appropriate. Organizations normally enforce MFA through Conditional Access policies.

Note

MFA is not a replacement for least privilege. A phished account with MFA fatigue approval and subscription Owner rights can still create resources, assign roles, or delete evidence. Use MFA, scoped roles, budgets, and logs together.

0.4 Installing and Authenticating Tooling on Windows

0.4.1 Azure CLI on Windows

Azure CLI is the primary command-line tool used in this book. It works across platforms and is well supported by Azure documentation [24]. On Windows, install it with the Microsoft

Installer package, with `winget`, or from Microsoft's installation page. After installation, open a new PowerShell window so the updated `PATH` is available.

```
# Option A: install with Windows Package Manager.
winget install --exact --id Microsoft.AzureCLI

# Confirm the CLI is available.
az version

# Browser-based sign-in. Complete MFA in the browser when prompted.
az login

# List subscriptions visible to this identity.
az account list --output table

# Select the subscription used for this book.
az account set --subscription "00000000-0000-0000-0000-000000000000"

# Confirm the selected subscription.
az account show --output table
```

Listing 2: Installing Azure CLI and signing in from Windows PowerShell.

If `az login` opens a browser for the wrong account, sign out of the browser or use a private browser session. If your organization requires device-code login, Azure CLI will display a code and URL. Complete the flow with the approved identity.

Tip

Pin the active subscription in every new terminal session before running destructive commands. A short `az account show -output table` is cheaper than deleting a resource group from the wrong subscription.

0.4.2 Azure PowerShell as an Alternative

Azure PowerShell is a PowerShell-native module family named `Az`. It is excellent when an organization standardizes on PowerShell scripts, Desired State Configuration, or administrative automation [23]. This book uses Azure CLI for consistency, but Azure PowerShell is a first-class alternative.

```
# Run PowerShell as a normal user unless your module path requires elevation.
Install-Module -Name Az -Scope CurrentUser -Repository PSGallery -Force

Connect-AzAccount

Set-AzContext -Subscription "00000000-0000-0000-0000-000000000000"

Get-AzContext | Select-Object Name, Subscription, Tenant
```

Listing 3: Installing the Az module and selecting a subscription.

Do not mix Azure CLI and Azure PowerShell contexts without checking both. `az account set` affects Azure CLI. `Set-AzContext` affects Azure PowerShell. They may point to different subscriptions in the same terminal.

0.4.3 Azure Cloud Shell

Azure Cloud Shell is a browser-based shell hosted by Microsoft and available from the Azure Portal [32]. It includes Azure CLI and Azure PowerShell, requires no local installation, and au-

thenticates with the portal session. It is useful for locked-down workstations, quick inspections, and demonstrations.

Cloud Shell is not the same as your local machine. It has its own file system, mounted storage, installed tools, and network path. For book labs, local PowerShell gives you clearer control over files in this repository. Use Cloud Shell when local installation is blocked, but copy important commands and outputs back into your lab notes.

0.5 Cost Guardrails

0.5.1 Spending Limit, Budgets, and Alerts

Cost control starts before deployment. Azure free and credit-based subscriptions may have a spending limit that prevents charges beyond the available credit until the subscription is upgraded or the offer changes. Do not rely on that limit as your only protection. Spending limits are offer-specific, and some resources can consume credit rapidly. Microsoft Cost Management provides cost analysis, budgets, forecasts, and alerts [37].

A budget does not usually stop resources by itself. It alerts when actual or forecasted cost crosses thresholds. That alert is valuable because it gives you time to delete, resize, pause, or investigate resources before a small mistake becomes a large bill. For learning subscriptions, create a monthly budget with early thresholds such as 25%, 50%, 75%, and 90%.

Tip

Measure it. Every chapter that creates Azure resources should end with two measurements: current resources in the lab resource group and current or forecasted cost in Cost Management. If you cannot measure it, assume it can bill.

0.5.2 Create a Budget from PowerShell

The following listing creates a lab resource group and a monthly budget at subscription scope. Replace the email address, subscription ID, and amount with your own values. The budget amount is intentionally small for a learning environment.

```
$subscriptionId = "00000000-0000-0000-0000-000000000000"
$location = "eastus"
$rg = "rg-de-azbook-ch00-dev"
$budgetName = "budget-de-azbook-monthly"
$alertEmail = "student@example.com"
$startDate = "2026-08-01"
$endDate = "2027-08-01"

az account set --subscription $subscriptionId

az group create `
  --name $rg `
  --location $location `
  --tags project=azure-book environment=dev chapter=chapter0 owner=student

az consumption budget create `
  --budget-name $budgetName `
  --amount 25 `
  --time-grain Monthly `
  --time-period start-date=$startDate end-date=$endDate `
  --category Cost `
  --notifications `
```

```
    actual25 enabled true operator GreaterThan threshold 25 contact-emails
    $alertEmail '
    actual50 enabled true operator GreaterThan threshold 50 contact-emails
    $alertEmail '
    forecast90 enabled true operator GreaterThan threshold 90 contact-emails
    $alertEmail
```

Listing 4: Creating a tagged resource group and subscription budget with Azure CLI.

If your Azure CLI version rejects the notification syntax, create the budget in the portal under *Cost Management + Billing* and document the same thresholds.

0.5.3 Cost Analysis Workflow

Cost Analysis in the Azure Portal lets you group spending by service, resource group, location, tag, and meter. For this book, use three views repeatedly. First, group by resource group to confirm that lab spending is isolated. Second, group by service name to see whether storage, compute, networking, or monitoring is driving cost. Third, filter by tag so you can verify that your naming and tagging strategy is working.

Common Pitfall

Assuming deletion stops every charge immediately. Some meters are delayed, some retained data bills until removed, and some services create hidden supporting resources. After deleting a lab, check Cost Analysis over the next day and verify the resource group no longer exists.

0.6 Regions, Naming, and Tagging Used by This Book

0.6.1 Default Region

Unless a chapter says otherwise, this book uses `eastus` as the default region because it is widely available and commonly supports data platform services. If `eastus` is unavailable in your subscription, choose a nearby region that supports the required service and record the substitution. Do not mix regions casually inside a lab. Cross-region data movement, private endpoints, availability zones, and service availability all affect behavior and cost.

Note

Azure service availability varies by region and subscription type. If a command fails because a SKU is unavailable, choose the nearest supported region and keep the rest of the resource names and tags unchanged.

0.6.2 Naming Conventions

Azure naming rules differ by resource type. Resource groups can contain hyphens. Storage account names must be globally unique, 3–24 characters, lowercase letters and numbers only. Key vault names, Data Factory names, Synapse workspaces, and Databricks workspaces each have their own constraints. Microsoft's Cloud Adoption Framework recommends predictable names with components such as resource type, workload, environment, region, and instance number [17].

This book uses the following learning convention:

- **Resource group**: `rg-de-azbook-chNN-dev`, such as `rg-de-azbook-ch00-dev`.

- **Storage account**: `stdeazbkchNNxxxx`, where `xxxx` is a short unique suffix.
- **Location**: `eastus` unless the chapter or subscription requires another region.
- **Environment tag**: `environment=dev` for all book labs.
- **Chapter tag**: `chapter=chapter0`, `chapter=chapter1`, and so on.

Do not place sensitive information in names or tags. Resource names, DNS names, logs, and billing exports may be visible to operators or external systems.

Tag	Example	Purpose
`project`	`azure-book`	Groups all learning resources for reporting
`environment`	`dev`	Separates labs from production or shared sandboxes
`chapter`	`chapter0`	Enables chapter-level cost and cleanup review
`owner`	`student`	Identifies who should receive cleanup questions
`delete-after`	`2026-08-07`	Creates a human-readable cleanup expectation

Table 2: Tags make cost allocation, cleanup, and operational ownership visible.

0.7 Verify the Setup

0.7.1 Account and Subscription Check

Open a new Windows PowerShell terminal. The first verification is to prove that Azure CLI is installed, authenticated, and targeting the intended subscription.

```
az version
az login
az account show --query "{name:name, id:id, tenantId:tenantId, user:user.name}"
    --output table
```

Listing 5: Verifying the selected Azure subscription.

Expected output resembles the following. Your subscription name, ID, tenant ID, and user will differ.

```
Name                  Id                                    TenantId
                       User
--------------------  ------------------------------------
    ------------------------------------  --------------------
Azure subscription 1  00000000-0000-0000-0000-000000000000
    11111111-1111-1111-1111-111111111111  student@example.com
```

Listing 6: Expected shape of account verification output.

0.7.2 Create a Resource Group and Storage Account

The following lab creates a tagged resource group and a standard locally redundant storage account. The storage account name must be globally unique, so the script appends a short random suffix. The account uses inexpensive settings appropriate for a setup test. Chapter 1 creates a more complete ADLS Gen2 account.

```
$location = "eastus"
$rg = "rg-de-azbook-ch00-dev"
$suffix = (New-Guid).Guid.Replace("-", "").Substring(0, 6).ToLower()
$storage = "stdeazbkch00$suffix"

az group create `
  --name $rg `
  --location $location `
  --tags project=azure-book environment=dev chapter=chapter0 owner=student

az storage account create `
  --name $storage `
  --resource-group $rg `
  --location $location `
  --sku Standard_LRS `
  --kind StorageV2 `
  --https-only true `
  --min-tls-version TLS1_2 `
  --allow-blob-public-access false `
  --tags project=azure-book environment=dev chapter=chapter0 owner=student

az storage account list `
  --resource-group $rg `
  --query "[].{name:name, location:location, sku:sku.name, httpsOnly:
    enableHttpsTrafficOnly}" `
  --output table
```

Listing 7: Creating a temporary resource group and storage account.

Expected output resembles:

```
Name                 Location    Sku            HttpsOnly
-------------------  ----------  -------------  ---------
stdeazbkch00a1b2c3 eastus      Standard_LRS   True
```

Listing 8: Expected shape of storage account listing output.

If the command reports that the storage account name is unavailable, rerun the script or choose a different suffix. If it reports that the region or SKU is unavailable, change `eastus` to another supported region and record the difference.

0.7.3 Delete the Verification Resource Group

Cleanup is part of verification, not an optional afterthought. Delete the resource group and confirm that it is gone.

```
$rg = "rg-de-azbook-ch00-dev"

az group delete --name $rg --yes --no-wait

# Wait a few minutes, then verify deletion status.
az group exists --name $rg
```

Listing 9: Deleting the temporary verification resource group.

The final command returns `false` when deletion has completed. If it returns `true`, wait and run it again. Some services take several minutes to delete.

Tip

For every later chapter, save three pieces of evidence: creation command or deployment file, validation output, and cleanup output. This habit makes labs reproducible and protects

against surprise bills.

0.8 Cross-Cloud Setup Equivalence

Cross-Cloud Equivalence

Setup step	AWS	Azure	GCP
Free offer	Free Tier (12 mo + always free)	Free account ($200/30 days + always free)	Free trial ($300/90 days + Always Free)
Sign-up guard	Card + root email	Card + Microsoft account	Card + Google account
Identity model	IAM / IAM Identity Center	Microsoft Entra ID + RBAC	Cloud IAM
Admin isolation	Never use root daily	Avoid Global Admin daily	Avoid Owner; least privilege
CLI	AWS CLI v2 (`aws configure`)	Azure CLI (`az login`)	gcloud CLI (`gcloud init`)
Browser shell	AWS CloudShell	Azure Cloud Shell	Cloud Shell
Cost guardrail	AWS Budgets + alerts	Cost Management budgets	Budgets & alerts
Console	AWS Management Console	Azure Portal	Google Cloud Console

Microsoft Fabric uses a Microsoft 365/Power BI work account plus a Fabric trial capacity (no card, org tenant); Snowflake uses a 30-day/$400 trial (no card) guarded by Resource Monitors; MongoDB Atlas offers a forever-free M0 cluster (no card) secured by database users and an IP allowlist.

0.9 Environment Checklist Before Chapter 1

A complete Chapter 0 environment has the following evidence:

1. Azure free account or approved organizational sandbox subscription exists.
2. Subscription ID and tenant ID are recorded in a private lab notebook.
3. MFA is enabled for the account used to access Azure.
4. Daily lab identity is not a Global Administrator account when separation is possible.
5. Azure CLI runs from Windows PowerShell and `az account show` displays the intended subscription.
6. Optional Azure PowerShell `Az` module is installed if your organization requires PowerShell-native automation.
7. Monthly budget exists with at least one alert that reaches your email address.
8. Default region is selected, normally `eastus`, or a documented substitute is chosen.
9. Naming and tagging conventions are written down before resources are deployed.
10. Verification resource group and storage account were created, listed, and deleted successfully.

0.10 Further Reading

Start with the Azure free account overview [14], then review the Azure resource organization hierarchy [31]. For identity and access, read Microsoft Entra ID fundamentals [47] and Azure RBAC documentation [44]. For tooling, use the Azure CLI installation guide [24], Azure PowerShell installation guide [23], and Azure Cloud Shell overview [32]. For guardrails, review Microsoft Cost Management budgets [37], the Azure Well-Architected Framework [28], the Cloud Adoption Framework [29], and Azure naming guidance [17].

Key Takeaways

- The Azure free account combines introductory credit, selected 12-month free services, and always-free quotas; each has limits.
- Azure resources are easiest to manage when subscriptions, resource groups, tags, budgets, and cleanup commands are planned before deployment.
- Microsoft Entra ID provides identity, while Azure RBAC grants scoped management permissions; daily work should avoid Global Administrator and broad Owner access.
- Azure CLI is the default tool for this book on Windows PowerShell; Azure PowerShell and Azure Cloud Shell are valid alternatives.
- Budgets alert; they do not replace measurement, cleanup, and conservative service selection.
- Chapter 1 should begin only after account selection, MFA, CLI authentication, budget alerts, naming conventions, and cleanup verification are complete.

0.11 Exercises

1. **(Conceptual)** Explain the difference between a Microsoft Entra tenant, an Azure subscription, a resource group, and a resource. Give one example of each from your setup.
2. **(Conceptual)** Why is Global Administrator inappropriate for daily data engineering work? Contrast it with Contributor at resource-group scope.
3. **(Hands-on)** Install Azure CLI on Windows, run `az login`, select the book subscription, and capture `az account show -output table`.
4. **(Hands-on)** Create the Chapter 0 verification resource group and storage account. Save the command output and confirm that required tags exist.
5. **(Hands-on)** Delete the verification resource group and prove that `az group exists -name rg-de-azbook-ch00-dev` returns `false`.
6. **(Cost)** Create a monthly budget with alerts. Record the amount, thresholds, contact email, and where you will check Cost Analysis after each chapter.
7. **(Security)** Enable MFA and write a short note describing how you would recover access if your primary authenticator device is lost.
8. **(Design)** Propose a naming and tagging convention for a team of five learners sharing one subscription. Include owner, chapter, environment, and delete-after tags.
9. **(Troubleshooting)** List three signs that Azure CLI is pointed at the wrong subscription. For each, state the command you would run to confirm or fix it.

10. **(Preparation checklist)** Before starting Chapter 1, verify: account created, MFA enabled, CLI authenticated, subscription selected, budget configured, default region chosen, naming convention recorded, Chapter 0 resources deleted, and Cost Analysis reviewed.

Part I

Core Data Infrastructure and Storage

Chapter 1

Azure Data Lake Storage Gen2 and Storage Foundations

Learning Objectives

- Explain the Azure Storage account abstraction and the relationship between Blob Storage and ADLS Gen2.
- Distinguish flat object namespaces from hierarchical namespaces, including effects on analytics workloads, access control, and rename operations.
- Select an appropriate redundancy option among LRS, ZRS, GRS, GZRS, RA-GRS, and RA-GZRS for a data-engineering workload.
- Apply access tiers, lifecycle management policies, soft delete, and blob versioning to manage cost and recoverability.
- Compare online and offline migration tools, including AzCopy, Azure Storage Explorer, and Azure Data Box.
- Deploy an ADLS Gen2 account with Azure CLI and validate access using the `azure-storage-file-datalake` Python SDK.
- Design a compliant, multi-region disaster-recovery storage architecture for large-scale healthcare imaging data.

Cross-Cloud Equivalence

Although this book teaches **Azure**, the Week 1 storage foundations map almost one-to-one onto the other clouds—learn one mental model and carry it everywhere.

Capability	AWS	Azure	GCP
Object storage	Amazon S3	ADLS Gen2 / Blob	Cloud Storage
Hot tier	S3 Standard	Hot	Standard
Infrequent access	S3 Standard-IA	Cool	Nearline
Archive (instant)	Glacier Instant Retrieval	Cold	Coldline
Deep archive	Glacier Deep Archive	Archive	Archive
Lifecycle rules	Lifecycle policies	Lifecycle management	Object Lifecycle Mgmt
Versioning	S3 Versioning	Blob versioning	Object Versioning
Immutability (WORM)	Object Lock	Immutable blob storage	Bucket Lock
Cross-region copy	CRR / SRR	Object replication	Dual/multi-region + Transfer
Online transfer	DataSync	AzCopy	Storage Transfer Service
Offline transfer	Snow Family	Data Box	Transfer Appliance
Design durability	11 nines	11 nines (LRS) / 16 (GRS)	11 nines
Availability SLA	99.9% (Standard)	99.9% (RA-GRS read 99.99%)	99.95% (multi-region)
Encryption at rest	SSE-S3 / SSE-KMS / SSE-C	SSE + CMK via Key Vault	Google-managed / CMEK / CSEK
Access control	IAM & bucket policies, ACLs	Azure RBAC, SAS, POSIX ACLs	IAM, ACLs, signed URLs
Pricing levers	Storage + requests + egress + retrieval	Storage + operations + egress + retrieval	Storage + operations + egress + retrieval
Consistency	Strong read-after-write	Strong read-after-write	Strong (global)

Beyond raw object storage, the same layer reappears as a managed abstraction: **Fabric OneLake** is a tenant-wide lake built on ADLS Gen2 (Delta–Parquet), **Snowflake** persists data as immutable micro-partitions on the provider's object store, and **MongoDB**'s WiredTiger is a node-local engine rather than object storage—the contrast is itself instructive.

Key Terms

Storage account: the Azure management boundary for storage services, networking, redundancy, encryption, identity integration, and billing. **Container**: a Blob Storage namespace that becomes a file system when hierarchical namespace is enabled. **Hierarchical namespace**: ADLS Gen2 capability that provides directory semantics, atomic directory operations, and POSIX-like ACLs. **Access tier**: a cost and retrieval-latency class for blob data. **Lifecycle policy**: JSON rules that transition or delete blobs based on age, tier, prefix, and metadata.

1.1 Week 1 Overview: Storage as the Root of the Azure Data Platform

Azure data engineering begins with storage, not with Spark clusters, orchestration pipelines, dashboards, or machine-learning models. Storage is where raw evidence lands, where curated data products persist, where batch and streaming systems checkpoint state, and where audit trails prove what happened. If the storage layer is poorly named, weakly secured, underprotected, or expensive to operate, every downstream service inherits that fragility. Azure Data Lake Storage Gen2 (ADLS Gen2) is Microsoft Azure's primary cloud object store for analytical

data lakes and lakehouses [27].

A storage foundation chapter is therefore a design chapter, not merely a catalog of buttons in the portal. A senior data engineer must connect data characteristics to platform controls: object size, arrival rate, regulatory retention, locality, recovery-point objective (RPO), recovery-time objective (RTO), analytics engine behavior, identity boundaries, and expected access pattern. The same storage account can host a quiet archive, a high-throughput ingestion zone, or a governed lakehouse serving hundreds of analysts; those workloads need different tiering, redundancy, namespace, and access decisions.

Definition

ADLS Gen2 is the combination of Azure Blob Storage durability and scale with a hierarchical namespace designed for big-data analytics. It supports file-system semantics, directory-level access control, and APIs used by Hadoop-compatible engines such as Spark, Synapse, and Databricks.

Note

Throughout this chapter, "Blob Storage" refers to Azure's object storage service, while "ADLS Gen2" refers to Blob Storage accounts or containers with hierarchical namespace enabled. The same underlying platform is involved; the difference is the namespace and analytics-oriented semantics.

1.2 Monday: Azure Storage Account Architecture

1.2.1 The Storage Account as a Control Boundary

An Azure Storage account is both a management-plane resource and a data-plane endpoint. The management plane is controlled through Azure Resource Manager (ARM): resource group, location, tags, redundancy, networking rules, private endpoints, diagnostic settings, encryption settings, and identity-related configuration. The data plane is where applications create containers, upload blobs, list directories, acquire leases, read ranges, and set object metadata. Confusing those planes leads to common troubleshooting mistakes: a user may have Owner rights over the Azure resource but still be unable to read data-plane objects, or may have blob data permissions but be unable to alter firewall rules.

Storage accounts expose multiple service endpoints: Blob, Data Lake, Queue, Table, and File services. A data engineering lake normally depends on Blob and Data Lake endpoints. The Blob endpoint exposes flat object APIs. The Data Lake endpoint exposes file-system semantics for hierarchical namespace accounts. Many engines use ABFS or ABFSS URI forms such as `abfss://bronze@acct.dfs.core.windows.net/path/`. The `dfs.core.windows.net` endpoint is significant because it indicates the Data Lake namespace endpoint rather than the blob endpoint.

Definition

A **container** is a grouping of blobs inside a storage account. In an ADLS Gen2 account, a container is also presented as a **file system**. Data lake designs often map containers to security and lifecycle boundaries such as `raw`, `bronze`, `silver`, `gold`, or domain-specific zones.

Tip

Use separate storage accounts when you need hard isolation for networking, encryption keys, redundancy, account-level immutability, or blast-radius reduction. Use separate containers when the data shares account-level controls but needs different access, prefixes, or lifecycle rules.

1.2.2 Blob Storage vs. ADLS Gen2

Azure Blob Storage is optimized for massive, durable, low-cost object storage [26]. Objects are addressed by name, metadata, and container. Traditional object stores treat the slash in `raw/year=2026/month=07/file.parquet` as part of the object name, not as a real directory tree. Tools can simulate folders by filtering names with common prefixes, but operations such as renaming a directory become many object copy-and-delete operations.

ADLS Gen2 adds a hierarchical namespace. Directories become first-class metadata entries. Directory rename and delete operations can be atomic metadata operations instead of large object rewrites. POSIX-like access control lists can be attached at directory and file levels. This matters for analytics engines that produce many files and commit directory-level outputs. It also matters for governance: a lake can grant a data science group read access to `/curated/imaging/features/` without granting access to raw protected health information.

Capability	Blob Storage flat namespace	ADLS Gen2 hierarchical namespace
Namespace model	Object names with prefixes	File systems, directories, and files
Rename directory	Copy and delete many objects	Metadata operation for directory path
Access control	Container and blob RBAC or SAS patterns	RBAC plus POSIX-like ACLs at path level
Analytics engines	Works for object reads and writes	Designed for Hadoop, Spark, Synapse, Databricks
Best fit	General object storage, backups, media	Enterprise data lakes and lakehouses

Table 1.1: Flat Blob Storage and ADLS Gen2 share durability but differ in namespace behavior and analytics ergonomics.

Common Pitfall

Retrofitting hierarchy too late. Hierarchical namespace is a foundational account capability. Treat it as an up-front architecture decision for analytical lakes rather than a cosmetic folder setting.

1.2.3 Redundancy Options: LRS, ZRS, GRS, GZRS, and RA Variants

Azure Storage redundancy determines how many copies exist and where those copies reside [9]. It is a durability and availability design choice, not a substitute for backups, versioning, immutability, or application-level disaster recovery. The most common options are:

- **Locally redundant storage (LRS):** keeps three copies within a single data center in the primary region. It is cost-effective but vulnerable to zone or regional outages.
- **Zone-redundant storage (ZRS):** synchronously replicates across availability zones in the primary region. It improves availability for zone failures.

- **Geo-redundant storage (GRS):** replicates from the primary region to a paired secondary region. It protects against regional disaster but secondary access is controlled by failover behavior.
- **Geo-zone-redundant storage (GZRS):** combines ZRS in the primary region with asynchronous geo-replication to a secondary region.
- **Read-access geo-redundant storage (RA-GRS) and RA-GZRS:** expose read-only access to secondary replicas before failover, useful for read-only DR validation and emergency analytics.

Note

Asynchronous geo-replication means the secondary region may lag. For regulated workloads, document the expected RPO and validate whether the lag is acceptable for each data domain.

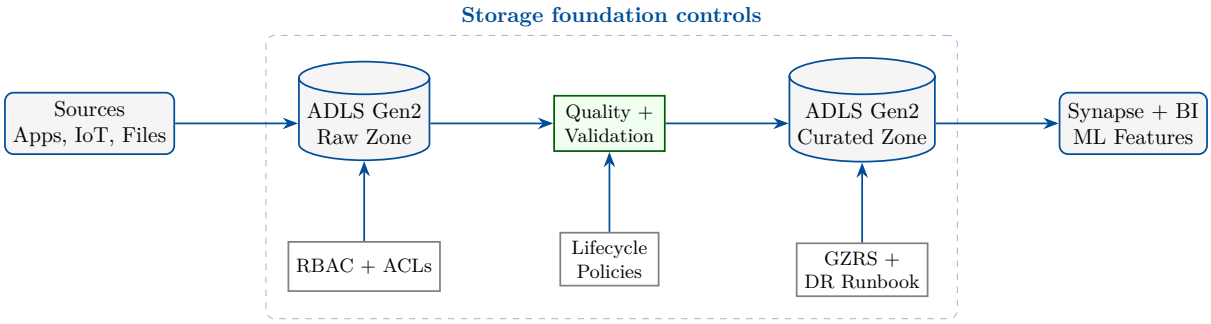


Figure 1.1: ADLS Gen2 as the durable foundation for ingestion, validation, curation, serving, governance, lifecycle control, and disaster recovery.

1.2.4 Monday Design Checklist

A storage account review should answer five questions. First, what data domains and environments will the account hold? Second, what identities need management-plane access versus data-plane access? Third, is hierarchical namespace required for analytics engines and path-level ACLs? Fourth, what regional failure mode is in scope for this workload? Fifth, what service endpoints must be private, public, or disabled? These questions are mundane, but they prevent expensive migration later.

1.3 Tuesday: Access Tiers, Lifecycle Management, and Recoverability

1.3.1 Access Tiers: Hot, Cool, Cold, and Archive

Azure Blob access tiers trade storage cost against access cost and retrieval latency [3]. The **Hot** tier is designed for frequently accessed data. **Cool** is cheaper to store but more expensive to read and generally appropriate for data accessed infrequently but still online. **Cold** extends that cost profile for very infrequent online access where supported. **Archive** is the lowest-cost offline tier, but data must be rehydrated before it can be read.

Definition

An **access tier** is a blob-level or account-default cost class that expresses how often data is expected to be read. It changes storage price, read price, minimum retention expectations, and retrieval latency.

The important design principle is that tiering should follow observed or contractually expected access patterns. Raw landing files may be hot for a few days while quality rules are fixed, cool for a month while reports are reconciled, and archived after regulatory retention begins. Curated dimensional data may stay hot because every dashboard queries it. Training snapshots may be cool or archive once a model version is approved. Tiering is therefore a data-product decision, not only an infrastructure setting.

Tier	Typical use	Access behavior	Caution
Hot	Active ingestion, current marts, daily dashboards	Frequent read/write	Higher storage cost
Cool	Recently closed periods, audit extracts, occasional backfills	Infrequent reads	Higher read and early deletion costs
Cold	Rarely accessed online data	Very infrequent reads	Verify regional and feature support
Archive	Long-term retention, legal archive, deep history	Offline until rehydrated	Rehydration delay and operational runbook required

Table 1.2: Access tiers should be selected by data-product usage, not by a single account-wide habit.

Common Pitfall

Archiving without a retrieval plan. Archive tier can save money, but an urgent investigation may wait hours for rehydration. Define who can rehydrate, where restored data lands, and which compliance approvals are required.

1.3.2 Lifecycle Management Policies

Lifecycle management is Azure's rule engine for moving or deleting blobs based on conditions such as age, prefix, tier, and version state [30]. A lifecycle policy is JSON attached to the storage account. In data lakes, lifecycle rules should align with data classifications and medallion zones. For example, raw X-ray DICOM objects may transition to Cool after 30 days and Archive after 180 days, while curated ML feature tables remain Hot because they are part of active training pipelines.

Tip

Start lifecycle policies in report-only or narrow-prefix form when possible. Apply the first rule to a non-critical prefix, validate cost and retrieval behavior, then widen the policy. The most expensive lifecycle mistake is deleting or archiving the wrong prefix.

Lifecycle is not a governance substitute. It does not classify data, prove legal retention, or replace immutable storage. It should be one control in a broader retention design: metadata classification, legal hold requirements, data owner approval, backup or versioning strategy, observability, and exception handling.

1.3.3 Soft Delete and Blob Versioning

Soft delete protects against accidental deletion by retaining deleted blobs or containers for a configured period. Blob soft delete covers individual blobs. Container soft delete protects deleted containers. Blob versioning keeps prior versions when an overwrite occurs. Together, these controls reduce the blast radius of human error, failed jobs, and accidental overwrites.

Note

Soft delete is time-bounded recoverability, not a full backup strategy. If a malicious process overwrites sensitive data and waits until retention expires, soft delete alone is insufficient. Combine it with versioning, immutable policies where required, monitoring, and least-privilege access.

A data-engineering example clarifies the distinction. Suppose an ADF pipeline writes a corrected file over `/raw/claims/2026/07/30/batch.csv`. With blob versioning enabled, the previous object can remain available as an older version. If the pipeline deletes the file entirely, soft delete can restore it during the retention window. If the entire `raw` container is deleted, container soft delete can help. Each feature covers a different failure mode.

1.3.4 Tuesday Design Checklist

For each data zone, document freshness, read frequency, minimum retention, deletion approval, recovery expectation, legal hold requirements, and expected reprocessing cost. Then map those properties to access tier, lifecycle policy, soft delete period, versioning, immutability, and monitoring. If the table is unknown, do not guess: instrument actual access before enforcing aggressive tiering.

1.4 Wednesday: Migration Tools and Trade-offs

1.4.1 Online Migration with AzCopy

AzCopy is a command-line tool for high-performance copy operations to and from Azure Storage [18]. It supports local-to-blob, blob-to-blob, synchronization, include and exclude patterns, SAS or identity-based authentication, and job resume. It is ideal for repeatable online migrations, daily bulk loads, backfills from on-premises shares, and automation scripts.

Online migration is constrained by network bandwidth, change rate, reliability, and security. A 50 TB migration over a fast enterprise link may be practical. A 10 PB imaging migration over the public internet is usually not. The engineer must estimate transfer duration, throttling, retry behavior, validation strategy, and cutover mechanics. Hash validation, manifest files, and reconciliation queries matter as much as the copy command.

Tip

For large online migrations, split data by stable prefixes such as date, modality, business unit, or source system. Parallel jobs are easier to resume and reconcile when each job owns a deterministic prefix.

1.4.2 Azure Storage Explorer

Azure Storage Explorer is a graphical tool for inspecting and managing storage accounts. It is excellent for learning, debugging access, confirming that objects landed under the expected prefix, generating SAS tokens in controlled scenarios, and performing small manual operations.

It is not a substitute for infrastructure as code, automated migration scripts, or governed production operations.

Common Pitfall

Manual production edits through a GUI. Storage Explorer is useful, but manual deletes, uploads, and tier changes are hard to review and reproduce. Production storage changes should normally flow through scripted, logged, least-privilege paths.

1.4.3 Offline Migration with Azure Data Box

Azure Data Box is an offline transfer appliance for moving large datasets into Azure when network transfer is impractical [41]. Microsoft ships a ruggedized device, the customer copies data locally, ships it back, and Microsoft imports the encrypted data into the target storage account. This pattern is common when datasets are measured in hundreds of terabytes or petabytes, when a data center exit has a deadline, or when network egress is too slow.

Offline transfer introduces different constraints: device capacity, shipping logistics, chain of custody, encryption keys, staging procedures, copy windows, import verification, and delta synchronization. It rarely eliminates the need for online tools. A common pattern is to seed historical data with Data Box and then use AzCopy, ADF, or custom ingestion to synchronize changed data until cutover.

Tool	Best use	Trade-off
AzCopy	Scripted online copy, recurring loads, prefix-level sync	Depends on bandwidth and identity/SAS hygiene
Storage Explorer	Inspection, troubleshooting, small manual operations	Not ideal for reproducible production change
Azure Data Box	Large offline seeding and datacenter migration	Requires logistics, chain of custody, and later delta sync

Table 1.3: Migration tools differ by scale, automation, observability, and operational risk.

1.5 Thursday Hands-on Project: Deploy ADLS Gen2 with Lifecycle Management

This lab deploys a storage account with hierarchical namespace enabled, creates a file system, enables recoverability controls, and applies a lifecycle policy that transitions matching blobs to Archive after 30 days. The commands assume Azure CLI is installed and that you are authenticated with sufficient permissions in a non-production subscription.

Note

The examples use globally unique storage account names. Replace `stdeadlsweek1demo` with a compliant name that is unique across Azure: 3–24 lowercase letters and numbers, no hyphens.

```
az login
az account set --subscription "00000000-0000-0000-0000-000000000000"

$rg = "rg-de-azure-book-week1"
$location = "eastus"
$account = "stdeweek1adlsdemo01"

az group create `
```

```
  --name $rg `
  --location $location

az storage account create `
  --name $account `
  --resource-group $rg `
  --location $location `
  --sku Standard_GZRS `
  --kind StorageV2 `
  --hierarchical-namespace true `
  --min-tls-version TLS1_2 `
  --https-only true `
  --allow-blob-public-access false `
  --default-action Deny

az storage account blob-service-properties update `
  --account-name $account `
  --resource-group $rg `
  --enable-delete-retention true `
  --delete-retention-days 30 `
  --enable-container-delete-retention true `
  --container-delete-retention-days 30 `
  --enable-versioning true

az storage fs create `
  --account-name $account `
  --name raw `
  --auth-mode login

az storage fs create `
  --account-name $account `
  --name curated `
  --auth-mode login
```

Listing 1.1: Deploying an ADLS Gen2 account and basic lake file systems with Azure CLI.

The account uses `Standard_GZRS`, hierarchical namespace, HTTPS-only access, TLS 1.2 minimum, and disabled public blob access. The example also sets the account firewall default action to deny. In a real lab you would add a trusted virtual network, private endpoint, or temporary client IP rule before attempting data-plane operations. If you cannot create file systems after setting `-default-action Deny`, confirm that your network path is allowed.

```
{
  "rules": [
    {
      "enabled": true,
      "name": "raw-to-archive-after-30-days",
      "type": "Lifecycle",
      "definition": {
        "filters": {
          "blobTypes": ["blockBlob"],
          "prefixMatch": ["raw/landing/"]
        },
        "actions": {
          "baseBlob": {
            "tierToArchive": {
              "daysAfterModificationGreaterThan": 30
            }
          }
        }
      }
    }
  ]
```

```
}
```

Listing 1.2: Lifecycle policy that archives raw objects after 30 days.

Save the JSON as `lifecycle-policy.json` in the book folder or pipeline workspace, then apply it with Azure CLI:

```
az storage account management-policy create `
  --account-name $account `
  --resource-group $rg `
  --policy .\lifecycle-policy.json

az storage account management-policy show `
  --account-name $account `
  --resource-group $rg
```

Listing 1.3: Applying the lifecycle management policy.

Common Pitfall

Prefix mismatch in lifecycle policies. In ADLS Gen2, the policy prefix is evaluated against the blob path from the container boundary. If the intended container is `raw` and the intended directory is `landing`, validate the exact prefix format in a non-production account before rollout.

The Python SDK snippet below writes a small file through the Data Lake endpoint. In production, prefer managed identity from Azure compute. For a local lab, `DefaultAzureCredential` may use Azure CLI authentication.

```
from azure.identity import DefaultAzureCredential
from azure.storage.filedatalake import DataLakeServiceClient

account_name = "stdeweek1adlsdemo01"
account_url = f"https://{account_name}.dfs.core.windows.net"
credential = DefaultAzureCredential()

service_client = DataLakeServiceClient(
    account_url=account_url,
    credential=credential,
)

file_system = service_client.get_file_system_client("raw")
directory = file_system.get_directory_client("landing/week1")
directory.create_directory()

file_client = directory.create_file("sample-observation.txt")
payload = b"patient_id,event_time,source\n1001,2026-07-30T21:33:37Z,xray\n"
file_client.append_data(data=payload, offset=0, length=len(payload))
file_client.flush_data(len(payload))

read_back = file_client.download_file().readall().decode("utf-8")
print(read_back)
```

Listing 1.4: Writing and reading a file with azure-storage-file-datalake.

Tip

After a lab, delete the resource group if the account is not needed. GZRS storage and retained versions can create continuing charges, especially if you upload test data and lifecycle policies move it to tiers with early-deletion considerations.

1.5.1 Validation Steps

A complete lab validation includes the following checks: the storage account shows hierarchical namespace enabled; the account redundancy is GZRS; public blob access is disabled; soft delete and versioning are enabled; the `raw` and `curated` file systems exist; the lifecycle policy is visible; and the Python snippet can write and read a file through the DFS endpoint. Capture command output or screenshots for your engineering notebook.

1.6 Friday Use Case and Architecture: Healthcare X-ray Lake at 10 PB

1.6.1 Scenario

A healthcare provider processes 10 PB of X-ray images across regional hospitals. The platform must ingest new images daily, retain raw images for compliance and research, support downstream AI model development, serve governed analytics, and survive regional outages. The organization is subject to HIPAA and HITRUST-aligned controls, expects encryption at rest and in transit, requires auditability, and must control storage growth.

The first architectural decision is to separate data zones and duties. Raw images are immutable evidence and should be protected from accidental overwrite. Curated derivatives such as thumbnails, de-identified metadata, and feature vectors have different access and lifecycle needs. Research sandboxes may use derived, de-identified data and should not have broad access to protected health information. Operations teams need monitoring and recovery runbooks but not unrestricted patient data access.

Definition

A **compliant disaster-recovery storage architecture** defines not only where replicated bytes reside, but also who can read them, how failover is authorized, what data loss is acceptable, how recovery is tested, and how audit evidence is retained.

1.6.2 Target Architecture

The proposed design uses ADLS Gen2 storage accounts in a primary Azure region with GZRS or RA-GZRS where available and appropriate. Private endpoints keep data traffic on private networks. Microsoft Entra ID, Azure RBAC, POSIX-like ACLs, and managed identities enforce least privilege. Customer-managed keys may be used when the compliance program requires key ownership and rotation evidence. Diagnostic logs flow to a security workspace. Lifecycle policies move older raw images to Cool, Cold, or Archive based on clinical and research retrieval requirements. Immutable storage policies or legal holds are considered for records that must not be altered.

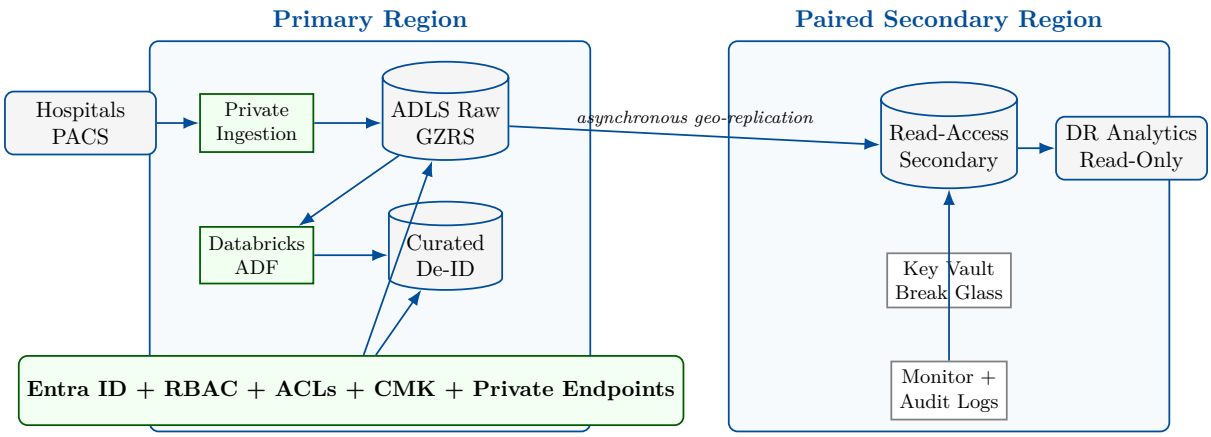


Figure 1.2: Multi-region ADLS Gen2 architecture for compliant healthcare imaging with GZRS/RA-GZRS, private ingestion, governed processing, and read-access disaster recovery.

1.6.3 Compliance and Security Design

HIPAA and HITRUST programs require administrative, technical, and physical controls; Azure services provide building blocks, but the customer owns configuration, access governance, monitoring, and operating procedures [20]. For storage, key technical controls include encryption at rest, TLS in transit, private networking, disabled public access, managed identities, least-privilege RBAC, ACL inheritance, diagnostic logging, retention controls, and incident-response evidence. Key administrative controls include data classification, role approval, periodic access reviews, backup and restore tests, and documented RPO/RTO acceptance.

Note

Compliance is not achieved by choosing a single service. It is achieved by configuring services, operating them consistently, collecting evidence, and proving that controls work under failure and audit conditions.

1.6.4 Lifecycle and Cost Model for 10 PB

At 10 PB, lifecycle mistakes are material. Even small per-GB price differences become large monthly numbers. The architecture should classify images by age, clinical status, research value, and legal hold. Recent images may remain Hot for active care and model validation. Images older than 30 or 90 days may move to Cool or Cold if clinical retrieval is rare. Deep history may move to Archive if rehydration times meet policy. De-identified derivatives used for model training may remain online because recomputing them from raw archive could be slower and more expensive than storing them.

Cost control also requires object sizing and file-format decisions. Medical images are naturally large binary objects, but metadata and derived features should be stored in analytics-friendly formats such as Parquet and Delta Lake. The lakehouse literature emphasizes unifying low-cost object storage with reliable table abstractions and analytics engines [2, 1]. For the X-ray platform, raw DICOM objects and curated Delta tables serve different purposes and deserve different lifecycle policies.

1.6.5 Disaster Recovery Runbook

A DR design is incomplete without a runbook. The runbook should specify monitoring signals for regional outage, the decision authority for failover, how to assess replication lag, which applications can operate read-only from the secondary, when to pause writes, how to communicate expected data loss, and how to return to the primary region. For RA-GZRS, read-only secondary access can support emergency analytics and validation before account failover. For write recovery, applications must be designed to avoid split-brain writes and to reconcile any lag after failback.

Common Pitfall

Assuming geo-replication equals application recovery. Storage replication preserves data copies, but applications, identities, private endpoints, keys, DNS, orchestration jobs, and user access paths must also be recoverable.

1.7 Architectural Decision Record Template

Use a short architectural decision record (ADR) for storage account decisions. A useful ADR includes: context, decision, alternatives considered, consequences, security impact, cost impact, operational runbook, and validation evidence. For Chapter 1, write an ADR that states why hierarchical namespace is enabled, why GZRS or RA-GZRS is selected, what lifecycle rules apply to each prefix, and what recovery tests must pass before production.

1.8 Operational Deep Dive: Naming, Zones, and Evidence

A production lake is easier to operate when naming, zones, and evidence are standardized from the first week. Storage account names are globally unique and should encode organization, environment, region, workload, and purpose without exposing sensitive details. Container names should be stable because they appear in URIs, ACL rules, pipeline parameters, and documentation. Directory names should support partition pruning and operational ownership. A common pattern is `/source-system/entity/year=YYYY/month=MM/day=DD/`, but event-time and processing-time partitions must not be confused.

Tip

Prefer boring names that operations staff can predict. A clever naming convention that requires a lookup table will fail during an incident.

Data zones should express quality and access expectations. The raw zone preserves source fidelity. The bronze zone may add ingestion metadata and schema capture. The silver zone standardizes, deduplicates, and validates. The gold zone serves business-ready products. Not every organization uses these exact labels, but every organization needs an explicit vocabulary for trust level. Without it, users will query whatever path is easiest to find.

Evidence completes the operating model. For every storage account, retain deployment records, policy JSON, access reviews, diagnostic settings, lifecycle rules, and DR test results. These artifacts are useful during audits, but they are also useful during ordinary engineering work. When a pipeline fails after a storage firewall change, a recorded change history is faster than guessing.

1.9 Troubleshooting Patterns

The most common ADLS Gen2 failures appear as authorization errors, network errors, path errors, or lifecycle surprises. Authorization errors require checking both RBAC and ACLs. A managed identity may have `Storage Blob Data Contributor` at the account level but still lack execute permission on an intermediate directory. Network errors require checking public network access, firewall rules, private endpoint DNS, and whether the client is using the blob or DFS endpoint. Path errors often come from case sensitivity, unexpected URL encoding, or prefix assumptions inherited from flat object storage.

Lifecycle surprises are harder because they may appear days later. A report fails because a file was archived; a backfill becomes slow because historical data moved to Cool; an overwrite creates many versions and increases cost. The remedy is observability: storage metrics, inventory reports, policy reviews, and cost analysis grouped by account, container, prefix, tier, and tag.

Common Pitfall

Granting broad permissions to solve a narrow error. If a pipeline cannot read one path, do not immediately grant account-wide contributor rights. Identify whether the missing permission is RBAC, ACL execute, ACL read, firewall, private DNS, or an incorrect endpoint.

1.10 Week 1 Lab Rubric

A complete Week 1 submission should include: a diagram of the proposed storage account and containers; Azure CLI output or screenshots proving the account settings; lifecycle policy JSON; a short Python SDK run log; an ADR explaining redundancy and tiering; and a cleanup note showing whether the resource group was deleted. Grade the work on correctness, reproducibility, security posture, and operational reasoning. A beautiful diagram without commands is incomplete; commands without architecture reasoning are also incomplete.

Criterion	Meets expectation	Needs improvement
Reproducibility	CLI commands create the described account and controls	Manual portal-only steps or missing parameters
Security	Public access disabled, TLS enforced, identity-based access preferred	Shared keys or broad permissions used without reason
Lifecycle	Policy targets a specific prefix and retention logic is explained	Policy is account-wide or prefix is unvalidated
Recovery	Soft delete, versioning, redundancy, and DR assumptions documented	Replication is assumed to solve every recovery scenario
Cost	Tiering and cleanup are discussed	GZRS and versions left running without cost awareness

Table 1.4: Week 1 rubric for storage foundation deliverables.

1.11 Interview Q&A

1.11.1 Concepts and Trade-offs

1. **Q: Why is ADLS Gen2 preferred for analytics lakes over a flat Blob Storage namespace?**

A: It adds hierarchical namespace, atomic directory operations, and path-level ACLs while retaining Blob Storage scale and durability.

2. **Q: When would you choose ZRS instead of GZRS?**
   A: ZRS is appropriate when zone failure is in scope but cross-region replication cost or data-residency constraints make geo-replication unsuitable.
3. **Q: Why might Archive tier be wrong for data that is rarely read?**
   A: Rare reads may still be urgent. Archive requires rehydration, so RTO and operational approval can make Cool or Cold a better fit.
4. **Q: What does soft delete protect that versioning does not?**
   A: Soft delete protects deleted blobs or containers during a retention window; versioning primarily protects prior object states after overwrite.

1.12 Further Reading

Begin with Microsoft Learn's introduction to ADLS Gen2 [27] and Blob Storage [26]. Review Azure Storage redundancy for regional design [9], access tiers [3], and lifecycle management [30]. For migration planning, read the Azure Data Box overview [41] and AzCopy documentation [18]. Use the Azure Well-Architected Framework [28] and Cloud Adoption Framework [29] to connect technical choices to reliability, security, operations, and cost governance. For lakehouse table-storage context, see Delta Lake and lakehouse architecture papers [1, 2].

Key Takeaways

- Azure Storage accounts are management, security, networking, redundancy, and billing boundaries for data workloads.
- ADLS Gen2 combines Blob Storage durability with hierarchical namespace and path-level controls designed for analytics.
- Redundancy choices express failure assumptions and RPO/RTO trade-offs; they do not replace backups or DR runbooks.
- Access tiers and lifecycle policies should follow data-product usage, retention, and retrieval requirements.
- Soft delete, container soft delete, and blob versioning address different accidental-deletion and overwrite scenarios.
- AzCopy, Storage Explorer, and Azure Data Box solve different migration problems; large programs often combine offline seeding with online delta sync.
- Healthcare-scale storage architecture requires private networking, least privilege, encryption, auditability, lifecycle economics, and tested recovery procedures.

1.13 Exercises

1. **(Conceptual)** Explain the difference between the management plane and data plane for an Azure Storage account. Give one permission from each category.
2. **(Conceptual)** Compare LRS, ZRS, GRS, and GZRS for a lake that supports daily regulatory reports. State which outage each option can tolerate.

3. **(Conceptual)** A dashboard reads a curated Delta table every 15 minutes. Should that table be Hot, Cool, Cold, or Archive? Justify your answer using latency and cost.
4. **(Hands-on)** Run the Azure CLI lab in a sandbox subscription. Capture evidence that hierarchical namespace, soft delete, versioning, and the lifecycle policy are enabled.
5. **(Hands-on)** Modify the lifecycle policy so that objects under `raw/landing/` move to Cool after 7 days and Archive after 90 days. Explain how you would test the policy safely.
6. **(Hands-on)** Use the Python SDK to create directories for `bronze`, `silver`, and `gold`. Upload a small file to each and list paths recursively.
7. **(Design)** Choose a migration tool for 500 TB of historical files plus 2 TB of daily changes. State whether you would use AzCopy, Data Box, both, or another service, and explain cutover.
8. **(Design)** Write an ADR for the healthcare X-ray architecture. Include redundancy, lifecycle, encryption, access model, monitoring, and DR validation.
9. **(Design)** Identify three ways a lifecycle policy could harm production data and specify a guardrail for each.
10. **(Interview)** Prepare a two-minute explanation of why geo-redundant storage is necessary but insufficient for disaster recovery.

1.14 Capstone Project: A Compliant Medical Imaging Archive on ADLS Gen2

Capstone Project

Mission: Design and prototype the storage foundation for a compliant medical imaging archive on Azure Data Lake Storage Gen2. Your solution must show how a healthcare provider can ingest, protect, tier, replicate, and audit X-ray images while meeting security, disaster-recovery, and cost objectives.
Estimated time: 4–6 hours.
What you will practice:

- Choosing ADLS Gen2 over a flat Blob Storage namespace when analytics, directory operations, and path-level controls are required.
- Enabling hierarchical namespace, GZRS redundancy, blob versioning, blob and container soft delete, lifecycle tiering, and immutable storage.
- Designing a Hot-to-Cool-to-Archive lifecycle for a 10 PB healthcare archive.
- Planning object replication to a secondary region and documenting RPO, RTO, and failover assumptions.
- Applying encryption with customer-managed keys, least-privilege Azure RBAC, and operational evidence for HIPAA and HITRUST-aligned controls.

Scenario

A regional healthcare provider is consolidating 10 PB of X-ray images from hospital picture archiving and communication systems into Azure. The images are clinically important, legally sensitive, expensive to store, and valuable for downstream analytics. Radiologists and care teams need recent images online. Compliance officers require retention evidence and tamper resistance. Data scientists need governed access to de-identified derivatives for model devel-

opment. Operations leaders require a disaster-recovery design that survives a regional outage without pretending that storage replication alone is a complete application recovery plan.

The archive must support HIPAA and HITRUST-aligned operating practices. Azure supplies storage, identity, encryption, logging, and replication capabilities, but the provider remains responsible for configuration, access governance, incident response, and audit evidence [20]. The design must also be cost-aware: at 10 PB, an incorrect tiering rule, overly broad version retention, or accidental hot-tier default can become a large recurring expense. Treat this capstone as an architecture exercise and a small implementation prototype, not as a full production rollout.

Requirements

Your target design must satisfy the following requirements.

- **Storage foundation:** Use ADLS Gen2 with hierarchical namespace enabled so file systems, directories, atomic directory operations, and path-level ACLs support analytics workloads [27].
- **Redundancy:** Select `Standard_GZRS` for the primary storage account so data is zone-redundant in the primary region and geo-replicated to a paired secondary region [9].
- **Lifecycle:** Move raw images from Hot to Cool after 30 days and to Archive after 180 days, with a separate rule for previous versions so retained versions do not grow without review [3, 30].
- **Recoverability:** Enable blob versioning, blob soft delete, and container soft delete. Document restore procedures for accidental overwrite, accidental blob deletion, and accidental container deletion.
- **WORM retention:** Configure immutable storage with a time-based retention policy and legal hold capability for records that must be preserved in write-once, read-many form.
- **Replication:** Configure object replication from the primary account to a secondary-region account or document the exact policy that would be applied in production.
- **Encryption:** Use encryption at rest with customer-managed keys stored in Azure Key Vault. Document key rotation ownership, break-glass access, and monitoring.
- **Access control:** Enforce least privilege with Microsoft Entra ID, Azure RBAC, managed identities, and ADLS Gen2 ACLs. Avoid shared keys and account-wide contributor assignments unless a documented exception exists [44].

Reference Architecture

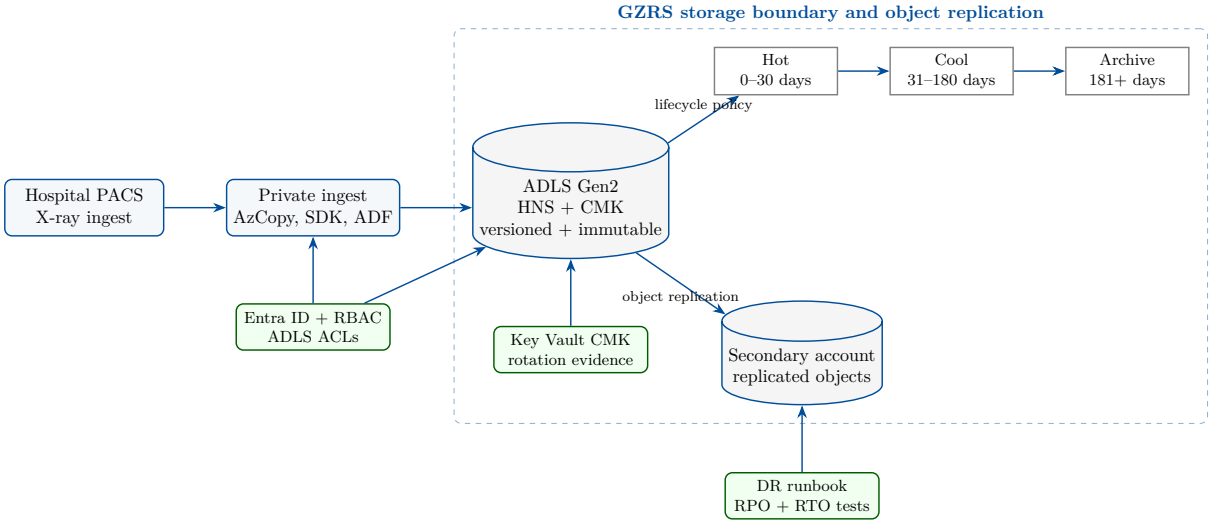


Figure 1.3: Capstone reference architecture for a compliant medical imaging archive on ADLS Gen2 with hierarchical namespace, customer-managed keys, lifecycle tiering, immutable retention, GZRS, and object replication.

Implementation Steps

Use a sandbox subscription and unique names. The commands below are written for Windows PowerShell and Azure CLI. Replace placeholder subscription, tenant, object IDs, locations, and account names before running.

1. **Create the secured ADLS Gen2 accounts.** Deploy a primary GZRS account with hierarchical namespace enabled, a secondary account in another region, and baseline recoverability controls.

```
az login
az account set --subscription "00000000-0000-0000-0000-000000000000"

$rg = "rg-medimg-capstone"
$primaryLocation = "eastus"
$secondaryLocation = "westus"
$primary = "stmedimgprimary001"
$secondary = "stmedimgsecondary001"
$keyVault = "kv-medimg-capstone-001"
$keyName = "cmk-medimg-archive"
$container = "xray-raw"

az group create `
  --name $rg `
  --location $primaryLocation

az keyvault create `
  --name $keyVault `
  --resource-group $rg `
  --location $primaryLocation `
  --enable-rbac-authorization true

az keyvault key create `
  --vault-name $keyVault `
  --name $keyName `
```

```
  --kty RSA `
  --size 3072

az storage account create `
  --name $primary `
  --resource-group $rg `
  --location $primaryLocation `
  --sku Standard_GZRS `
  --kind StorageV2 `
  --hierarchical-namespace true `
  --min-tls-version TLS1_2 `
  --https-only true `
  --allow-blob-public-access false `
  --default-action Deny `
  --assign-identity

az storage account create `
  --name $secondary `
  --resource-group $rg `
  --location $secondaryLocation `
  --sku Standard_LRS `
  --kind StorageV2 `
  --hierarchical-namespace true `
  --min-tls-version TLS1_2 `
  --https-only true `
  --allow-blob-public-access false `
  --default-action Deny `
  --assign-identity

az storage account blob-service-properties update `
  --account-name $primary `
  --resource-group $rg `
  --enable-versioning true `
  --enable-delete-retention true `
  --delete-retention-days 30 `
  --enable-container-delete-retention true `
  --container-delete-retention-days 30

az storage fs create `
  --account-name $primary `
  --name $container `
  --auth-mode login

az storage fs create `
  --account-name $secondary `
  --name $container `
  --auth-mode login
```

Listing 1.5: PowerShell plus Azure CLI for the primary ADLS Gen2 account.

2. **Attach customer-managed keys and least-privilege roles.** Grant the storage account managed identity access to the Key Vault key, set the account encryption key source to Microsoft Key Vault, and assign data-plane roles only to ingestion and operations identities. In production, use groups or managed identities rather than individual users.

3. **Apply lifecycle management.** Save the following as `capstone-lifecycle.json`. Review the prefix carefully before applying it to production.

```
{
  "rules": [
    {
      "enabled": true,
      "name": "xray-hot-cool-archive",
```

```
      "type": "Lifecycle",
      "definition": {
        "filters": {
          "blobTypes": ["blockBlob"],
          "prefixMatch": ["xray-raw/dicom/"]
        },
        "actions": {
          "baseBlob": {
            "tierToCool": {
              "daysAfterModificationGreaterThan": 30
            },
            "tierToArchive": {
              "daysAfterModificationGreaterThan": 180
            }
          },
          "version": {
            "tierToCool": {
              "daysAfterCreationGreaterThan": 30
            },
            "delete": {
              "daysAfterCreationGreaterThan": 365
            }
          }
        }
      }
    ]
}
```

Listing 1.6: Lifecycle policy for raw X-ray images and older versions.

```
az storage account management-policy create `
  --account-name $primary `
  --resource-group $rg `
  --policy .\capstone-lifecycle.json

az storage container immutability-policy create `
  --account-name $primary `
  --container-name $container `
  --period 2555 `
  --allow-protected-append-writes true `
  --auth-mode login

az storage container legal-hold set `
  --account-name $primary `
  --container-name $container `
  --tags "regulatory-review" `
  --auth-mode login

az storage account or-policy create `
  --resource-group $rg `
  --account-name $secondary `
  --source-account $primary `
  --destination-account $secondary `
  --rules .\object-replication-rules.json
```

Listing 1.7: Applying lifecycle, immutable retention, legal hold, and object replication.

4. **Ingest and verify a sample image manifest.** Use a non-production sample file, never real protected health information. The snippet writes one DICOM placeholder and verifies length and metadata through the ADLS Gen2 endpoint.

```
from azure.identity import DefaultAzureCredential
from azure.storage.filedatalake import DataLakeServiceClient
```

```

account_name = "stmedimgprimary001"
file_system_name = "xray-raw"
path = "dicom/facility=denver/year=2026/month=07/image-0001.dcm"
payload = b"DICOM-SAMPLE-PLACEHOLDER\n"

credential = DefaultAzureCredential()
service = DataLakeServiceClient(
    account_url=f"https://{account_name}.dfs.core.windows.net",
    credential=credential,
)

file_system = service.get_file_system_client(file_system_name)
directory_name, file_name = path.rsplit("/", 1)
directory = file_system.get_directory_client(directory_name)
directory.create_directory()

file_client = directory.create_file(file_name)
file_client.append_data(payload, offset=0, length=len(payload))
file_client.flush_data(len(payload))
file_client.set_metadata({
    "classification": "phi",
    "modality": "xray",
    "retention": "worm",
})

properties = file_client.get_file_properties()
assert properties.size == len(payload)
assert properties.metadata["classification"] == "phi"
print(f"verified {path} with {properties.size} bytes")
```

Listing 1.8: Ingesting and verifying a sample object with azure-storage-file-datalake.

5. **Record validation evidence.** Capture account configuration, lifecycle policy, versioning, soft delete settings, immutability policy state, role assignments, and a DR test note. Evidence is a required deliverable, not optional paperwork.

Deliverables

Submit the following artifacts.

- A one-page architecture diagram showing primary region, secondary region, GZRS, ADLS Gen2, Key Vault, lifecycle tiers, and object replication.
- Azure CLI or infrastructure-as-code snippets that create the storage accounts, enable hierarchical namespace, configure soft delete and versioning, and apply lifecycle management.
- A short ADR explaining why ADLS Gen2, GZRS, immutable storage, customer-managed keys, and object replication were selected.
- A sample ingestion script or AzCopy command plus verification output.
- A retention and restore runbook covering overwrite recovery, blob deletion recovery, container deletion recovery, legal hold release, archive rehydration, and regional outage response.
- A cost note estimating how much of the 10 PB archive remains Hot, Cool, and Archive during normal operations.

Acceptance Criteria

- The account design uses ADLS Gen2 with hierarchical namespace enabled and does not rely on a flat namespace for analytics paths.
- Redundancy is explicitly set to GZRS or the ADR documents why a stricter data-residency rule prevents GZRS.
- Lifecycle policy transitions the intended X-ray prefix from Hot to Cool to Archive and has a separate rule for previous versions.
- Blob versioning, blob soft delete, and container soft delete are enabled and each restore scenario is explained.
- Immutable storage includes time-based retention and legal hold procedures for WORM requirements.
- Object replication policy or production-ready policy draft maps the primary X-ray container to a secondary-region account.
- Encryption uses customer-managed keys in Key Vault and includes ownership for rotation, monitoring, and emergency access.
- RBAC and ACL assignments are least-privilege and avoid shared keys for normal ingestion.
- Evidence is reproducible: another engineer can run the commands in a sandbox after changing only names and IDs.
- The DR runbook distinguishes storage replication from full application recovery and states RPO, RTO, and failover authority.

Stretch Goals

- Build a lifecycle cost model for 10 PB that compares all-Hot storage with a Hot, Cool, and Archive distribution over three years.
- Add Microsoft Purview scanning and classification for the raw and curated containers, including evidence that protected health information is tagged.
- Add private endpoints, private DNS validation, storage firewall rules, and a network diagram showing how ingestion avoids public internet exposure.
- Use Azure Policy to audit storage accounts that lack hierarchical namespace, CMK encryption, soft delete, or public access restrictions.
- Add monitoring alerts for lifecycle failures, replication lag, key access failures, denied data-plane operations, and sudden version-count growth.
- Prototype archive rehydration into a controlled restore prefix and measure the operational steps required before clinicians or auditors can read restored images.

Chapter 2

Relational Databases: Azure SQL and PostgreSQL

Learning Objectives

- Compare Azure SQL Database purchasing models, including DTU, vCore, Serverless, and Hyperscale.
- Explain how Azure Database for PostgreSQL Flexible Server provides managed PostgreSQL operations and high availability.
- Design active geo-replication and auto-failover groups for relational workloads with clear RPO and RTO expectations.
- Deploy an Azure SQL Database Serverless tier, create a geo-replica, and simulate failover using Azure CLI and Python.
- Plan a zero-downtime migration from an on-premises SQL Server monolith to Azure SQL Managed Instance.
- Evaluate managed relational database patterns across AWS, Azure, and GCP for portability and interview readiness.
- Build a high-availability, geo-replicated Azure SQL platform that supports a SQL Server to Managed Instance cutover.

Cross-Cloud Equivalence

Managed relational databases use the same operating pattern across clouds: the provider runs backups, patching, storage durability, and high-availability plumbing while engineers retain responsibility for schema design, workload isolation, identity, network access, observability, and cost.

Capability	AWS	Azure	GCP
Managed relational	RDS / Aurora	Azure SQL / Azure DB for PostgreSQL	Cloud SQL / AlloyDB
Scale-out relational	Aurora	Azure SQL Hyperscale	AlloyDB
High availability	Multi-AZ	Zone-redundant HA / auto-failover groups	Regional HA
Read replicas	RDS replicas / Aurora replicas	Azure SQL replicas / PostgreSQL read replicas	Cloud SQL replicas / AlloyDB read pools
Global / geo	Aurora Global Database	Active geo-replication / geo-replicas	Cross-region replicas
Migration	DMS + SCT	Azure Database Migration Service	GCP DMS

MongoDB Atlas, Microsoft Fabric warehouses, and Snowflake databases echo the same managed-service lesson: teams still design data contracts, security, failover behavior, and cost controls even when the provider abstracts servers.

Key Terms

DTU: a blended Azure SQL purchasing unit representing CPU, memory, reads, and writes. **vCore**: a purchasing model that exposes compute cores, hardware family, and storage choices. **Serverless**: an Azure SQL compute tier that auto-scales and can auto-pause for intermittent workloads. **Hyperscale**: an Azure SQL architecture that separates compute from a distributed storage layer for very large databases. **Flexible Server**: the current managed PostgreSQL deployment model with zone-aware HA options.

2.1 Week 2 Overview: Relational Engines in a Data Engineering Platform

Relational databases remain central to data engineering. Operational systems publish transactions from SQL Server, PostgreSQL, Oracle, and MySQL. Analytical pipelines land change data capture events into lakes. Dimension models, reference data, master data, orchestration metadata, and serving APIs frequently depend on relational semantics. A data engineer who only knows object storage and Spark will struggle when source systems require transactional consistency, query tuning, migration windows, stored procedures, and disaster-recovery runbooks.

Azure provides several managed relational choices. **Azure SQL Database** is a platform-as-a-service relational database based on the SQL Server engine. **Azure SQL Managed Instance** provides broader SQL Server compatibility for instance-level features. **Azure Database for PostgreSQL Flexible Server** provides managed open-source PostgreSQL with Azure identity, networking, backups, and high availability [45, 42]. The engineer's job is not to memorize product names, but to map workload requirements to service boundaries.

Definition

A **managed relational database** is a database service where the cloud provider automates infrastructure operations such as host provisioning, storage durability, backup orchestration, patching, and high-availability components, while the customer owns schema, permissions, queries, workload design, data protection requirements, and application behavior.

Note

Relational service selection is often a compatibility decision before it is a performance decision. If the application requires SQL Server Agent jobs, cross-database queries, CLR, Service Broker, or near instance-level parity, evaluate Azure SQL Managed Instance before assuming single-database Azure SQL Database is enough.

2.2 Monday: Azure SQL Database Purchasing Models

2.2.1 DTU and vCore Models

Azure SQL Database offers two primary purchasing models: DTU and vCore [8]. The DTU model bundles CPU, memory, reads, and writes into a single unit. It is convenient for smaller workloads, early prototypes, and teams that want a simple knob. Its weakness is diagnostic

opacity: when a workload is constrained, a DTU percentage does not immediately say whether CPU, log throughput, memory grants, or IO latency is the root cause.

The vCore model exposes compute cores, hardware family, service tier, and storage configuration more directly. It aligns better with enterprise sizing, reserved capacity, Azure Hybrid Benefit, and conversations with database administrators who already reason in cores, memory, and IO. General Purpose separates compute from remote storage for balanced cost. Business Critical uses local SSD replicas for low-latency IO and higher availability. Hyperscale uses a different distributed storage architecture.

Model	Best fit	Trade-off
DTU	Simple workloads, early applications, predictable small databases	Blended metric can hide which resource is saturated
vCore	Enterprise sizing, reserved capacity, hybrid licensing, clearer resource reasoning	Requires more explicit capacity planning
Serverless	Intermittent applications, development, unpredictable low duty cycle workloads	Cold starts and auto-pause settings must match application expectations
Hyperscale	Very large databases, rapid scale, fast restore, read scale-out	Different operational limits and feature behavior require validation

Table 2.1: Azure SQL purchasing choices should follow workload pattern, compatibility, and operating model rather than habit.

Tip

When migrating a production SQL Server workload, begin with observed baselines: peak CPU, memory pressure, data file size, log generation rate, tempdb use, longest queries, wait statistics, maintenance windows, and business RPO/RTO. Do not size only by database size.

2.2.2 Serverless for Variable Demand

Azure SQL Database Serverless is a compute tier in the vCore model that automatically scales compute within configured limits and can auto-pause after an inactivity delay [36]. Billing is based on compute used per second and storage allocated. It is useful for development databases, departmental applications, bursty APIs, training labs, and infrequently used operational stores.

Serverless is not a universal cost saver. If an application is always active, provisioned compute may be cheaper and more predictable. If a database auto-pauses, the first connection after inactivity must resume compute. Applications with strict latency service-level objectives may need a minimum vCore setting or disabled auto-pause. Workloads with background health checks may never pause.

Common Pitfall

Treating auto-pause as free savings. Auto-pause changes user experience. Test connection retry logic, login timeouts, monitoring probes, and application pools before using Serverless for anything user-facing.

2.2.3 Hyperscale Architecture

Hyperscale supports very large Azure SQL databases by separating compute nodes from a distributed storage subsystem with page servers and log services [22]. Compute replicas read pages from the storage layer and cache hot pages locally. The log service sequences writes and enables fast restore because data pages already live in distributed storage. Read scale-out replicas can serve reporting traffic without overloading the primary compute.

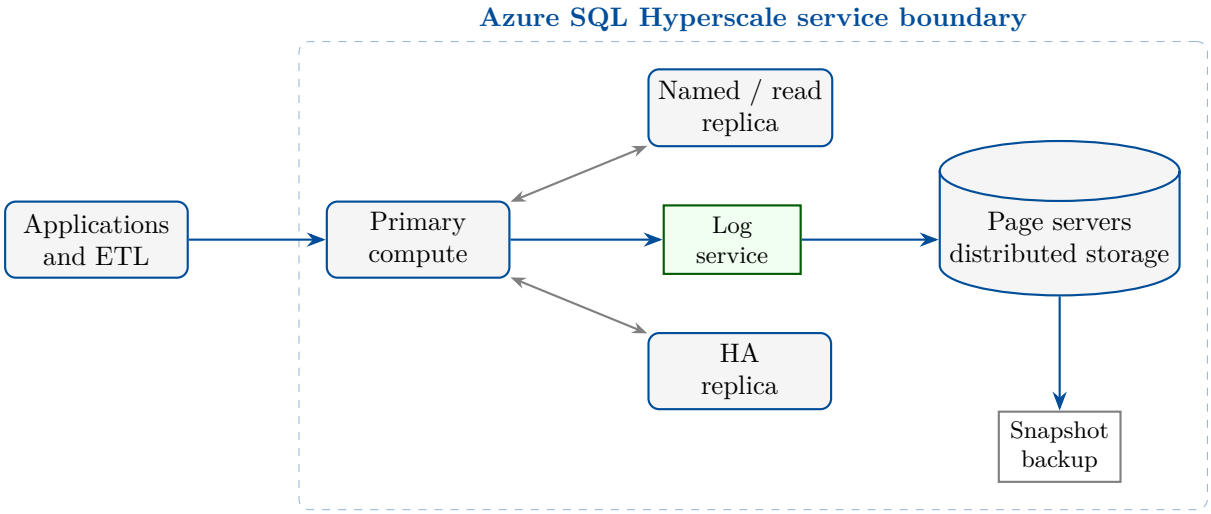


Figure 2.1: Azure SQL Hyperscale separates compute from log and page storage so large databases can scale and restore differently from traditional monolithic storage.

2.3 Tuesday: Azure Database for PostgreSQL Flexible Server and High Availability

2.3.1 Flexible Server as the Default PostgreSQL Model

Azure Database for PostgreSQL Flexible Server is the managed PostgreSQL deployment option used for most new Azure PostgreSQL workloads [42]. It provides configurable compute and storage, automatic backups, point-in-time restore, maintenance windows, private networking, read replicas, and high-availability options. Because it runs community PostgreSQL, application portability is usually stronger than with proprietary engines, but extensions, version support, parameter settings, and operational limits must still be checked.

Flexible Server separates concerns that on-premises teams often blend together. Azure owns host replacement, storage durability, backups, and platform patching. The application team owns schema migrations, connection pooling, query plans, vacuum behavior, bloat management, extension choice, and transaction design. PostgreSQL performance failures in the cloud are still often ordinary PostgreSQL failures: missing indexes, unbounded transactions, chatty connections, overloaded temp files, or poor statistics.

Definition

Flexible Server high availability provisions a standby server that can take over when the primary becomes unavailable. Zone-redundant HA places standby capacity in a different availability zone when the region supports zones; same-zone HA focuses on local resilience with lower network latency.

2.3.2 High Availability and Backups

High availability and backups solve different problems. HA reduces outage duration when a node or zone fails. Backups restore data to a previous point in time after corruption, accidental delete, bad deployments, or application mistakes. A standby faithfully replicates bad data changes; backups allow recovery before those changes. Data engineers must include both in the architecture.

Note

PostgreSQL HA failover changes the server endpoint target, but applications still need connection retry behavior. Use connection pooling carefully: stale connections can make a healthy failover look like an application outage.

Capability	What it provides	What it does not provide
Zone-redundant HA	Standby in another availability zone and automated failover	Protection from bad SQL, accidental drops, or cross-region disaster
Point-in-time restore	Recovery to a prior timestamp within backup retention	Zero data loss after a primary-region outage
Read replica	Read scale-out and reporting isolation	Synchronous HA or automatic application cutover by itself
Maintenance window	Predictable platform patching window	Application release governance or schema migration safety

Table 2.2: PostgreSQL operational controls must be combined to meet availability and recoverability goals.

2.3.3 Designing PostgreSQL for Data Engineering

PostgreSQL often appears in data platforms as a metadata store, operational data mart, feature-serving database, orchestration catalog, or domain service database. The design should choose indexes for access patterns, keep transactions short, use connection pooling, monitor vacuum, and isolate reporting from write-heavy workloads. Read replicas are valuable for BI or extraction workloads, but replica lag must be measured before using them for near-real-time pipelines.

Tip

For ingestion-heavy PostgreSQL workloads, track write-ahead-log generation, autovacuum lag, dead tuples, lock waits, checkpoint behavior, and replica lag. These signals are often more useful than a generic CPU chart.

2.4 Wednesday: Active Geo-Replication and Auto-Failover Groups

2.4.1 Active Geo-Replication

Active geo-replication creates up to four readable secondary databases in different regions for Azure SQL Database [4]. Replication is asynchronous. The secondary can support read-only workloads and can be promoted during disaster recovery. Because replication is asynchronous, the platform can lose the most recent committed transactions during a regional disaster. The business must decide whether that RPO is acceptable.

Active geo-replication is database-scoped. It is excellent when one critical database needs a readable copy in another region. However, many applications use several databases together. If

those databases must fail over as a unit, an auto-failover group is usually a better control-plane abstraction.

Common Pitfall

Ignoring consistency across databases. If an application writes to five databases, failing over only one database can create broken business state. Group related databases and test the full application, not only the database endpoint.

2.4.2 Auto-Failover Groups

Auto-failover groups provide a listener endpoint and group-level failover for one or more databases on Azure SQL Database logical servers or SQL Managed Instances [5]. Applications connect to a read-write listener and optionally a read-only listener. During failover, the listener redirects to the new primary. This lowers DNS and connection-string complexity, but it does not remove the need for retries, idempotent writes, and operational decision authority.

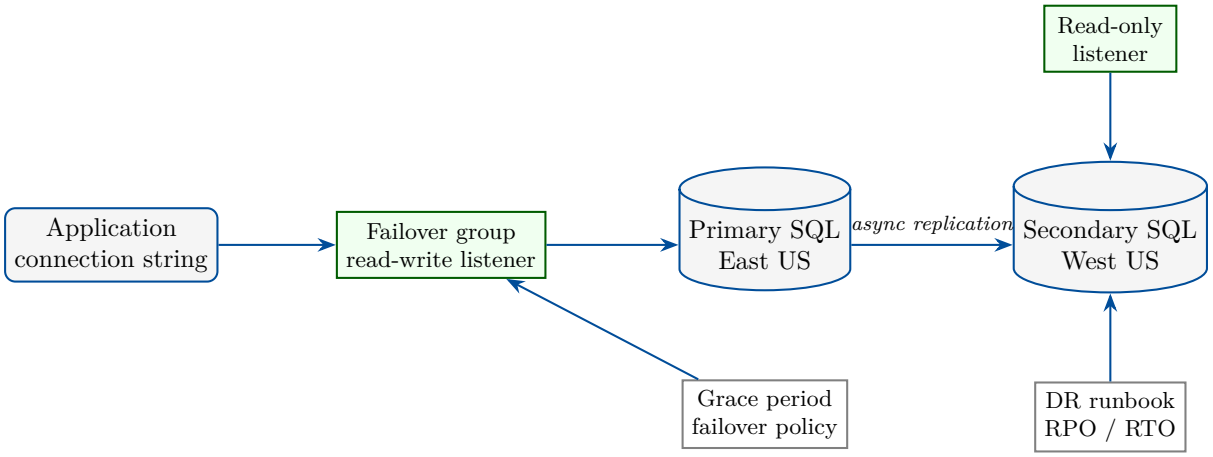


Figure 2.2: Azure SQL auto-failover groups provide listener endpoints over asynchronously replicated databases for regional resilience.

2.4.3 Testing Failover Without Fooling Yourself

A failover test should include more than pressing a button. Capture baseline lag, pause nonessential writes, run synthetic transactions, fail over, reconnect through the listener, validate data, observe application retries, and fail back only after leadership approves. If the database feeds ETL, test downstream idempotency: a pipeline may reread rows or miss rows if watermark logic is not designed for failover.

Note

Disaster recovery is a socio-technical process. Azure can provide replicas and listeners, but people still decide when to fail over, how much data loss is acceptable, who communicates impact, and how systems return to normal.

2.5 Thursday Hands-on Project: Serverless Azure SQL with Geo-Replica and Failover

This lab deploys an Azure SQL logical server, a Serverless database, a secondary server in another region, and a geo-replica. It then uses Python to write and validate a small workload before a manual failover simulation. Run it only in a sandbox subscription and delete the resource group when finished.

Note

The examples are written for Windows PowerShell with Azure CLI. Replace server names with globally unique names and use a strong administrator password stored outside source control.

```
az login
az account set --subscription "00000000-0000-0000-0000-000000000000"

$rg = "rg-de-azure-book-week2"
$primaryLocation = "eastus"
$secondaryLocation = "westus"
$primaryServer = "sql-week2-primary-001"
$secondaryServer = "sql-week2-secondary-001"
$db = "sqldb-orders-serverless"
$admin = "sqladminuser"
$password = Read-Host "SQL admin password" -AsSecureString
$plainPassword = [Runtime.InteropServices.Marshal]::PtrToStringAuto(
  [Runtime.InteropServices.Marshal]::SecureStringToBSTR($password))

az group create `
  --name $rg `
  --location $primaryLocation

az sql server create `
  --name $primaryServer `
  --resource-group $rg `
  --location $primaryLocation `
  --admin-user $admin `
  --admin-password $plainPassword

az sql server create `
  --name $secondaryServer `
  --resource-group $rg `
  --location $secondaryLocation `
  --admin-user $admin `
  --admin-password $plainPassword

az sql db create `
  --resource-group $rg `
  --server $primaryServer `
  --name $db `
  --edition GeneralPurpose `
  --compute-model Serverless `
  --family Gen5 `
  --capacity 2 `
  --min-capacity 0.5 `
  --auto-pause-delay 60 `
  --zone-redundant false

az sql db replica create `
  --resource-group $rg `
  --server $primaryServer `
```

```
  --name $db `
  --partner-resource-group $rg `
  --partner-server $secondaryServer
```

Listing 2.1: Deploying Azure SQL Database Serverless and creating a geo-replica with Azure CLI.

Open firewall access only for a controlled client IP or private network. The simple rule below is a lab convenience and should not be copied into production without review.

```
$clientIp = (Invoke-RestMethod -Uri "https://api.ipify.org")

az sql server firewall-rule create `
  --resource-group $rg `
  --server $primaryServer `
  --name "client-ip" `
  --start-ip-address $clientIp `
  --end-ip-address $clientIp

az sql server firewall-rule create `
  --resource-group $rg `
  --server $secondaryServer `
  --name "client-ip" `
  --start-ip-address $clientIp `
  --end-ip-address $clientIp

az sql db replica list-links `
  --resource-group $rg `
  --server $primaryServer `
  --name $db `
  --output table
```

Listing 2.2: Adding a narrow client firewall rule and checking replica state.

The Python snippet creates a table, inserts a row, and reads it back. Install `pyodbc` only in a local lab environment with the Microsoft ODBC Driver for SQL Server available.

```
import os
import pyodbc

server = os.environ["AZURE_SQL_SERVER"]
database = os.environ.get("AZURE_SQL_DATABASE", "sqldb-orders-serverless")
username = os.environ["AZURE_SQL_USER"]
password = os.environ["AZURE_SQL_PASSWORD"]

connection_string = (
    "Driver={ODBC Driver 18 for SQL Server};"
    f"Server=tcp:{server}.database.windows.net,1433;"
    f"Database={database};Uid={username};Pwd={password};"
    "Encrypt=yes;TrustServerCertificate=no;Connection Timeout=30;"
)

with pyodbc.connect(connection_string) as conn:
    cursor = conn.cursor()
    cursor.execute("""
        IF OBJECT_ID('dbo.orders') IS NULL
        CREATE TABLE dbo.orders (
            order_id int NOT NULL PRIMARY KEY,
            status nvarchar(40) NOT NULL,
            updated_at datetime2 NOT NULL DEFAULT sysutcdatetime()
        )
    """)
    cursor.execute("""
```

```
        MERGE dbo.orders AS target
        USING (SELECT 1001 AS order_id, N'paid' AS status) AS source
        ON target.order_id = source.order_id
        WHEN MATCHED THEN UPDATE SET status = source.status,
             updated_at = sysutcdatetime()
        WHEN NOT MATCHED THEN INSERT (order_id, status)
             VALUES (source.order_id, source.status);
    """)
    conn.commit()
    row = cursor.execute(
        "SELECT order_id, status FROM dbo.orders WHERE order_id = 1001"
    ).fetchone()
    print(row.order_id, row.status)
```

Listing 2.3: Validating a write before failover with Python and pyodbc.

After validating the primary, simulate regional failover by promoting the replica. Update the Python environment variable to the secondary server and rerun the read query.

```
az sql db replica set-primary `
  --resource-group $rg `
  --server $secondaryServer `
  --name $db

$env:AZURE_SQL_SERVER = $secondaryServer
$env:AZURE_SQL_DATABASE = $db
python .\validate-orders.py
```

Listing 2.4: Simulating regional failover by promoting the geo-replica.

Common Pitfall

Failover without client testing. A database can promote successfully while applications still fail because connection strings, DNS, firewall rules, secret stores, or retry policies still point to the old region.

2.6 Friday Use Case: Zero-Downtime Migration to Azure SQL Managed Instance

2.6.1 Scenario

A retailer runs a monolithic on-premises SQL Server that supports orders, inventory, billing, and customer service. The database is 6 TB, uses SQL Server Agent jobs, linked-server dependencies, cross-database queries, and several vendor stored procedures. The business wants to move to Azure with almost no downtime. Azure SQL Managed Instance is selected because it provides high SQL Server compatibility while reducing infrastructure operations [46].

Zero downtime is an aspiration, not a magic switch. The practical target is a short controlled cutover window with continuous replication before the switch. The migration team must assess compatibility, remediate blockers, establish private connectivity, provision Managed Instance, migrate logins and jobs, seed data, replicate changes, rehearse cutover, freeze risky releases, and validate application behavior after DNS or connection-string changes.

Definition

Azure SQL Managed Instance is a managed SQL Server-compatible platform-as-a-service deployment option that preserves many instance-level capabilities while Azure operates the underlying infrastructure, patching, backups, and availability components.

2.6.2 Migration Architecture

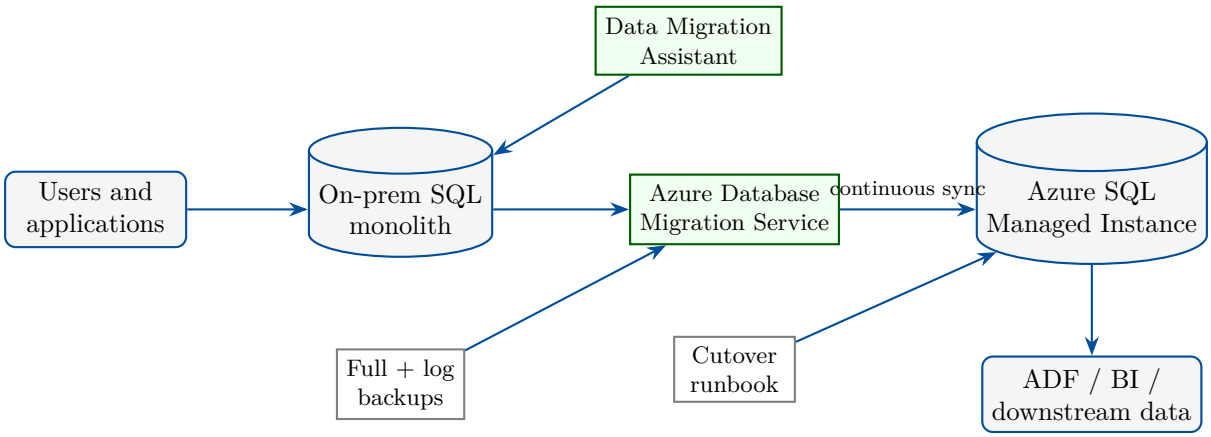


Figure 2.3: Zero-downtime migration pattern from an on-premises SQL Server monolith to Azure SQL Managed Instance using assessment, seeding, continuous synchronization, and controlled cutover.

2.6.3 Cutover Runbook

The runbook begins weeks before migration night. Data Migration Assistant or Azure Migrate assessments identify unsupported features, deprecated syntax, collation issues, SQL Agent job dependencies, and performance risks. Network teams validate ExpressRoute or VPN paths. Security teams create Microsoft Entra groups, managed identities, private DNS zones, and Key Vault entries. DBAs rehearse restore and synchronization. Data engineers validate downstream extract jobs, watermark logic, and reporting latency.

During the final cutover, the team freezes writes or places the application in maintenance mode, lets replication catch up, validates row counts and checksums, promotes Managed Instance, updates connection endpoints, starts smoke tests, watches error rates, and confirms downstream pipelines. Rollback must be explicit: either point applications back to on-premises before new writes occur in Azure, or accept Azure as the system of record and remediate forward.

Tip

For near-zero downtime, rehearse the exact cutover multiple times with production-like data and timing. The first successful migration should never be the production migration.

2.7 Architectural Decision Record Template

A Week 2 ADR should capture engine choice, compatibility constraints, purchasing model, HA pattern, backup retention, geo-replication, failover authority, private networking, identity model, migration approach, and cost controls. Include alternatives: Azure SQL Database, SQL Managed Instance, PostgreSQL Flexible Server, self-managed virtual machines, and cross-cloud options. The ADR is strongest when it records rejected options and why they were rejected.

2.8 Operational Deep Dive: Performance, Identity, and Observability

Relational platforms fail through ordinary engineering gaps. Queries miss indexes, statistics go stale, connection pools saturate, transactions hold locks, deployments change query plans, and extract jobs create accidental denial of service. Azure adds tools such as Query Store, automatic tuning, metrics, diagnostic logs, alerts, Microsoft Defender for SQL, and Azure Monitor, but the team still needs ownership.

Identity should prefer Microsoft Entra ID, managed identities, group-based RBAC, and contained database users where supported. Avoid embedding SQL administrator passwords in pipelines. Network access should use private endpoints or private link patterns for production. Observability should include CPU, storage, log IO, deadlocks, failed connections, long-running queries, replica lag, failover events, backup state, and cost by environment.

Common Pitfall

Letting ETL act like an unbounded user. A nightly extraction that scans every table can starve the operational workload. Use read replicas, change tracking, CDC, batching, and workload windows rather than brute-force reads.

2.9 Troubleshooting Patterns

Common relational incidents fall into five groups. Connectivity failures require checking firewall rules, private DNS, TLS settings, credentials, and listener names. Performance failures require Query Store, wait statistics, indexes, plans, and workload changes. HA failures require examining failover events and client retry logic. Replication surprises require checking lag, unsupported operations, and region availability. Migration failures require compatibility reports, login mappings, SQL Agent jobs, linked servers, and collation differences.

A disciplined response avoids broad fixes. Do not scale from two vCores to thirty-two vCores before checking whether a missing index caused the outage. Do not disable firewall rules because one pipeline cannot connect. Do not force failover until you know replication lag and business impact. As with storage, small precise changes preserve trust.

2.10 Week 2 Lab Rubric

A complete Week 2 submission includes Azure CLI deployment evidence, a diagram of primary and secondary relational regions, Python validation output before and after failover, an ADR explaining purchasing and HA choices, and a migration runbook for the Managed Instance scenario.

Criterion	Meets expectation	Needs improvement
Model selection	DTU, vCore, Serverless, and Hyperscale trade-offs are explained	Service tier chosen without workload evidence
HA / DR	Geo-replication, failover groups, RPO, and RTO are documented	Replica assumed to solve all application recovery
Hands-on	CLI creates database and replica; Python validates data before and after failover	Commands are portal-only or validation is missing
Security	Private access, Entra ID, least privilege, and secrets hygiene are addressed	Broad firewall or admin password habits remain
Migration	Managed Instance cutover has assessment, sync, validation, rollback, and downstream checks	Migration is described as a one-step restore

Table 2.3: Week 2 rubric for relational database architecture and hands-on deliverables.

2.11 Interview Q&A

2.11.1 Concepts and Trade-offs

1. **Q: When would you choose vCore over DTU for Azure SQL Database?**
   A: Choose vCore when you need explicit control over cores, hardware family, storage, reserved capacity, hybrid licensing, or enterprise capacity planning.

2. **Q: Why is Serverless not always cheaper?**
   A: Always-active workloads may pay continuously, auto-pause may never trigger, and cold-start latency can force a minimum compute setting.

3. **Q: What makes Hyperscale different from Business Critical?**
   A: Hyperscale separates compute from distributed page and log storage for very large databases and fast restore; Business Critical focuses on local SSD replicas and low-latency IO.

4. **Q: What does PostgreSQL Flexible Server HA protect against?**
   A: It reduces outage from server or zone failures, but it does not protect against bad application writes; backups and PITR are still required.

5. **Q: Why use an auto-failover group instead of only active geo-replication?**
   A: A failover group provides listener endpoints and group-level failover for related databases, simplifying application connection management.

6. **Q: What is the largest risk in a zero-downtime SQL Server migration?**
   A: Unrehearsed cutover dependencies: compatibility gaps, logins, jobs, connection strings, downstream ETL, and rollback rules can break despite successful data copy.

2.12 Further Reading

Start with Azure SQL Database documentation [45], purchasing model guidance [8], Serverless [36], and Hyperscale [22]. Review Azure Database for PostgreSQL Flexible Server [42] and its high-availability concepts [21]. For regional resilience, read active geo-replication [4] and auto-failover groups [5]. For migration, study Azure SQL Managed Instance [46], Azure Database Migration Service [43], and the Data Migration Assistant [15]. Use the Azure Well-Architected Framework [28] to evaluate reliability, security, operational excellence, performance, and cost.

Key Takeaways

- Azure SQL Database purchasing models differ by abstraction: DTU is blended, vCore is explicit, Serverless is elastic for intermittent demand, and Hyperscale changes storage architecture.
- PostgreSQL Flexible Server provides managed operations, but PostgreSQL fundamentals such as indexes, vacuum, transactions, and connection pooling still matter.
- HA, backups, read replicas, active geo-replication, and failover groups solve different reliability problems.
- Asynchronous geo-replication requires explicit RPO acceptance and application-level retry, idempotency, and validation.
- Zero-downtime migration depends on assessment, rehearsal, continuous sync, cutover governance, rollback rules, and downstream data validation.
- Managed database services reduce infrastructure toil; they do not remove schema, security, performance, observability, or DR ownership.

2.13 Exercises

1. **(Conceptual)** Compare DTU and vCore for a 200 GB order-processing database. Which model gives clearer capacity planning and why?
2. **(Conceptual)** Explain when Azure SQL Serverless is appropriate and list two application behaviors that can prevent auto-pause savings.
3. **(Conceptual)** Draw the difference between Azure SQL active geo-replication and an auto-failover group.
4. **(Hands-on)** Run the Serverless Azure SQL deployment in a sandbox and capture evidence of the database compute model and replica link.
5. **(Hands-on)** Modify the Python validation script to insert ten orders, fail over, and confirm all ten rows exist on the promoted replica.
6. **(Hands-on)** Create a PostgreSQL Flexible Server design checklist that includes HA, backups, private access, parameter changes, and read replicas.
7. **(Design)** Choose between Azure SQL Database, Azure SQL Managed Instance, and PostgreSQL Flexible Server for a vendor application with SQL Agent jobs and cross-database queries.
8. **(Design)** Write a zero-downtime migration runbook for the retailer scenario, including rollback criteria and downstream ETL validation.

2.14 Capstone Project: HA Geo-Replicated Azure SQL Platform for Managed Instance Cutover

Capstone Project

Mission: Design and prototype a high-availability, geo-replicated Azure SQL platform that supports a controlled SQL Server to Azure SQL Managed Instance cutover. Your solution must combine assessment, private networking, continuous migration, failover design, validation, and operational evidence.
Estimated time: 5–7 hours.
What you will practice:

- Selecting Azure SQL Managed Instance when instance-level SQL Server compatibility matters.
- Designing regional resilience with failover groups, backups, PITR, monitoring, and application retry behavior.
- Preparing a near-zero-downtime migration using assessment, seeding, continuous synchronization, and cutover rehearsal.
- Validating data integrity, downstream ETL behavior, and rollback criteria.
- Producing evidence suitable for an architecture review and migration go/no-go meeting.

Scenario

A financial services company must move its 4 TB on-premises SQL Server monolith to Azure SQL Managed Instance. The application supports payments, customer service, finance reconciliation, and regulatory reporting. Planned downtime is limited to 30 minutes. The platform must be resilient to zone failure and prepared for regional failover. Data engineers own downstream extracts into ADLS Gen2 and must prove that the cutover will not duplicate or lose data in the lake.

Requirements

- **Compatibility:** Use Azure SQL Managed Instance because the workload depends on SQL Server Agent jobs, cross-database queries, and high SQL Server surface-area compatibility.
- **High availability:** Use a Business Critical or General Purpose configuration with documented zone redundancy where supported and clear maintenance windows.
- **Geo-resilience:** Design an auto-failover group between primary and secondary Managed Instances and state asynchronous replication RPO assumptions.
- **Migration:** Use Azure Database Migration Service or a documented backup and log-shipping style approach for initial load plus continuous synchronization.
- **Security:** Use private endpoints or delegated subnet integration, Microsoft Entra groups, Key Vault for secrets, TLS, auditing, and least-privilege roles.
- **Validation:** Reconcile row counts, checksums for critical tables, SQL Agent job behavior, application smoke tests, and downstream ETL watermarks.
- **Operations:** Provide monitoring alerts, failover authority, rollback criteria, communication plan, and cleanup steps for obsolete on-premises dependencies.

Reference Architecture

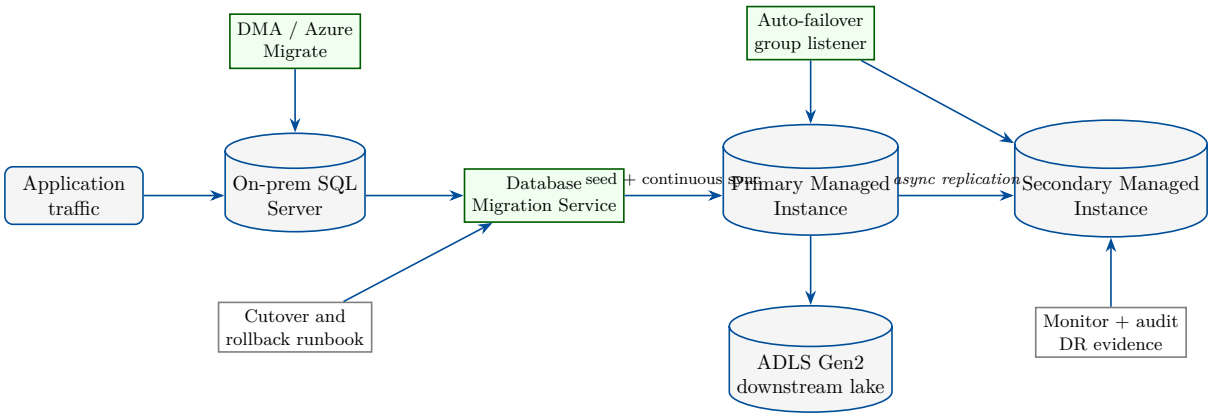


Figure 2.4: Capstone reference architecture for a high-availability Azure SQL Managed Instance migration with continuous sync, failover group, downstream lake validation, and operational evidence.

Implementation Steps

Use a sandbox subscription for the prototype. Managed Instance deployments can take significant time and incur cost, so adapt names and sizes to your lab constraints. The snippets below show the structure rather than a production sizing recommendation.

1. **Create resource group and plan the Managed Instance network.** Managed Instance requires a dedicated subnet with appropriate delegation and network controls.

```
az login
az account set --subscription "00000000-0000-0000-0000-000000000000"

$rg = "rg-sqlmi-cutover-capstone"
$location = "eastus"
$vnet = "vnet-sqlmi-cutover"
$subnet = "snet-managed-instance"
$nsg = "nsg-sqlmi-cutover"
$routeTable = "rt-sqlmi-cutover"

az group create `
  --name $rg `
  --location $location

az network vnet create `
  --resource-group $rg `
  --location $location `
  --name $vnet `
  --address-prefixes 10.40.0.0/16 `
  --subnet-name $subnet `
  --subnet-prefixes 10.40.1.0/24

az network nsg create `
  --resource-group $rg `
  --location $location `
  --name $nsg

az network route-table create `
  --resource-group $rg `
  --location $location `
```

```
  --name $routeTable

az network vnet subnet update `
  --resource-group $rg `
  --vnet-name $vnet `
  --name $subnet `
  --network-security-group $nsg `
  --route-table $routeTable `
  --delegations Microsoft.Sql/managedInstances
```

Listing 2.5: PowerShell plus Azure CLI skeleton for Managed Instance network preparation.

2. **Prototype the primary Managed Instance and capture configuration.** Use Azure Portal, Bicep, Terraform, or Azure CLI according to organizational standards. The key deliverable is reproducible evidence: SKU, storage, collation, time zone, subnet, administrator model, backup retention, auditing, and maintenance window.

```
$mi = "mi-sql-cutover-primary-001"
$admin = "sqlmiadmin"
$password = Read-Host "Managed Instance admin password" -AsSecureString
$plainPassword = [Runtime.InteropServices.Marshal]::PtrToStringAuto(
  [Runtime.InteropServices.Marshal]::SecureStringToBSTR($password))
$subnetId = az network vnet subnet show `
  --resource-group $rg `
  --vnet-name $vnet `
  --name $subnet `
  --query id `
  --output tsv

az sql mi create `
  --resource-group $rg `
  --name $mi `
  --location $location `
  --subnet $subnetId `
  --admin-user $admin `
  --admin-password $plainPassword `
  --license-type BasePrice `
  --storage 256GB `
  --capacity 4 `
  --tier GeneralPurpose `
  --family Gen5

az sql mi show `
  --resource-group $rg `
  --name $mi `
  --query "{name:name,state:state,sku:sku.name,storage:storageSizeInGB,
    subnet:subnetId}" `
  --output table
```

Listing 2.6: Managed Instance creation skeleton and configuration evidence commands.

3. **Plan the geo-replicated cutover.** Create a secondary Managed Instance in the paired region, configure an auto-failover group, and require application teams to connect through the listener. Document RPO expectations because replication is asynchronous.

4. **Run assessment and migration rehearsal.** Execute Data Migration Assistant reports, remediate blockers, seed a copy with Azure Database Migration Service, start continuous sync, and measure lag during normal and peak workloads.

5. **Validate data and downstream pipelines.** Before cutover, compare critical table row counts, checksums, SQL Agent jobs, permissions, and ETL watermarks. After cutover, run application smoke tests and the downstream ADLS Gen2 extraction validation.

Deliverables

- Architecture diagram showing on-premises SQL Server, DMS or migration tooling, primary Managed Instance, secondary Managed Instance, failover group listener, private network, Key Vault, monitoring, and ADLS Gen2 downstream consumers.
- ADR explaining why Azure SQL Managed Instance was selected and why Azure SQL Database, PostgreSQL Flexible Server, and SQL Server on virtual machines were rejected.
- Reproducible deployment snippets or infrastructure-as-code for network, Managed Instance, auditing, backup retention, and failover group configuration.
- Migration runbook covering assessment, remediation, seed load, continuous sync, pre-cutover checks, cutover, validation, rollback, and communication.
- Evidence pack with assessment results, row-count reconciliation, checksum queries, failover test output, application smoke-test results, and downstream ETL validation.

Acceptance Criteria

- Managed Instance is justified by documented SQL Server compatibility requirements.
- Network design uses a dedicated subnet and avoids public administrative habits for production.
- HA, backup retention, PITR, failover group, RPO, and RTO are explicitly documented.
- Migration process includes initial load, continuous synchronization, measured lag, and at least one rehearsal.
- Cutover plan includes freeze window, final sync validation, listener or connection-string switch, smoke tests, and rollback criteria.
- Downstream data engineering jobs have validated watermarks so cutover does not duplicate or skip records.
- Security design uses least privilege, Key Vault, auditing, and group-based access instead of shared administrator credentials.
- Evidence is complete enough for a go/no-go meeting and for another engineer to reproduce the lab with changed names.

Stretch Goals

- Add a Bicep or Terraform implementation for the full Managed Instance network and primary instance.
- Build a failover drill script that records listener endpoint, replication role, application smoke-test status, and elapsed recovery time.
- Add Query Store baseline capture before and after migration to compare top queries and regressions.
- Use Microsoft Defender for SQL, vulnerability assessment, and audit logs to build a security evidence appendix.
- Add a downstream CDC pipeline that writes to ADLS Gen2 and proves idempotent replay across cutover.

- Estimate three-year cost for General Purpose versus Business Critical, reserved capacity, Azure Hybrid Benefit, and secondary-region DR.

Chapter 3

High-Scale NoSQL: Azure Cosmos DB

Learning Objectives

- Explain Azure Cosmos DB core concepts: Request Units, logical partitions, physical partitions, replicas, and global distribution.
- Compare the five Cosmos DB consistency levels and select the weakest level that still satisfies business correctness.
- Choose between the Core (NoSQL), MongoDB, and Cassandra APIs based on application model, tooling, and portability requirements.
- Use the Change Feed to connect operational NoSQL writes to streaming, lakehouse, search, and serving pipelines.
- Create a Cosmos DB for NoSQL account, design a single-container forum schema, insert items with Python, and configure multi-region writes.
- Design a globally distributed gaming leaderboard capable of high read and write rates with Session consistency.
- Build a capstone architecture for a multi-region leaderboard with partitioning, observability, cost controls, and operational evidence.

Cross-Cloud Equivalence

NoSQL and document databases look different across vendors, but data engineers ask the same questions everywhere: What is the partition key? How is throughput billed? Which consistency model is exposed? How do changes flow downstream? What happens during regional failure?

Concept	Azure Cosmos DB	AWS DynamoDB	GCP Firestore / Bigtable	MongoDB
Primary partitioning	Container partition key routes items to logical partitions	Partition key, optional sort key	Firestore document paths; Bigtable row key	Shard key
Throughput unit	Request Units per second (RU/s) or serverless RUs	Read / write capacity units (RCU/WCU) or on-demand	Firestore operations; Bigtable nodes and storage	Cluster tier, IOPS, and workload capacity
Secondary indexes	Automatic indexing policy with included / excluded paths and composites	Global and local secondary indexes	Firestore composite indexes; Bigtable schema by row key	Secondary and compound indexes
Change stream	Change Feed and Change Feed Processor	DynamoDB Streams	Firestore triggers / exports; Bigtable change streams	Change streams
TTL	Item or container time-to-live	Item TTL attribute	Firestore TTL policies; Bigtable garbage collection	TTL indexes
Multi-region writes	Multi-region account with write regions and conflict policy	Global tables	Multi-region Firestore; Bigtable replication	Global clusters / sharded clusters
Consistency	Five tunable levels from Strong to Eventual	Strong or eventual reads, transactional options	Strong regional semantics vary by service	Read / write concern and causal consistency

A portable NoSQL design names its access patterns, partition strategy, indexing choices, retry behavior, and conflict rules instead of assuming one product's defaults will transfer unchanged to another cloud.

Key Terms

Request Unit (RU): a normalized charge for CPU, memory, and IO consumed by an operation. **Logical partition**: all items in a container with the same partition-key value. **Physical partition**: service-managed storage and compute that hosts one or more logical partitions. **Consistency level**: a contract that controls read freshness and ordering. **Change Feed**: an ordered feed of item changes used for event-driven processing.

3.1 Week 3 Overview: Why Cosmos DB Matters to Data Engineers

Azure Cosmos DB is a globally distributed, multi-model NoSQL database service designed for low-latency reads and writes, elastic throughput, automatic indexing, and configurable consistency [38]. It appears in data platforms as an operational serving layer, user-profile store, catalog store, IoT metadata database, event-sourced projection, feature-serving database, and high-scale API backend.

Relational databases optimize joins, transactions, and normalized constraints. Cosmos DB optimizes predictable item access by partition key, low-latency point reads, horizontal scale, regional distribution, and flexible JSON documents. The change is not merely syntax. A Cosmos DB engineer must design containers around access patterns, choose partition keys that distribute load, estimate RU consumption, define indexing policies, and accept the consistency contract needed by the product.

Definition

Azure Cosmos DB for NoSQL stores JSON items in containers. Every item has an `id`, belongs to one logical partition selected by a partition key, is automatically indexed by default, and consumes Request Units for reads, writes, queries, and maintenance operations.

Note

Cosmos DB is not a replacement for every relational database. If the workload depends on multi-row joins, large ad hoc aggregations, cross-container transactions, or strict relational integrity, use Cosmos DB as a serving projection or event store rather than as the only system of record.

3.2 Monday: Core Concepts — RUs, Partitioning, and Global Distribution

3.2.1 Request Units as the Throughput Currency

Every Cosmos DB operation consumes RUs. A point read of a 1 KB item is often close to 1 RU, while larger documents, complex queries, cross-partition scans, writes, indexing, and consistency choices consume more [35]. Provisioned throughput allocates RU/s to a database or container. Autoscale raises and lowers throughput within a configured maximum. Serverless charges per consumed RU and suits intermittent or development workloads.

RUs make capacity planning explicit. Instead of asking only how many vCores a database needs, estimate operations per second multiplied by RU per operation. For example, 20,000 point reads per second at 1 RU each require roughly 20,000 RU/s before retries, spikes, consistency overhead, and regional replication are considered. Queries that scan many partitions can consume orders of magnitude more.

Tip

Measure RU charge from real SDK responses. Theoretical estimates are useful for sizing, but query shape, item size, index policy, consistency level, and partition fan-out determine actual cost.

3.2.2 Partition Keys and Logical Partitions

A container's partition key is one of the most important design decisions. Cosmos DB uses the partition-key value to group items into logical partitions and to route requests. Good partition keys have high cardinality, even distribution, and alignment with common point-read and query filters. Poor partition keys create hot partitions, cross-partition queries, throttling, and expensive migrations.

For a forum container, `/forumId` may group all posts and comments for a small forum, but it can become hot if one public forum dominates traffic. `/tenantId` may work for business SaaS tenants with balanced activity. A synthetic key such as `/leaderboardBucket` or `/gameIdSeasonShard` may be necessary for gaming workloads where one game or region can receive massive writes.

Common Pitfall

Choosing a partition key because it is easy to explain. Choose it because it distributes writes, supports dominant reads, fits transaction boundaries, and avoids unbounded growth in a single logical partition.

3.2.3 Physical Partitions and Hot Keys

Cosmos DB manages physical partitions behind the scenes. As storage and throughput grow, the service splits data across physical partitions. Logical partition keys are still the engineer's responsibility. If one key receives most writes, the service cannot magically distribute that single logical partition across many keys. Hot partitions show up as 429 throttling, uneven RU consumption, high latency for specific key values, and queries that require fan-out.

3.2.4 Global Distribution

Cosmos DB accounts can replicate data to multiple Azure regions. A single-write-region account sends writes to one region and replicates reads elsewhere. A multi-write account allows writes in more than one region and uses conflict resolution when concurrent updates target the same item [19]. Multi-region design reduces user latency and improves resilience, but it increases operational complexity and cost.

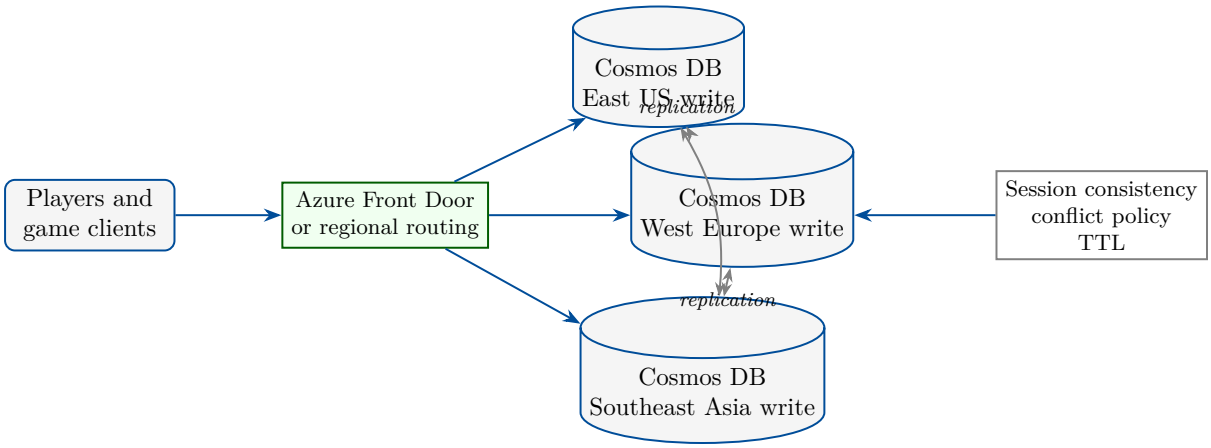


Figure 3.1: Cosmos DB global distribution places replicas near users while consistency and conflict policies define correctness during replication.

3.3 Tuesday: The Five Consistency Levels

Cosmos DB exposes five consistency levels: Strong, Bounded Staleness, Session, Consistent Prefix, and Eventual [13]. They form a practical spectrum between freshness, availability, latency, and cost. Strong consistency gives linearizable reads within supported regional configurations. Eventual consistency gives the weakest freshness guarantee but the lowest coordination burden.

Definition

A **consistency level** is the read contract an application receives after writes. It controls whether reads must observe the latest write, a bounded lag, a user's own writes, a prefix-preserving order, or eventual convergence.

3.3.1 Strong and Bounded Staleness

Strong consistency returns the most recent committed version of an item, but it limits some global distribution choices because replicas must coordinate before reads complete. Use Strong when correctness requires immediate global visibility, such as inventory reservation for scarce resources. Do not choose it reflexively for personalization, telemetry, or leaderboards where slightly stale reads are acceptable.

Bounded Staleness allows reads to lag behind writes by a configured number of versions or time interval. It is useful when users can tolerate a known delay but the business wants a hard upper bound. It costs more coordination than Session or Eventual and should be justified by a measurable requirement.

3.3.2 Session, Consistent Prefix, and Eventual

Session consistency is the default for many application workloads because it provides read-your-writes within a client session while allowing global distribution. A player who submits a score should see that score immediately in their session even if other players in far regions see it after replication catches up. The SDK carries a session token to preserve that guarantee.

Consistent Prefix guarantees that reads never see writes out of order. If updates were A, B, and C, a client may see A or A,B, but not A,C without B. Eventual consistency only guarantees convergence when writes stop. It can be appropriate for derived counters, telemetry summaries, caches, and recommendation signals where temporary staleness is acceptable.

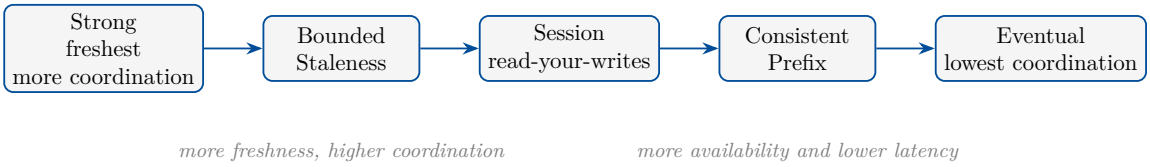


Figure 3.2: Cosmos DB consistency levels trade read freshness and ordering for lower coordination, latency, and regional independence.

Tip

Choose the weakest consistency level that satisfies the user story. For global applications, Session often gives excellent user experience because users see their own writes without forcing every region to agree before every read.

3.4 Wednesday: APIs and the Change Feed

3.4.1 Core (NoSQL), MongoDB, and Cassandra APIs

Cosmos DB offers several APIs over a managed distributed database service. The Core API, commonly called API for NoSQL, is the native JSON document model with SQL-like querying, automatic indexing, stored procedures, transactional batches within a partition key, SDK support, and Change Feed integration. It is the default choice when building new Azure-native document applications.

The MongoDB API supports applications and tools that speak MongoDB wire protocol. It is useful for migrations or teams with MongoDB models and drivers, but compatibility level, feature support, indexing behavior, and operational differences must be tested. The Cassandra API supports Cassandra Query Language patterns and wide-column workloads. It can help teams move selected Cassandra workloads to a managed Azure service, but partitioning and query design remain mandatory.

Note

API compatibility does not eliminate data-model review. A MongoDB or Cassandra application moved to Cosmos DB still needs throughput estimation, partition-key validation,

index review, latency testing, and failure-mode testing.

3.4.2 Change Feed as the Event Bridge

The Change Feed emits inserts and updates in logical partition order and lets processors resume from checkpoints [11]. It is a foundation for event-driven data engineering: write operational items once, then project them to Azure Functions, Event Hubs, ADLS Gen2, Synapse, Databricks, search indexes, caches, and materialized views. The Change Feed Processor library manages leases across workers so processing can scale horizontally.

Change Feed is not a universal audit log. Depending on mode and configuration, deletes may require soft-delete patterns or TTL awareness. Consumers should be idempotent because retries are normal. Downstream systems should store high-water marks or item versions so a processor can safely replay changes after failure.

Common Pitfall

Treating Change Feed consumers as exactly-once. Design them as at-least-once processors with idempotent writes, checkpoints, poison-message handling, and observability.

3.5 Thursday Hands-on Project: Forum Container with Python Writes and Multi-Region Writes

This lab creates a Cosmos DB for NoSQL account, a database, and a single container for a forum application. The schema stores posts, comments, reactions, and moderator events in one container so common forum-page reads can use the same partition key. Run it only in a sandbox subscription and delete the resource group when finished.

Note

The examples are written for Windows PowerShell with Azure CLI. Use globally unique account names, avoid committing keys, and prefer managed identity for production applications.

3.5.1 Single-Container Forum Schema

A forum page usually needs the forum metadata, recent threads, thread starter post, comments, reaction counts, and moderation state. A single-container design can store multiple item types with a shared partition key such as `/forumId`. Item IDs encode type and business key: `thread_1001`, `post_1001_0001`, `reaction_1001_user42`. The `type` field supports filtered queries and indexing. For very large public forums, introduce a synthetic partition key such as `forumId#bucket`.

```
az login
az account set --subscription "00000000-0000-0000-0000-000000000000"

$rg = "rg-de-azure-book-week3"
$primaryLocation = "eastus"
$secondaryLocation = "westus"
$account = "cosmos-week3-forum-001"
$database = "forumdb"
$container = "forumItems"
$partitionKey = "/forumId"

az group create `
```

```
  --name $rg `
  --location $primaryLocation

az cosmosdb create `
  --name $account `
  --resource-group $rg `
  --locations regionName=$primaryLocation failoverPriority=0 isZoneRedundant=
    False `
  --default-consistency-level Session `
  --enable-automatic-failover true

az cosmosdb sql database create `
  --account-name $account `
  --resource-group $rg `
  --name $database

az cosmosdb sql container create `
  --account-name $account `
  --resource-group $rg `
  --database-name $database `
  --name $container `
  --partition-key-path $partitionKey `
  --throughput 1000 `
  --ttl 2592000
```

Listing 3.1: Creating a Cosmos DB for NoSQL account, database, and forum container.

Add a second write region only after the basic account is validated. Multi-region writes affect conflict handling, testing, and cost, so document why the application needs active-active writes instead of single-write failover.

```
az cosmosdb update `
  --name $account `
  --resource-group $rg `
  --locations regionName=$primaryLocation failoverPriority=0 isZoneRedundant=
    False `
              regionName=$secondaryLocation failoverPriority=1 isZoneRedundant=
    False

az cosmosdb update `
  --name $account `
  --resource-group $rg `
  --enable-multiple-write-locations true

az cosmosdb show `
  --name $account `
  --resource-group $rg `
  --query "{name:name,consistency:consistencyPolicy.defaultConsistencyLevel,
    writeLocations:writeLocations[].locationName,enableMultiWrite:
    enableMultipleWriteLocations}" `
  --output table
```

Listing 3.2: Adding a second region and enabling multi-region writes for the lab account.

3.5.2 Inserting Forum Items with Python

Install `azure-cosmos` in a local virtual environment for the lab. Store the endpoint and key in environment variables or use managed identity in production. The script below writes several item types and prints RU charges so students can see how writes consume throughput.

```
import os
from datetime import datetime, timezone
from azure.cosmos import CosmosClient, PartitionKey
```

```

endpoint = os.environ["COSMOS_ENDPOINT"]
key = os.environ["COSMOS_KEY"]
database_name = os.environ.get("COSMOS_DATABASE", "forumdb")
container_name = os.environ.get("COSMOS_CONTAINER", "forumItems")

client = CosmosClient(endpoint, credential=key)
database = client.create_database_if_not_exists(database_name)
container = database.create_container_if_not_exists(
    id=container_name,
    partition_key=PartitionKey(path="/forumId"),
    offer_throughput=1000,
)

now = datetime.now(timezone.utc).isoformat()
items = [
    {
        "id": "thread_1001",
        "forumId": "azure-data-engineering",
        "type": "thread",
        "title": "How do RUs work?",
        "authorId": "user_7",
        "createdAt": now,
        "lastActivityAt": now,
    },
    {
        "id": "post_1001_0001",
        "forumId": "azure-data-engineering",
        "threadId": "thread_1001",
        "type": "post",
        "authorId": "user_7",
        "body": "A point read is cheap; fan-out queries are not.",
        "createdAt": now,
    },
    {
        "id": "reaction_1001_user_42",
        "forumId": "azure-data-engineering",
        "threadId": "thread_1001",
        "postId": "post_1001_0001",
        "type": "reaction",
        "userId": "user_42",
        "reaction": "upvote",
        "createdAt": now,
    },
]

for item in items:
    response = container.upsert_item(item)
    charge = container.client_connection.last_response_headers.get(
        "x-ms-request-charge"
    )
    print(response["id"], "RU charge", charge)
```

Listing 3.3: Writing forum items with the azure-cosmos SDK and recording RU charges.

Query by partition key for predictable RU usage. Avoid broad scans in production APIs; use Change Feed or analytical replicas for downstream processing.

```
query = """
SELECT c.id, c.type, c.title, c.body, c.createdAt
FROM c
WHERE c.forumId = @forumId AND c.threadId = @threadId
ORDER BY c.createdAt
"""
```

```python
parameters = [
    {"name": "@forumId", "value": "azure-data-engineering"},
    {"name": "@threadId", "value": "thread_1001"},
]

for item in container.query_items(
    query=query,
    parameters=parameters,
    partition_key="azure-data-engineering",
):
    print(item)
```

Listing 3.4: Reading a forum thread inside one logical partition.

3.6 Friday Use Case: Global Gaming Leaderboard at 1,000,000 Reads/Writes per Second

3.6.1 Scenario

A global game studio runs a competitive leaderboard for tens of millions of players. During tournaments, the service must handle 1,000,000 combined reads and writes per second globally. Players submit scores from North America, Europe, and Asia. They expect to see their own score immediately after submission, but a leaderboard visible to all players may be seconds behind. The design uses Cosmos DB with multi-region writes and Session consistency.

3.6.2 Partitioning and Serving Strategy

The main container stores score submissions and current leaderboard entries. The partition key uses a synthetic value such as `/boardShard`: `gameId#seasonId#region#bucket`. Buckets distribute writes for a hot tournament. A Change Feed processor maintains materialized top-N documents per game, season, region, and bucket. API reads point to precomputed leaderboard documents instead of running expensive global aggregations on every request.

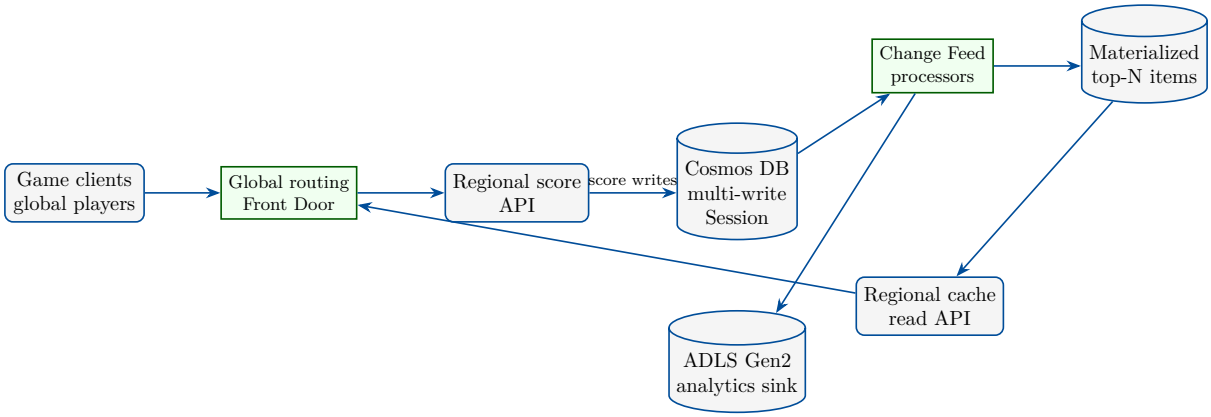


Figure 3.3: Global leaderboard serving architecture using Cosmos DB multi-region writes, Session consistency, Change Feed projections, and regional read paths.

3.6.3 Correctness and Cost Controls

Session consistency ensures a player sees their own submitted score when the SDK preserves the session token. Global top-N displays can be eventually updated through Change Feed

projections. Conflict resolution should be deterministic: for example, keep the highest score for a player and use earliest timestamp as a tie breaker. The API should make score submissions idempotent by using item IDs derived from player, match, and attempt.

Cost control starts with point reads and materialized views. The system should avoid cross-partition aggregation in the hot read path. Autoscale or provisioned throughput must be modeled per region and per container. Observability should track RU consumption, 429 throttling, p99 latency, conflict counts, Change Feed lag, item size, index utilization, and regional availability.

3.7 Architectural Decision Record Template

A Week 3 ADR should capture the access patterns, container strategy, partition key, consistency level, API choice, indexing policy, throughput model, multi-region write decision, conflict policy, TTL, Change Feed consumers, retry rules, and cost guardrails. It should also list rejected alternatives such as Azure SQL, Redis-only storage, Event Hubs plus lake-only storage, DynamoDB, Firestore, MongoDB Atlas, and self-managed Cassandra.

3.8 Operational Deep Dive: Performance, Identity, and Observability

Cosmos DB performance incidents usually come from predictable causes: hot partition keys, cross-partition scans, large items, unbounded arrays, excessive indexing, missing composite indexes, overly strong consistency, retry storms, or Change Feed processors falling behind. Scaling RU/s can help, but it will not fix a partition key that sends all writes to one logical partition.

Identity should prefer Microsoft Entra ID and managed identities where supported by the chosen SDK and API. Production accounts should use private endpoints, role-based access, diagnostic settings, customer-managed keys where required, and alerts on throttling and regional failover. Keys should be rotated and kept out of notebooks, source code, logs, and screenshots.

Common Pitfall

Using Cosmos DB as a reporting warehouse. Large scans and ad hoc aggregations are expensive in an operational NoSQL serving store. Use Change Feed, analytical store, Synapse Link, or lakehouse patterns for broad analytics.

3.9 Troubleshooting Patterns

A Cosmos DB troubleshooting workflow starts with the failing access pattern. For point reads, confirm item ID, partition key, region, consistency, and retry behavior. For writes, check item size, index policy, partition hot spots, conflict rates, and 429 throttling. For queries, inspect whether the query is scoped to one partition, whether it uses an index, whether it requires a composite index, and how many RUs it consumes. For Change Feed, verify leases, checkpoint progress, poison records, and downstream idempotency.

Do not hide 429 responses by adding unbounded retries. Cosmos DB SDKs already include retry policies. Applications should apply backpressure, monitor request charge, and isolate noisy tenants or hot boards before raising throughput blindly.

3.10 Week 3 Lab Rubric

A complete Week 3 submission includes Azure CLI deployment evidence, a documented forum single-container schema, Python insert and query output with RU observations, evidence of multi-region write configuration, and an ADR explaining partitioning, consistency, and Change Feed decisions.

Criterion	Meets expectation	Needs improvement
Partitioning	Partition key matches access patterns and distributes load	Key is low-cardinality or creates hot partitions
Consistency	Five levels are compared and Session is justified for user experience	Strong or Eventual is chosen without correctness analysis
Hands-on	CLI creates account, database, container, and multi-region write setting; Python writes items	Portal-only steps or no RU evidence
Change Feed	Consumers are idempotent and checkpointed with lag monitoring	Feed is treated as exactly-once or audit-complete by default
Global design	Regional writes, conflict policy, routing, and failover behavior are documented	Multi-region writes are enabled without conflict or cost analysis

Table 3.1: Week 3 rubric for Cosmos DB architecture, hands-on implementation, and global serving design.

3.11 Interview Q&A

3.11.1 Concepts and Trade-offs

1. **Q: What is a Request Unit?**
   A: A Request Unit is Cosmos DB's normalized measure of CPU, memory, and IO used by reads, writes, queries, and maintenance operations.

2. **Q: What makes a good partition key?**
   A: High cardinality, even distribution, alignment with common point reads and queries, and support for transaction boundaries within a logical partition.

3. **Q: Why is Session consistency common for global apps?**
   A: It provides read-your-writes for a user session while allowing lower latency and more regional independence than Strong consistency.

4. **Q: When would you avoid multi-region writes?**
   A: Avoid them when the workload cannot tolerate conflict resolution complexity or when single-write failover meets latency and availability goals.

5. **Q: Why use Change Feed for data engineering?**
   A: It turns operational item changes into scalable downstream projections for functions, streams, lakes, caches, search, and materialized views.

6. **Q: What is the biggest risk in a high-scale leaderboard?**
   A: A hot partition or global aggregation path can throttle the system even if the total account RU/s appears high.

3.12 Further Reading

Start with the Azure Cosmos DB overview [38], Request Unit guidance [35], partitioning guidance [33], consistency levels [13], global distribution [19], and multi-region write concepts [12]. Review the Change Feed documentation [11], API for NoSQL quickstart [34], Python SDK documentation [7], and the Azure Well-Architected Framework [28] for reliability, security, performance, operations, and cost review.

Key Takeaways

- Cosmos DB scales by partitioned item access, not by hiding poor data modeling behind more throughput.
- RUs make performance and cost visible; every access pattern should be measured by request charge and latency.
- Partition-key choice determines distribution, transaction scope, query efficiency, and hot-key risk.
- The five consistency levels are business correctness tools, not trivia; Session is often the best global user-experience compromise.
- Change Feed is the bridge from operational NoSQL writes to analytics, search, cache, and materialized-serving projections.
- Multi-region writes require deterministic conflict rules, idempotent clients, observability, and explicit cost justification.

3.13 Exercises

1. **(Conceptual)** Explain RUs to a product manager and show why a cross-partition query can cost more than a point read.
2. **(Conceptual)** Compare Strong, Bounded Staleness, Session, Consistent Prefix, and Eventual consistency for a shopping-cart application.
3. **(Conceptual)** Choose a partition key for a forum with thousands of small private communities and one massive public community.
4. **(Hands-on)** Run the Cosmos DB account and container creation commands in a sandbox and capture the consistency and region output.
5. **(Hands-on)** Modify the Python script to insert comments, reactions, and moderation events, then report RU charges for each item type.
6. **(Hands-on)** Add a query that reads one thread by partition key and compare its RU charge with a cross-partition query.
7. **(Design)** Design a Change Feed consumer that writes forum events to ADLS Gen2 with idempotent replay and checkpoint evidence.
8. **(Design)** Write an ADR for the gaming leaderboard, including partition key, consistency, conflict policy, multi-region writes, and cost controls.

3.14 Capstone Project: Global Gaming Leaderboard on Cosmos DB

Capstone Project

Mission: Design and prototype a global gaming leaderboard on Azure Cosmos DB that uses Session consistency, multi-region writes, synthetic partition keys, Change Feed projections, and operational evidence suitable for a launch review.
Estimated time: 5–7 hours.
What you will practice:

- Modeling a high-scale NoSQL serving workload around point reads, writes, and materialized top-N views.
- Selecting Session consistency and explaining the user-experience guarantee it provides.
- Designing partition keys and buckets to avoid hot logical partitions during tournaments.
- Enabling multi-region writes with deterministic conflict handling and idempotent score submission.
- Using Change Feed to maintain leaderboard projections and analytics exports.

Scenario

A game studio is launching a worldwide ranked tournament. Players submit match scores from the Americas, Europe, and Asia. The public leaderboard must update quickly, but it may lag by a few seconds. Each player must immediately see their own submitted score. The platform must support regional write paths, survive a regional outage, and export score events to the analytics lake.

Requirements

- **Database:** Use Azure Cosmos DB for NoSQL with Session consistency and multi-region writes enabled.
- **Partitioning:** Use a synthetic partition key such as `/boardShard` combining game, season, region, and bucket to distribute writes.
- **Data model:** Store score submissions, current player-best records, leaderboard projection records, and processor checkpoints with clear item types.
- **Conflict handling:** Keep the highest score per player and use earliest timestamp as a deterministic tie breaker.
- **Serving:** Read public leaderboards from materialized top-N records rather than scanning raw score submissions.
- **Change Feed:** Maintain projections and export immutable score events to ADLS Gen2 or a lakehouse landing zone.
- **Operations:** Monitor RU consumption, 429 throttling, p95 and p99 latency, conflict count, regional health, Change Feed lag, and cost.

Reference Architecture

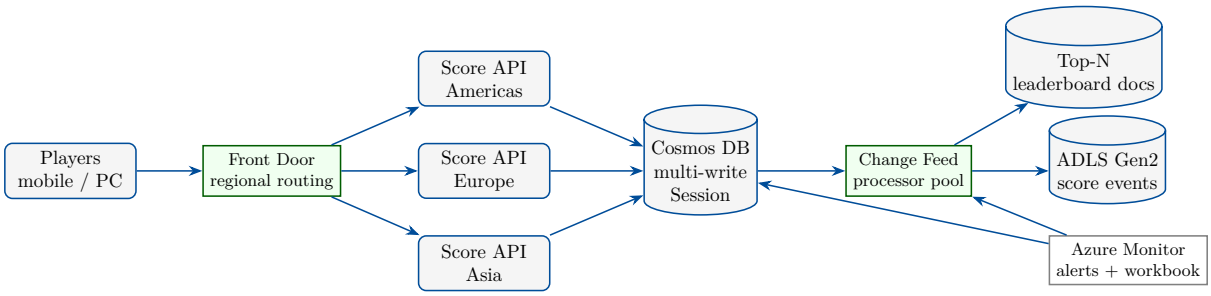


Figure 3.4: Capstone reference architecture for a global leaderboard using regional score APIs, Cosmos DB multi-region writes, Change Feed projections, lake export, and monitoring evidence.

Implementation Steps

Use a sandbox subscription. The snippets below demonstrate the structure; production systems should use infrastructure-as-code, private endpoints, managed identity, and formal capacity tests.

1. **Create a multi-region Cosmos DB account and containers.** Use one container for score items and one container for Change Feed leases if you build a processor.

```
$rg = "rg-cosmos-leaderboard-capstone"
$primaryLocation = "eastus"
$secondaryLocation = "westeurope"
$thirdLocation = "southeastasia"
$account = "cosmos-leaderboard-capstone-001"
$database = "leaderboarddb"
$scores = "scores"
$leases = "leases"

az group create --name $rg --location $primaryLocation

az cosmosdb create `
  --name $account `
  --resource-group $rg `
  --locations regionName=$primaryLocation failoverPriority=0
    isZoneRedundant=False `
              regionName=$secondaryLocation failoverPriority=1
    isZoneRedundant=False `
              regionName=$thirdLocation failoverPriority=2 isZoneRedundant=
    False `
  --default-consistency-level Session `
  --enable-multiple-write-locations true `
  --enable-automatic-failover true

az cosmosdb sql database create `
  --account-name $account `
  --resource-group $rg `
  --name $database

az cosmosdb sql container create `
  --account-name $account `
  --resource-group $rg `
  --database-name $database `
  --name $scores `
  --partition-key-path "/boardShard" `
```

```
  --throughput 4000

az cosmosdb sql container create `
  --account-name $account `
  --resource-group $rg `
  --database-name $database `
  --name $leases `
  --partition-key-path "/id" `
  --throughput 400
```

Listing 3.5: PowerShell plus Azure CLI skeleton for the capstone Cosmos DB account.

2. **Write idempotent score submissions.** The item ID combines player, match, and attempt so retries do not create duplicate submissions.

```
import os
from datetime import datetime, timezone
from azure.cosmos import CosmosClient, PartitionKey

client = CosmosClient(os.environ["COSMOS_ENDPOINT"], os.environ["COSMOS_KEY"])
db = client.create_database_if_not_exists("leaderboarddb")
scores = db.create_container_if_not_exists(
    id="scores",
    partition_key=PartitionKey(path="/boardShard"),
    offer_throughput=4000,
)

def board_shard(game_id, season_id, region, player_id, buckets=64):
    bucket = abs(hash(player_id)) % buckets
    return f"{game_id}#{season_id}#{region}#{bucket:02d}"

def submit_score(game_id, season_id, region, player_id, match_id, attempt, value):
    timestamp = datetime.now(timezone.utc).isoformat()
    item = {
        "id": f"score_{player_id}_{match_id}_{attempt}",
        "type": "scoreSubmission",
        "gameId": game_id,
        "seasonId": season_id,
        "region": region,
        "playerId": player_id,
        "matchId": match_id,
        "attempt": attempt,
        "score": value,
        "submittedAt": timestamp,
        "boardShard": board_shard(game_id, season_id, region, player_id),
    }
    return scores.upsert_item(item)

for i in range(10):
    saved = submit_score("racer", "2026-summer", "americas", f"player_{i}", "m77", 1, 1000 + i)
    print(saved["id"], saved["boardShard"])
```

Listing 3.6: Submitting leaderboard scores with deterministic IDs and synthetic partition keys.

3. **Build the projection worker.** Read changes, compute player-best and bucket top-N records, and write projection items with idempotent upserts. Store checkpoints through the Change Feed Processor or an equivalent lease mechanism.

4. **Validate consistency and conflict behavior.** Submit a score, read it back with the same SDK session, then test concurrent submissions for the same player and confirm the

deterministic winner.

5. **Create operational evidence.** Capture RU charge per operation, 429 count, p99 latency, regional write status, Change Feed lag, conflict count, and a cost estimate for normal and tournament traffic.

Deliverables

- Architecture diagram showing clients, routing, regional APIs, Cosmos DB regions, Change Feed processors, top-N projection items, lake export, and monitoring.
- ADR explaining why Cosmos DB for NoSQL, Session consistency, multi-region writes, and the synthetic partition key were selected.
- Reproducible Azure CLI or infrastructure-as-code snippets for account, database, containers, regions, consistency, throughput, and diagnostic settings.
- Python scripts or pseudocode for idempotent score submission, partition-key generation, and projection updates.
- Evidence pack with RU measurements, load-test assumptions, throttling behavior, conflict tests, Change Feed lag, and failover observations.

Acceptance Criteria

- The design explains why Session consistency satisfies player read-your-writes while allowing global leaderboard projections to lag briefly.
- Partition keys distribute tournament writes across buckets and avoid a single hot logical partition.
- Public leaderboard reads use materialized top-N items rather than cross-partition scans of raw score submissions.
- Score submissions are idempotent and conflict resolution is deterministic.
- Multi-region write behavior, failover assumptions, RPO, and regional routing are documented.
- Change Feed processing is idempotent, checkpointed, monitored for lag, and safe to replay.
- Security design avoids committed keys and addresses private access, managed identity or least-privilege roles, and diagnostic logging.
- Operational evidence is complete enough for another engineer to reproduce the lab with different names and regions.

Stretch Goals

- Add a synthetic load script that estimates RU/s for 1,000,000 combined reads and writes per second across regions.
- Implement custom conflict resolution logic or an application-level merge function for player-best records.
- Add Azure Functions or a containerized worker that processes Change Feed leases and writes top-N projection documents.
- Export immutable score submissions to ADLS Gen2 in partitioned folders for downstream analytics.

- Create an Azure Monitor workbook that displays RU charge, 429s, latency percentiles, region health, conflict counts, and Change Feed lag.
- Estimate cost for provisioned throughput, autoscale, and serverless variants under normal day and tournament traffic.

Chapter 4

Hybrid Ingestion and Caching

Learning Objectives

- Explain why low-latency data products need caching, admission control, TTLs, and clear invalidation rules.
- Use Azure Cache for Redis to accelerate Azure SQL Database reads with a cache-aside pattern.
- Compare Redis-based caching with Cosmos DB Integrated Cache for read-heavy document workloads.
- Position Azure API Management as an ingestion gateway for data APIs, mobile applications, partners, and event producers.
- Deploy an Azure Cache for Redis instance and write a Python cache-aside script over Azure SQL Database query results.
- Design a high-throughput, low-latency mobile data API that survives extreme traffic spikes during live sports events.
- Complete the Part I milestone by integrating Azure SQL Database Serverless, Cosmos DB, ADLS Gen2, Redis, APIM, and Change Feed.

Cross-Cloud Equivalence

Caching is a portable architecture pattern even when product names differ. The engineer must define what is cached, where the cache lives, how entries expire, how writes update or invalidate entries, and what happens when the cache is unavailable.

Concept	Azure	AWS	GCP	Other platforms
Managed Redis	Azure Cache for Redis with Enterprise tiers for advanced Redis features	Amazon ElastiCache for Redis or MemoryDB for Redis	Memorystore for Redis Cluster or Memorystore for Redis	Redis Enterprise Cloud, self-managed Redis, KeyDB
Database-integrated cache	Cosmos DB Integrated Cache on dedicated gateway	DynamoDB Accelerator (DAX) for DynamoDB	No exact single-service equivalent; combine Memorystore with Firestore or Spanner patterns	MongoDB Atlas tiered storage plus application caches; Cassandra row cache
Cache-aside	Application reads Redis first, then Azure SQL, Cosmos DB, or API source on miss	Application reads ElastiCache first, then RDS, DynamoDB, or service source	Application reads Memorystore first, then Cloud SQL, Firestore, or Spanner	Common pattern across Redis, Memcached, Hazelcast, and application memory
Write-through / write-behind	Application or service updates cache with the write path; less common for Azure PaaS databases than cache-aside	ElastiCache libraries or custom write-through paths	Custom service layer with Memorystore and database writes	Product-specific data grids and stream processors
TTL and eviction	Redis TTL, maxmemory policies, and Cosmos DB Integrated Cache staleness window	Redis TTL, eviction policies, DAX item cache TTL	Memorystore TTL and eviction policies	Memcached expiration, Redis-compatible TTL, local LRU caches

A cloud-neutral cache design documents freshness, ownership, invalidation, fallback, warm-up, and observability. Product choice matters, but these contracts matter more.

Key Terms

Cache hit: a request served from cache without contacting the origin database. **Cache miss**: a request that falls through to the origin and may populate the cache. **TTL**: time-to-live, the maximum age of a cached entry. **Eviction**: removal of cached entries due to memory pressure or policy. **Ingestion gateway**: an API boundary that validates, throttles, authenticates, and routes incoming data.

4.1 Week 4 Overview: Why Caching and Hybrid Ingestion Close Part I

Part I has introduced storage foundations, relational databases, and document-scale NoSQL. Week 4 adds the serving patterns that make those systems usable under real product traffic: cache hot reads, shield databases from spikes, and place a governed API gateway in front of ingestion paths. Azure Cache for Redis provides managed in-memory acceleration for databases and services [40]. Cosmos DB Integrated Cache reduces RU consumption for read-heavy API for NoSQL workloads that use the dedicated gateway [25]. Azure API Management provides the policy and gateway layer for data APIs [39].

Caching is not only a performance trick. It is an architectural contract. A cache introduces a second copy of data, so the team must explain freshness, invalidation, fallback, and observability. Done well, caching reduces database load and user-visible latency. Done poorly, it hides stale data, creates retry storms, and makes incidents harder to reason about.

Definition

Hybrid ingestion combines synchronous API ingestion, operational databases, caches, and asynchronous downstream flows. It is common when mobile apps, partner systems, and web backends need low-latency writes while analytics systems need durable event history.

Note

This chapter closes Milestone 1. The final capstone combines the storage, relational, NoSQL, and caching concepts from Weeks 1–4 into an e-commerce data backend.

4.2 Monday: Azure Cache for Redis for Database Acceleration

4.2.1 What Redis Adds to a Data Platform

Azure Cache for Redis is a managed, in-memory key-value store based on Redis. Data engineers use it for hot database query results, session state, rate-limit counters, distributed locks, pub/sub notifications, feature flags, and small materialized views. For database acceleration, the usual goal is simple: keep the most frequently requested objects close to the application so Azure SQL Database or another origin can focus on writes, uncommon reads, and durable transactions.

Redis is fast because it keeps data in memory and uses simple data structures. That speed does not remove the need for durable storage. Treat Redis as an acceleration layer unless the tier and architecture explicitly provide the persistence and availability characteristics your workload requires.

Definition

Cache-aside is a pattern where the application looks in cache first. On a miss, it reads from the origin database, stores the result in cache with a TTL, and returns the value to the caller.

4.2.2 Cache-Aside Read Path

The cache-aside pattern is the default starting point for Azure SQL acceleration because it keeps the origin database authoritative. The application owns cache keys and serialization. The cache can be flushed without data loss because Azure SQL remains the system of record.

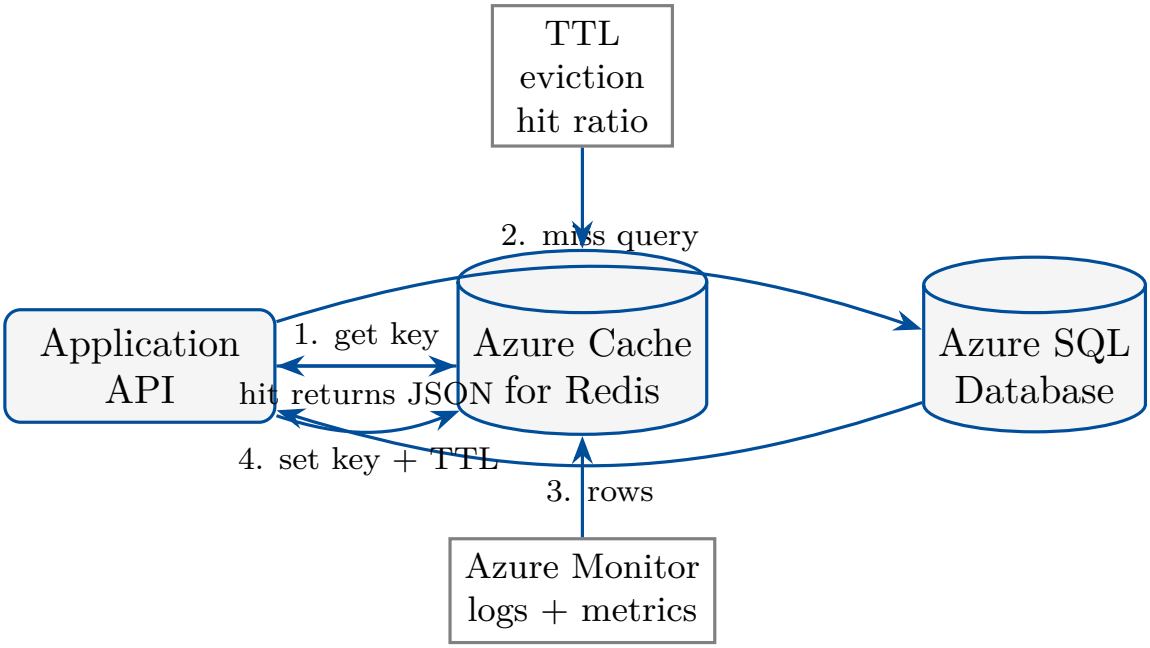


Figure 4.1: Cache-aside between an application, Azure Cache for Redis, and Azure SQL Database keeps Azure SQL authoritative while Redis absorbs hot reads.

4.2.3 Keys, TTLs, and Invalidation

Good cache keys are stable, specific, and versionable. A profile query might use `profile:v1:user:42`. A product-card query might use `product-card:v3:sku:ABC123`. Version prefixes let teams invalidate a family of entries by changing the key format during deployment. TTLs bound staleness and prevent old entries from living forever.

Invalidation is the difficult part. For read-mostly reference data, a five-minute TTL may be enough. For product prices, billing status, inventory reservations, and permissions, the write path should delete or update affected cache keys immediately. If the application cannot name every affected key, use short TTLs and design user experiences that tolerate brief staleness.

Common Pitfall

Caching data without a freshness contract. A cache entry with no TTL and no invalidation rule is a future incident. Every cached object needs an owner, expiration, and fallback behavior.

4.2.4 Availability and Failure Modes

A database acceleration cache must fail safely. If Redis is unavailable, the application should usually fall back to Azure SQL for critical reads, apply backpressure to avoid overwhelming the database, and record degraded-mode metrics. Do not allow thousands of application instances to stampede the database on the same cache miss. Use request coalescing, jittered TTLs, and sensible retries.

Tip

Track cache hit ratio, server load, memory pressure, evicted keys, connected clients, command latency, errors, and database query rate together. A high hit ratio that serves stale or wrong data is not success.

4.3 Tuesday: Cosmos DB Integrated Cache

4.3.1 Why an Integrated Cache Exists

Cosmos DB already provides low-latency reads, but every read consumes RUs unless served by the Integrated Cache through the dedicated gateway. Integrated Cache stores point reads and query results in a gateway-side cache. Repeated reads can return without consuming RUs, subject to the configured maximum integrated cache staleness [25].

This is different from Redis cache-aside. The application does not manually set keys for Integrated Cache. Instead, it sends supported requests through the dedicated gateway and accepts a staleness window. The operational model is closer to a database-integrated read cache than to an application-owned key-value cache.

Definition

Maximum integrated cache staleness defines how old a cached Cosmos DB response may be before the gateway must refresh it from the backend partitions.

4.3.2 When Integrated Cache Fits

Integrated Cache fits read-heavy document workloads where the same point reads or queries repeat often and slight staleness is acceptable. Examples include catalog pages, product recommendations, configuration documents, reference metadata, and public leaderboard projections. It is less useful for write-heavy workloads, unique one-off queries, cross-partition scans with low repetition, or correctness paths that require the newest committed item.

The cache can reduce RU cost and latency, but it does not fix bad data modeling. A query that fans out across partitions may still be an expensive origin operation when the cache misses. Partition keys, item size, index policy, and materialized views remain central to design.

Note

Use Redis when the application needs explicit keys, counters, sorted sets, rate limiting, pub/sub, or cross-origin caching. Use Cosmos DB Integrated Cache when repeated Cosmos DB reads can accept bounded staleness and the dedicated gateway model fits the application.

4.3.3 Consistency and Freshness Trade-offs

Integrated Cache is a deliberate freshness trade. If a user updates a shopping cart and immediately reads through a stale cache path, the application might show the previous state until the staleness window expires. Some workloads can tolerate this; others cannot. Critical user-owned writes should be read with session-aware direct reads or cache bypass where needed.

Common Pitfall

Assuming Integrated Cache has the same semantics as Session consistency. The cache has a staleness setting. Cosmos DB consistency still matters, but the cached response may intentionally be older than the newest backend value.

4.4 Wednesday: Azure API Management for Data Ingestion

4.4.1 APIM as the Data API Front Door

Azure API Management is a managed gateway for publishing, securing, transforming, throttling, and observing APIs. In data engineering, APIM often sits in front of ingestion endpoints used by mobile apps, partner systems, internal services, IoT gateways, and batch jobs. It provides a controlled boundary before traffic reaches Azure Functions, App Service, Container Apps, Event Hubs, Azure SQL, Cosmos DB, or custom services.

The gateway should not become a hidden monolith. Keep APIM responsible for boundary concerns: authentication, authorization, subscription keys, quotas, rate limits, schema validation, request transformation, routing, response shaping, and logging. Keep business validation and durable writes in backend services where they can be tested and versioned.

Definition

Ingestion gateway is the controlled API boundary that receives external or cross-team data, enforces policy, and routes validated requests to backend ingestion services.

4.4.2 Policies for Data Ingestion

APIM policies can validate JWTs, check subscription keys, limit call rates, reject oversized payloads, require headers, transform JSON, route versions, cache selected responses, and emit diagnostics. For ingestion, the most important policies are often admission-control policies. A backend cannot maintain data quality if a gateway accepts unlimited malformed requests during a traffic spike.

Typical APIM controls include per-consumer quotas, burst rate limits, request size limits, schema validation, correlation IDs, and backend timeout policies. For write APIs, avoid response caching unless the semantics are explicitly idempotent. For read APIs, APIM response caching may complement Redis or Integrated Cache, but it must follow the same freshness contract.

Tip

Use APIM to protect the backend before scaling the backend. Rejecting invalid or abusive traffic at the edge is cheaper and safer than letting it consume database RUs, DTUs, vCores, or queue capacity.

4.4.3 Synchronous and Asynchronous Ingestion

Hybrid ingestion often uses APIM for synchronous request acceptance and Event Hubs, queues, or Change Feed for asynchronous processing. A mobile app may POST a clickstream event through APIM, receive a quick accepted response, and let a backend write to Cosmos DB or Event Hubs. A partner may submit billing adjustments that require synchronous validation against Azure SQL. The correct pattern depends on user feedback, correctness, ordering, and replay requirements.

Note

APIM is not a message broker. If the system needs durable buffering, replay, or consumer groups, route accepted requests to Event Hubs, Service Bus, Storage queues, Cosmos DB, or another durable ingestion store.

4.5 Thursday Hands-on Project: Redis Cache-Aside for Azure SQL Query Results

This lab deploys Azure Cache for Redis and uses Python to cache Azure SQL Database query results. Run it only in a sandbox subscription. Delete the resource group when finished, and never commit database passwords, connection strings, or Redis access keys.

Note

The examples are written for Windows PowerShell with Azure CLI. Production systems should use managed identity for Azure SQL where possible, private endpoints, Key Vault, and infrastructure-as-code.

4.5.1 Deploying Azure Cache for Redis

The snippet below creates a resource group and a Basic C0 Redis instance for learning. Basic is not appropriate for production availability, but it keeps the lab small. Use Standard, Premium, or Enterprise tiers for real workloads that require replication, clustering, persistence, private networking, or stronger availability characteristics.

```
az login
az account set --subscription "00000000-0000-0000-0000-000000000000"
$rg = "rg-de-azure-book-week4"
$location = "eastus"
$redis = "redis-week4-sqlcache-001"

az group create --name $rg --location $location

az redis create `
  --name $redis --resource-group $rg --location $location `
  --sku Basic --vm-size c0 `
  --enable-non-ssl-port false

az redis show `
  --name $redis --resource-group $rg `
  --query "{name:name,hostName:hostName,sku:sku.name,sslPort:sslPort}" `
  --output table

az redis list-keys `
  --name $redis --resource-group $rg --query primaryKey --output tsv
```

Listing 4.1: Creating an Azure Cache for Redis instance for the Week 4 lab.

4.5.2 Caching Azure SQL Results with Python

Install `redis` and `pyodbc` in a local virtual environment. The script reads a user profile from Azure SQL Database, serializes the result as JSON, stores it in Redis with a TTL, and prints whether the response was a cache hit or miss.

```
import json
import os
import time

import pyodbc
import redis

CACHE_TTL_SECONDS = int(os.environ.get("CACHE_TTL_SECONDS", "300"))

cache = redis.Redis(
    host=os.environ["REDIS_HOST"],
```

```
    port=6380,
    password=os.environ["REDIS_KEY"],
    ssl=True,
    decode_responses=True,
)

def cache_key(user_id: int) -> str:
    return f"profile:v1:user:{user_id}"

def fetch_profile_from_sql(user_id: int) -> dict | None:
    query = """
    SELECT UserId, DisplayName, LoyaltyTier, UpdatedAt
    FROM dbo.UserProfiles
    WHERE UserId = ?
    """
    with pyodbc.connect(os.environ["SQL_CONNECTION_STRING"], timeout=5) as
    connection:
        row = connection.execute(query, user_id).fetchone()
    if row is None:
        return None
    return {
        "userId": row.UserId,
        "displayName": row.DisplayName,
        "loyaltyTier": row.LoyaltyTier,
        "updatedAt": row.UpdatedAt.isoformat(),
    }

def get_profile(user_id: int) -> dict | None:
    key = cache_key(user_id)
    cached = cache.get(key)
    if cached:
        print("cache hit", key)
        return json.loads(cached)

    print("cache miss", key)
    profile = fetch_profile_from_sql(user_id)
    if profile is not None:
        cache.setex(key, CACHE_TTL_SECONDS, json.dumps(profile))
    return profile

if __name__ == "__main__":
    start = time.perf_counter()
    result = get_profile(int(os.environ.get("USER_ID", "42")))
    elapsed_ms = (time.perf_counter() - start) * 1000
    print(json.dumps(result, indent=2))
    print(f"elapsed_ms={elapsed_ms:.1f}")
```

Listing 4.2: Cache-aside Python script using redis-py and Azure SQL query results.

4.5.3 Measuring the Load Reduction

Set `REDIS_HOST`, `REDIS_KEY`, `SQL_CONNECTION_STRING`, and `USER_ID` in PowerShell, install `redis` and `pyodbc`, then run the script twice. The first run should miss the cache and query Azure SQL. The second run should hit Redis until the TTL expires. Capture Azure SQL query duration, Redis latency, cache hit ratio, and database CPU before and after repeated reads.

4.6 Friday Use Case: Mobile Sports API with Extreme Traffic Spikes

4.6.1 Scenario

A sports media company runs a mobile app for live games. Traffic is normal during the week, then spikes by 50 times when a championship game starts, when a star player scores, or when fantasy-league results update. Users expect low-latency reads for scores, rosters, schedules, profile settings, and personalized offers. The backend must protect Azure SQL and Cosmos DB from sudden read amplification.

4.6.2 Serving Architecture

APIM sits at the public API boundary. Redis caches hot scoreboards, team pages, user profile summaries, and rate-limit counters. Cosmos DB stores high-volume event and personalization documents, with Integrated Cache for repeated public document reads where staleness is acceptable. Azure SQL stores billing, subscriptions, and relational user-profile records. The write path invalidates or refreshes affected cache entries after authoritative updates.

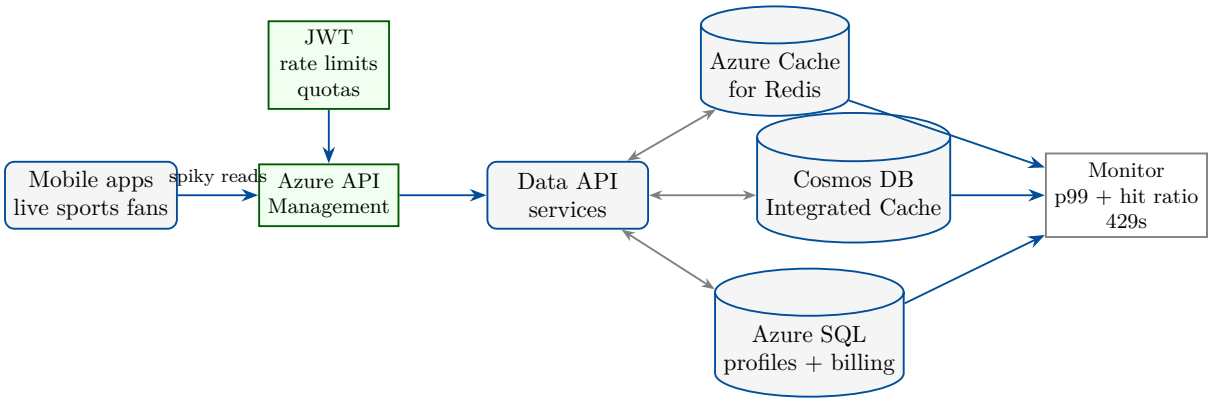


Figure 4.2: Spiky-traffic mobile data API using APIM policy controls, Redis hot-path acceleration, Cosmos DB Integrated Cache, Azure SQL, and shared observability.

4.6.3 Design Decisions

The live scoreboard read path should be cache-first with very short TTLs, often one to five seconds, because users value low latency and tolerate tiny delays. User billing status should have stricter invalidation because access control and payments require correctness. Personalized offers may use Redis or Cosmos DB Integrated Cache with a longer TTL if stale recommendations are acceptable.

APIM rate limits should distinguish anonymous public reads, authenticated users, partners, and internal services. The backend should degrade gracefully: serve slightly stale public scoreboards, disable nonessential personalization, and preserve billing and identity correctness. During the spike, observability should show p95 and p99 latency, cache hit ratio, Redis server load, Cosmos DB RU consumption, Azure SQL CPU, APIM rejected requests, and backend error rates.

4.7 Architectural Decision Record Template

A Week 4 ADR should capture each cached object, source of truth, key format, TTL, invalidation trigger, stale-read tolerance, fallback behavior, ownership, and metrics. It should also document APIM policies, consumer tiers, rate limits, authentication, backend routing, payload limits, idempotency, and rejected alternatives such as database-only reads, CDN-only caching, direct partner database access, or queue-only ingestion.

4.8 Operational Deep Dive: Stampedes, Security, and Observability

Cache incidents often come from stampedes, hot keys, overlarge values, missing TTLs, network isolation gaps, secret leakage, and retry storms. A stampede occurs when many callers miss the same key and all query the origin. Prevent it with jittered TTLs, request coalescing, background refresh, small stale-while-revalidate windows where acceptable, and backend concurrency limits.

Security follows the same principles as earlier chapters. Do not commit Redis keys, SQL passwords, connection strings, or APIM subscription keys. Prefer managed identity for backend services, Key Vault for secrets, private endpoints for production, TLS-only connections, least-privilege access, and diagnostic settings that do not log sensitive payloads.

Common Pitfall

Letting cache success hide origin weakness. If the origin database cannot survive a cold start, regional failover, or cache flush, the architecture needs backpressure and capacity planning before production launch.

4.9 Troubleshooting Patterns

Start with the user symptom: stale data, high latency, wrong data, database overload, or gateway rejection. For stale data, inspect TTL, invalidation, key version, and whether the request is using Redis, Integrated Cache, APIM cache, or a direct database path. For latency, separate APIM gateway time, backend service time, Redis command latency, Cosmos DB RU and gateway metrics, and Azure SQL query duration.

For wrong data, confirm cache keys include all dimensions that affect the result: tenant, user, locale, permissions, API version, and product version. For overload, check whether cache misses are correlated, whether APIM limits are too loose, whether database retries are multiplying traffic, and whether a single hot key is saturating Redis.

4.10 Week 4 Lab Rubric

A complete Week 4 submission includes Azure CLI deployment evidence for Redis, a working Python cache-aside script, documented SQL source query, before-and-after latency observations, cache hit and miss evidence, and an ADR explaining TTLs, invalidation, fallback, and APIM ingestion controls.

Criterion	Meets expectation	Needs improvement
Cache design	Keys, TTLs, invalidation, fallback, and owner are documented	Cache is added without freshness or ownership rules
Hands-on	Azure CLI deploys Redis; Python shows miss then hit over Azure SQL	Portal-only deployment or no measurable cache-hit evidence
Correctness	Sensitive or transactional data has stricter invalidation or bypass rules	Billing, permissions, or inventory rely on stale cache entries
APIM ingestion	Authentication, rate limits, quotas, payload limits, and diagnostics are specified	Gateway only forwards traffic without admission controls
Operations	Hit ratio, latency, memory, evictions, errors, SQL load, and RU usage are monitored together	Only average latency or a single service metric is reported

Table 4.1: Week 4 rubric for caching, hybrid ingestion, hands-on implementation, and operational evidence.

4.11 Interview Q&A

4.11.1 Concepts and Trade-offs

1. **Q: What is the cache-aside pattern?**
   A: The application checks cache first, reads the origin on a miss, stores the result with a TTL, and returns the value.

2. **Q: Why is Redis not usually the system of record?**
   A: It is optimized for in-memory speed. Durable databases should own authoritative state unless the Redis tier and design explicitly meet durability requirements.

3. **Q: When should you prefer Cosmos DB Integrated Cache over Redis?**
   A: When repeated Cosmos DB reads can use the dedicated gateway and accept a bounded staleness window without application-managed cache keys.

4. **Q: What causes a cache stampede?**
   A: Many callers miss the same key at once and all query the origin, often after a popular key expires or the cache is flushed.

5. **Q: What belongs in APIM for ingestion?**
   A: Authentication, authorization, quotas, rate limits, validation, transformations, routing, timeout policy, and diagnostics at the API boundary.

6. **Q: What metric is more useful than cache hit ratio alone?**
   A: Hit ratio combined with p95 and p99 latency, origin load, stale-read incidents, errors, evictions, and business correctness metrics.

4.12 Further Reading

Start with Azure Cache for Redis concepts and planning guidance [40], Redis best practices [10], Cosmos DB Integrated Cache [25], dedicated gateway guidance [16], and Azure API Management overview and policy documentation [39, 6]. Review Azure SQL Database [45], Cosmos DB Change Feed [11], ADLS Gen2 [27], and the Azure Well-Architected Framework [28] for the milestone architecture review.

Key Takeaways

- Caches improve latency and reduce origin load only when freshness, invalidation, and fallback are explicit.
- Azure Cache for Redis is the general-purpose in-memory acceleration layer for hot keys, counters, sessions, and query results.
- Cosmos DB Integrated Cache reduces repeated-read RU cost when bounded staleness and the dedicated gateway fit the workload.
- APIM protects ingestion backends with authentication, quotas, rate limits, validation, transformations, routing, and diagnostics.
- Cache stampedes, hot keys, secret leakage, stale permission data, and retry storms are common production risks.
- Milestone 1 combines durable storage, relational integrity, document scale, hot-path caching, and event-driven analytics export.

4.13 Exercises

1. **(Conceptual)** Explain cache-aside to a product manager and describe why Azure SQL remains the source of truth.
2. **(Conceptual)** Compare Redis and Cosmos DB Integrated Cache for a product catalog API with repeated public reads.
3. **(Conceptual)** Choose TTLs for live scores, product prices, user permissions, and marketing recommendations.
4. **(Hands-on)** Deploy Azure Cache for Redis in a sandbox and capture hostname, SKU, TLS port, and deletion plan.
5. **(Hands-on)** Run the Python cache-aside script twice and record cache miss latency, cache hit latency, and SQL query count.
6. **(Hands-on)** Add key invalidation after an Azure SQL profile update and prove that the next read refreshes Redis.
7. **(Design)** Write an APIM policy plan for a partner ingestion API, including authentication, quotas, payload limits, and correlation IDs.
8. **(Design)** Create an ADR for the live sports mobile API, including cached objects, TTLs, invalidation, fallback, and observability.

4.14 Milestone 1 Capstone: E-Commerce Multi-Tier Data Backend

Capstone Project

Mission: Design and prototype an e-commerce multi-tier data backend that integrates Weeks 1–4: Azure SQL Database Serverless for user profiles and billing, Cosmos DB for active shopping carts and clickstream tracking, ADLS Gen2 for product images with lifecycle and encryption controls, Azure Cache for Redis for hot-path acceleration, and Cosmos DB Change Feed to push events to storage for analytics.

Estimated time: 8–10 hours.
What you will practice:

- Combining relational, document, object-storage, cache, API, and event-driven patterns in one reference architecture.
- Separating systems of record from acceleration layers and analytics sinks.
- Designing hot-path reads, cart writes, image storage, billing correctness, and downstream clickstream analytics.
- Producing implementation evidence, operational metrics, and an architecture decision record suitable for a milestone review.

Scenario

An e-commerce company is preparing for a seasonal launch. Customers browse product images, sign in, update profiles, add items to carts, receive personalized recommendations, and complete billing workflows. Marketing expects unpredictable spikes after social media announcements. The platform must keep user-facing APIs fast while preserving billing correctness and exporting behavioral events for analytics.

Requirements

- **User profiles and billing:** Use Azure SQL Database Serverless for relational user, address, payment-status, invoice, and billing-reference data.
- **Carts and clickstream:** Use Cosmos DB for active shopping carts, cart item updates, clickstream events, and short-lived personalization state.
- **Images:** Store product images in ADLS Gen2 or Blob Storage with hierarchical namespaces where needed, lifecycle policies, encryption, and least-privilege access.
- **Caching:** Use Azure Cache for Redis for product cards, profile summaries, feature flags, and hot campaign landing-page data.
- **Analytics export:** Use Cosmos DB Change Feed to write cart and clickstream events to ADLS Gen2 in partitioned folders.
- **Gateway:** Use APIM to protect public and partner APIs with authentication, rate limits, quotas, request validation, and diagnostics.
- **Operations:** Monitor p95 and p99 latency, Redis hit ratio, SQL CPU and auto-pause behavior, Cosmos DB RU consumption, Change Feed lag, storage lifecycle transitions, and API rejections.

Reference Architecture

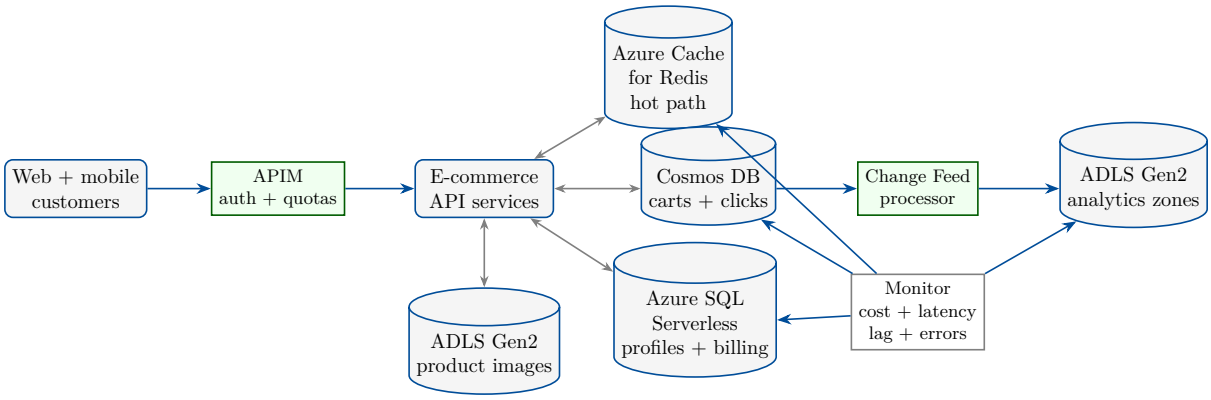


Figure 4.3: Milestone 1 reference architecture integrating ADLS Gen2, Azure SQL Database Serverless, Cosmos DB, Redis, APIM, and Change Feed analytics export.

Implementation Steps

Use a sandbox subscription and globally unique names. The snippets show a reproducible skeleton; production deployments should use Bicep, Terraform, or another reviewed infrastructure-as-code workflow.

1. **Create core resources.** Deploy one resource group, a storage account with hierarchical namespace, Azure SQL logical server and serverless database, Cosmos DB database and containers, and Azure Cache for Redis.

```
$rg = "rg-ecommerce-milestone1"
$location = "eastus"
$storage = "stecommilestone1001"
$sqlServer = "sql-ecomm-milestone-001"
$sqlDb = "profilesbilling"
$cosmos = "cosmos-ecomm-milestone-001"
$cosmosDb = "ecommerce"
$cartContainer = "carts"
$clickContainer = "clickstream"
$redis = "redis-ecomm-milestone-001"
$adminUser = "sqladminuser"
$adminPassword = "<use-Key-Vault-or-secure-prompt>"

az group create --name $rg --location $location

az storage account create `
  --name $storage --resource-group $rg --location $location `
  --sku Standard_LRS --kind StorageV2 --hierarchical-namespace true `
  --min-tls-version TLS1_2 `
  --allow-blob-public-access false

az sql server create `
  --name $sqlServer --resource-group $rg --location $location `
  --admin-user $adminUser --admin-password $adminPassword

az sql db create `
  --resource-group $rg --server $sqlServer --name $sqlDb `
  --compute-model Serverless --edition GeneralPurpose `
  --family Gen5 --capacity 2

az cosmosdb create `
```

```
  --name $cosmos --resource-group $rg `
  --locations regionName=$location failoverPriority=0 isZoneRedundant=False
    `
  --default-consistency-level Session

az cosmosdb sql database create `
  --account-name $cosmos --resource-group $rg --name $cosmosDb

az cosmosdb sql container create `
  --account-name $cosmos --resource-group $rg `
  --database-name $cosmosDb --name $cartContainer `
  --partition-key-path "/userId" --throughput 1000 --ttl 604800

az cosmosdb sql container create `
  --account-name $cosmos --resource-group $rg `
  --database-name $cosmosDb --name $clickContainer `
  --partition-key-path "/eventShard" --throughput 1000 --ttl 2592000

az redis create `
  --name $redis --resource-group $rg --location $location `
  --sku Basic --vm-size c0 `
  --enable-non-ssl-port false
```

Listing 4.3: Azure CLI skeleton for the Milestone 1 e-commerce backend.

2. **Prototype hot-path reads and event writes.** Use Redis for product-card reads and Cosmos DB for cart and clickstream writes. The sketch below shows the boundary between source-of-truth stores and acceleration.

```
import os
from datetime import datetime, timezone

import redis
from azure.cosmos import CosmosClient, PartitionKey

cache = redis.Redis(
    host=os.environ["REDIS_HOST"],
    port=6380,
    password=os.environ["REDIS_KEY"],
    ssl=True,
    decode_responses=True,
)

cosmos = CosmosClient(os.environ["COSMOS_ENDPOINT"], os.environ["COSMOS_KEY
    "])
db = cosmos.create_database_if_not_exists("ecommerce")
carts = db.create_container_if_not_exists(
    id="carts",
    partition_key=PartitionKey(path="/userId"),
    offer_throughput=1000,
)
clicks = db.create_container_if_not_exists(
    id="clickstream",
    partition_key=PartitionKey(path="/eventShard"),
    offer_throughput=1000,
)

def product_key(sku: str) -> str:
    return f"product-card:v1:sku:{sku}"

def get_product_card(sku: str, load_from_sql_or_catalog):
    key = product_key(sku)
    cached = cache.get(key)
```

```
    if cached:
        return cached
    product = load_from_sql_or_catalog(sku)
    cache.setex(key, 300, product)
    return product

def add_to_cart(user_id: str, sku: str, quantity: int):
    now = datetime.now(timezone.utc).isoformat()
    return carts.upsert_item({
        "id": f"cart_{user_id}_{sku}",
        "type": "cartItem",
        "userId": user_id,
        "sku": sku,
        "quantity": quantity,
        "updatedAt": now,
    })

def record_click(user_id: str, sku: str, action: str):
    now = datetime.now(timezone.utc).isoformat()
    shard = f"{now[:10]}#{abs(hash(user_id)) % 32:02d}"
    return clicks.create_item({
        "id": f"click_{user_id}_{sku}_{now}",
        "type": "clickstream",
        "userId": user_id,
        "sku": sku,
        "action": action,
        "eventTime": now,
        "eventShard": shard,
    })
```

Listing 4.4: Python sketch for product-card caching and clickstream writes.

3. **Configure image governance.** Create containers or file systems for product images, define lifecycle movement to cool or archive tiers, document encryption settings, and assign least-privilege roles to upload and serving identities.
4. **Implement Change Feed export.** Build or describe a Change Feed processor that reads cart and clickstream changes, writes JSON or Parquet files to ADLS Gen2 by date and event type, and stores checkpoints or leases.
5. **Add APIM policy evidence.** Document authentication, rate limits, quotas, request size limits, correlation IDs, backend timeouts, version routing, and diagnostic settings.
6. **Validate the hot path.** Demonstrate Redis miss then hit for product-card reads, point reads and writes for carts, SQL correctness for billing data, and Change Feed output arriving in storage.

Deliverables

- Architecture diagram showing APIM, API services, Azure SQL Database Serverless, Cosmos DB, Redis, ADLS Gen2 images, Change Feed processing, analytics storage, and monitoring.
- ADR explaining system-of-record boundaries, partition keys, TTLs, invalidation, APIM policies, storage lifecycle, and analytics export.
- Reproducible Azure CLI or infrastructure-as-code snippets for the core resources.
- Python scripts or pseudocode for cache-aside product reads, cart writes, clickstream event writes, and Change Feed export.

- Evidence pack with cache hit and miss output, SQL serverless configuration, Cosmos DB RU observations, storage lifecycle policy, and monitoring screenshots or query outputs.

Acceptance Criteria

- Azure SQL Database Serverless owns user profile and billing records; cache entries never become authoritative for billing correctness.
- Cosmos DB containers use partition keys that support cart point reads and distributed clickstream writes.
- ADLS Gen2 image storage includes lifecycle, encryption, access-control, and deletion-retention considerations.
- Redis keys, TTLs, invalidation triggers, fallback behavior, and monitored metrics are documented.
- Cosmos DB Change Feed export is idempotent, checkpointed, and writes analytics-ready events to storage partitions.
- APIM policies protect public and partner APIs from malformed, unauthenticated, or excessive traffic.
- The design explains how it degrades during traffic spikes without corrupting billing, carts, or analytics history.
- Another engineer can reproduce the prototype with different globally unique names and produce equivalent evidence.

Stretch Goals

- Replace password-based SQL access with managed identity and Microsoft Entra authentication.
- Add private endpoints for SQL, Redis, Cosmos DB, and storage, then document DNS implications.
- Implement an APIM product with separate quotas for anonymous users, authenticated customers, partners, and internal services.
- Add a lifecycle management policy that moves old product images to cool storage and deletes expired campaign images.
- Build a Change Feed processor with poison-record handling and a replay test.
- Create a small load test that demonstrates Redis protecting Azure SQL during repeated product-card reads.
- Estimate monthly cost for normal traffic, launch-day traffic, and a cache-cold recovery event.

Appendix A

Cross-Cloud Service Equivalence

This appendix is a living reference that maps conceptually equivalent services and ideas across the six platforms studied in this book series: Amazon Web Services (AWS), Microsoft Azure, Microsoft Fabric, Google Cloud Platform (GCP), MongoDB, and Snowflake. The names differ, but the underlying data-engineering concepts—durable object storage, tiered lifecycle economics, immutability, managed relational engines, elastic compute, and disciplined data organization—recur everywhere.

Note

This appendix grows with each chapter. Every new chapter contributes the equivalence tables introduced in its cross-cloud callout, so by the end of the book you will hold a single consolidated Rosetta Stone for moving fluently between clouds. Where a clean one-to-one mapping does not exist, the prose notes the closest analog and the caveat.

A.1 Object and Data-Lake Storage

The foundational layer covered in Chapter 1. Every platform offers durable, HTTP-addressable object storage with tiered pricing, lifecycle transitions, versioning, and write-once-read-many (WORM) controls.

Capability	AWS	Azure	GCP
Object storage	Amazon S3	ADLS Gen2 / Blob	Cloud Storage
Hot tier	S3 Standard	Hot	Standard
Infrequent access	S3 Standard-IA	Cool	Nearline
Archive (instant)	Glacier Instant Retrieval	Cold	Coldline
Deep archive	Glacier Deep Archive	Archive	Archive
Lifecycle rules	Lifecycle policies	Lifecycle management	Object Lifecycle Mgmt
Versioning	S3 Versioning	Blob versioning	Object Versioning
Immutability (WORM)	Object Lock	Immutable blob storage	Bucket Lock
Cross-region copy	CRR / SRR	Object replication	Dual/multi-region + Transfer
Encryption at rest	SSE-S3 / SSE-KMS	SSE + CMK (Key Vault)	Google-managed / CMEK

The three remaining platforms approach storage differently. **Microsoft Fabric** exposes a single logical lake, *OneLake*, and uses *shortcuts* to virtualize S3, ADLS Gen2, and Google Cloud Storage in place without copying. **Snowflake** abstracts storage entirely: table data lives in cloud object storage as immutable, columnar *micro-partitions* that the service manages on your

behalf. **MongoDB** persists documents through the *WiredTiger* storage engine, balancing an internal cache against the operating-system page cache rather than exposing storage tiers.

A.2 Managed Relational Databases

The focus of Chapter 2 for the three hyperscalers. Each provides a fully managed relational engine, a cloud-native high-performance variant, high availability, read replicas, and a migration service.

Capability	AWS	Azure	GCP
Managed relational	Amazon RDS	Azure SQL Database	Cloud SQL
Cloud-native engine	Amazon Aurora	SQL Hyperscale	AlloyDB
Managed PostgreSQL	RDS / Aurora PostgreSQL	Azure DB for PostgreSQL	Cloud SQL / AlloyDB
High availability	Multi-AZ	Zone-redundant HA	Regional HA
Read scaling	Read Replicas	Read replicas / geo-replicas	Read Replicas
Global / geo	Aurora Global Database	Active geo-replication	Cross-region replicas
Serverless compute	Aurora Serverless v2	Azure SQL Serverless	AlloyDB (elastic)
Migration service	DMS + SCT	Azure Database Migration Svc	Database Migration Svc

Snowflake, **Fabric**, and **MongoDB** are not managed relational engines, but they solve overlapping problems: Snowflake and Fabric provide SQL analytics over warehouse/lakehouse storage, while MongoDB provides a distributed document store whose schema design (Chapter 2) replaces rigid relational normalization with intentional, access-driven modeling.

A.3 Elastic Compute and Analytical Warehouses

Snowflake's Chapter 2 topic—separating compute from storage and scaling it elastically—has a counterpart in every analytics platform.

Concept	Snowflake	AWS	Azure / GCP
Compute unit	Virtual Warehouse	Redshift RA3 / Serverless	Synapse pool / BigQuery slots
Elastic concurrency	Multi-cluster warehouse	Concurrency scaling	Autoscale / on-demand
Pause when idle	Auto-suspend / resume	Pause / resume	Auto-pause / serverless
Billing unit	Credits	RPU / node-hours	DWU / slot-hours
Result reuse	Result cache	Result cache	Result / BI cache

Microsoft Fabric meters all workloads against a shared pool of Capacity Units (CU) on an F-SKU, an approach closer to a single elastic budget than to per-engine warehouses. **MongoDB Atlas** scales through cluster tiers with optional auto-scaling of compute and storage.

A.4 Data Organization and the Medallion Pattern

Fabric's Chapter 2 topic—the Bronze/Silver/Gold medallion—is a portable design pattern rather than a product. Every platform implements the same progressive-refinement idea with its own primitives.

Layer	Fabric	Data lake (AWS/Azure/GCP)	Snowflake
Bronze (raw)	Bronze Lakehouse	`raw/` prefix (S3/ADLS/GCS)	RAW schema / stage
Silver (validated)	Silver Lakehouse	`validated/` prefix	STAGING schema
Gold (curated)	Gold Lakehouse	`curated/` prefix	ANALYTICS schema
Storage format	Delta-Parquet	Parquet / Delta / Iceberg	Micro-partitions

MongoDB realizes the same separation with distinct collections or databases (for example `raw_events`, `validated`, `curated`) governed by schema-design patterns rather than a lake layout.

A.5 Document Data Modeling

MongoDB's Chapter 2 schema-design patterns generalize to every document and wide-column store.

Concept	MongoDB	Amazon DynamoDB	Azure Cosmos DB
Primary access unit	Document / collection	Item / table	Item / container
Partitioning	Shard key	Partition key	Partition key
One-table modeling	Embedding + patterns	Single-table design	Embedding + partitioning
Time-series grouping	Bucket Pattern	Composite sort keys	Bucketing by key
Optional attributes	Attribute Pattern	Sparse attributes	Sparse properties

Google Cloud's closest analogs are **Firestore** and **Bigtable**. In **Snowflake** and **Fabric**, semi-structured documents are stored in `VARIANT` (Snowflake) or JSON columns and queried with path expressions rather than modeled as collections.

A.6 Globally Distributed and NoSQL Databases

Chapter 3 branches by platform into high-scale NoSQL (AWS DynamoDB, Azure Cosmos DB), globally consistent NewSQL (GCP Cloud Spanner), and advanced document modeling (MongoDB). They all answer the same question: how to scale a database horizontally across partitions and regions.

Concept	DynamoDB	Cosmos DB	Cloud Spanner	MongoDB
Model	Key-value / doc	Multi-model	Distributed SQL	Document
Partitioning	Partition key	Partition key	Split by key range	Shard key
Throughput unit	RCU / WCU	Request Units (RU)	Nodes / processing units	Cluster tier
Secondary indexes	GSI / LSI	Automatic indexing	Secondary indexes	Indexes
Change stream	DynamoDB Streams	Change Feed	Change streams	Change streams
Expiry	TTL	TTL	(manual)	TTL index
Multi-region	Global Tables	Multi-region writes	Multi-region (synchronous)	Zones / global clusters
Consistency	Eventual / strong	5 tunable levels	External (TrueTime)	Tunable read/write concern

Snowflake and **Fabric** are analytical platforms rather than OLTP key-value stores; they ingest from these operational databases and store semi-structured payloads in `VARIANT`/JSON columns. Google's dedicated NoSQL analogs are **Firestore** (document) and **Bigtable** (wide-column).

A.7 Managed Apache Spark

Microsoft Fabric's Chapter 3 topic—managed Spark pools and notebooks—has a first-class equivalent on every cloud.

Capability	Fabric	AWS	Azure / GCP
Managed Spark	Starter / Custom pools	Glue / EMR / EMR Serverless	Synapse Spark / Databricks; Dataproc
Notebook UX	Fabric Notebooks	Glue Studio / EMR Studio	Synapse / Databricks; Vertex AI
Serverless option	Autoscaling pools	Glue / EMR Serverless	Synapse serverless; Dataproc Serverless
Lake format	Delta on OneLake	Delta / Iceberg / Hudi	Delta; Delta / Iceberg

Snowflake offers **Snowpark** for DataFrame-style processing, and **MongoDB** integrates through the Spark Connector.

A.8 Query Acceleration and Materialized Views

Snowflake's Chapter 3 acceleration tools—materialized views, the Search Optimization Service, and transient/temporary tables—map to comparable features in every warehouse.

Feature	Snowflake	AWS Redshift	Azure Synapse / GCP BigQuery
Materialized views	Materialized Views	Materialized views	Materialized views
Point-lookup accel.	Search Optimization Svc	Sort keys / zone maps	(indexing) / search indexes
Result reuse	Result cache	Result cache	Result-set cache / BI Engine
Ephemeral tables	Transient / temporary	Temp tables	Temp tables

Fabric accelerates reads through Direct Lake and its Warehouse engine, while **MongoDB** relies on secondary and compound indexes plus the in-memory working set.

A.9 In-Memory Caching and Hybrid Ingestion

The AWS and Azure Chapter 4 topic—low-latency caching in front of a durable store, fed by hybrid (edge-to-cloud) ingestion—has a managed Redis-compatible equivalent on every cloud.

Capability	AWS	Azure	GCP / others
Managed cache	ElastiCache (Redis/Memcached)	Cache for Redis	Memorystore (Redis/Memcached)
Real-time API layer	AppSync / API Gateway	API Management / SignalR	API Gateway / Apigee
Hybrid ingestion	DataSync / Snow family / IoT Core	Data Box / IoT Hub / Arc	Transfer Appliance / IoT Core
Streaming intake	Kinesis / MSK	Event Hubs	Pub/Sub

Snowflake and **Fabric** rely on result and Direct Lake caches rather than a standalone Redis tier, while **MongoDB** serves hot data from its in-memory working set and Atlas can pair with an external cache.

A.10 Wide-Column and Key-Value NoSQL

GCP's Chapter 4 topic—Cloud Bigtable—is a wide-column, high-throughput, low-latency NoSQL store. Its peers span the managed and open-source ecosystems.

Characteristic	GCP Bigtable	AWS	Azure / open source
Data model	Wide-column	DynamoDB (key-value/doc)	Cosmos DB (multi-model)
HBase-compatible	HBase API	Keyspaces (Cassandra)	Managed Instance for Cassandra
Scaling unit	Nodes / clusters	On-demand / provisioned	RU/s (Cosmos)
Best fit	Time-series, IoT, ad-tech	Serverless key-value	Global multi-model

Snowflake and **Fabric** address these workloads analytically rather than as operational key-value stores; **MongoDB** covers the document end of the same NoSQL spectrum.

A.11 Data Protection: Cloning, Time Travel, and Migration

Snowflake's Chapter 4 topic—zero-copy cloning, Time Travel, and Fail-safe—plus MongoDB's relational-to-document migration map to data-protection and migration tooling across the clouds.

Capability	Snowflake	AWS / Azure	GCP / Fabric
Instant clone	Zero-copy clone	Aurora clone / SQL DB copy	BigQuery table clone; OneLake shortcut
Point-in-time recovery	Time Travel	PITR (RDS/SQL DB backups)	PITR; Delta time travel
Retention safety net	Fail-safe (7 days)	Automated backups	Backups; Delta VACUUM window
Schema migration	—	DMS / SCT	Database Migration Service

MongoDB migrations use **Relational Migrator** plus the embed-vs-reference patterns from Chapter 4, and **Delta Lake** exposes historical versions through `VERSION AS OF` time-travel queries.

Tip

Use this appendix as a study aid for the book's lockstep, cross-cloud approach: when you learn a concept on one platform, immediately locate its equivalent here and predict how the other five would express the same idea. That habit turns six separate vocabularies into one mental model.

Glossary

This glossary grows with each chapter and collects the cross-cloud data-engineering terms introduced so far. Definitions are platform-neutral unless a platform is named explicitly.

Access tier A storage pricing class that trades retrieval cost and latency for lower at-rest price (for example Azure Hot, Cool, Cold, and Archive). See also *storage class*.

AlloyDB Google Cloud's PostgreSQL-compatible engine with a columnar accelerator for analytical queries, positioned as the cloud-native tier above Cloud SQL.

Apache Spark A distributed data-processing engine; Microsoft Fabric runs it on Starter or Custom pools, with counterparts in AWS Glue/EMR, Azure Synapse/Databricks, and GCP Dataproc.

Aurora Amazon's cloud-native relational engine with distributed storage, Global Databases, and a serverless (v2) compute option.

Availability SLA The contractual uptime a provider guarantees for a service, distinct from *durability*, which concerns data loss rather than reachability.

Bigtable Google Cloud's wide-column, high-throughput, low-latency NoSQL store for time-series, IoT, and ad-tech workloads; exposes the HBase API and scales by nodes and clusters.

Bucket (storage) The top-level container for objects in AWS S3 and Google Cloud Storage; Azure calls the equivalent a *container*.

Bucket Pattern A MongoDB schema-design pattern that groups many small time-series readings into a single document to cut document count and metadata overhead.

Capped collection A fixed-size MongoDB collection that overwrites its oldest documents when full, useful for high-volume logs and bounded activity feeds.

Change Feed / Streams An ordered log of item-level changes used for event-driven processing; called Change Feed in Cosmos DB, DynamoDB Streams in DynamoDB, and change streams in MongoDB.

Checkpoint In WiredTiger, a periodic flush of a consistent snapshot of cached data to disk, bounding crash-recovery time together with the journal.

Clustering key A user-defined column set that Snowflake uses to co-locate related rows within micro-partitions, improving partition pruning on selective queries.

CMEK / CMK Customer-managed encryption keys (GCP CMEK; Azure customer-managed keys via Key Vault), giving the customer control over the key that encrypts data at rest.

Cloud Spanner Google Cloud's horizontally scalable, globally distributed relational database offering external (strong) consistency backed by TrueTime.

Consistency level A tunable guarantee about read recency; Azure Cosmos DB offers five (Strong, Bounded Staleness, Session, Consistent Prefix, Eventual).

Cosmos DB Azure's globally distributed multi-model NoSQL database, billed in Request Units (RU) and supporting multiple APIs (NoSQL, MongoDB, Cassandra).

Cross-Region Replication (CRR) Asynchronous copying of objects to a bucket in another region for disaster recovery; Azure's equivalent user-configured feature is *object replication*.

DynamoDB Amazon's managed key-value and document NoSQL database, with capacity measured in read/write capacity units (RCU/WCU) or on-demand, and Global Tables for multi-region.

Durability The probability that stored data is not lost, commonly quoted as "eleven nines" (99.999999999%) for cloud object storage.

ElastiCache Amazon's managed in-memory cache (Redis- and Memcached-compatible) placed in front of a durable store to serve hot data at sub-millisecond latency; Azure's analog is Cache for Redis and GCP's is Memorystore.

Embedding vs referencing The MongoDB modeling choice between nesting related data in one document (embedding) or linking documents by reference (and joining with `lookup`).

Fail-safe Snowflake's non-configurable seven-day recovery window, available only to Snowflake support, that follows the Time Travel retention period as a last-resort safety net.

Global Secondary Index (GSI) A DynamoDB index with its own partition/sort key enabling queries on non-primary attributes; a Local Secondary Index (LSI) shares the table's partition key.

Hierarchical namespace (HNS) The Azure Data Lake Storage Gen2 feature that adds true directories and atomic directory operations on top of Blob storage.

Journal (WAL) The write-ahead log that records changes before they are checkpointed, enabling crash recovery in MongoDB's WiredTiger engine.

JSON Schema validation MongoDB's rule mechanism (via `jsonSchema` in `createCollection`) that enforces required fields, data types, and value constraints.

Interleaved table A Cloud Spanner layout that physically co-locates child rows with their parent row to make parent-child joins fast.

Lakehouse An architecture that stores open table formats (such as Delta) in a data lake while offering warehouse-like SQL and governance; the core storage abstraction in Microsoft Fabric.

Lifecycle policy A rule set that automatically transitions or expires objects across storage tiers based on age or other conditions.

Medallion architecture A progressive-refinement data layout with Bronze (raw), Silver (validated), and Gold (curated/enriched) layers.

Materialized view A precomputed, automatically maintained query result stored for fast reads, trading maintenance/storage cost for lower query latency (Snowflake, Redshift, Synapse, BigQuery).

Memorystore Google Cloud's managed in-memory cache service (Redis and Memcached), the GCP counterpart to AWS ElastiCache and Azure Cache for Redis.

Micro-partition Snowflake's internal storage unit: an immutable, compressed, columnar file of roughly 50–500 MB of uncompressed data, annotated with per-column metadata for pruning.

Multi-AZ An AWS deployment spanning multiple Availability Zones for high availability; Azure's analog is zone-redundant HA and GCP's is regional HA.

Multi-cluster warehouse A Snowflake virtual warehouse that adds and removes compute clusters automatically to absorb concurrent query load.

Object Lock The AWS S3 WORM feature (governance mode, compliance mode, and legal hold) that prevents object deletion or overwrite for a retention period.

OneLake Microsoft Fabric's single, tenant-wide logical data lake shared by all workspaces.

Partition pruning Skipping storage units (micro-partitions, files, or partitions) that cannot match a query's predicates, reducing scanned data.

Point-in-time recovery (PITR) Restoring a database or table to any moment within a retention window, using continuous backups or transaction logs; realized as Snowflake Time Travel, RDS/SQL Database PITR, and Delta Lake time travel.

Read replica A read-only copy of a database that offloads read traffic and can serve as a failover or geo-local endpoint.

Relational Migrator MongoDB's tool for migrating relational schemas and data to document collections, applying embed-versus-reference decisions from schema-design patterns.

Request Unit (RU) Azure Cosmos DB's normalized currency for throughput, abstracting the CPU, memory, and IOPS consumed by a database operation.

Search Optimization Service (SOS) A Snowflake feature that builds a persistent search access path to accelerate selective point-lookup queries on high-cardinality columns.

Shortcut (Fabric) A OneLake reference that virtualizes external data (S3, ADLS Gen2, GCS) in place so it can be queried without copying.

Storage class The AWS/GCP term for a pricing tier of object storage (for example S3 Standard, Standard-IA, Glacier; GCP Standard, Nearline, Coldline, Archive).

SSE-KMS Server-side encryption in AWS S3 using keys managed in AWS Key Management Service.

Time Travel Snowflake's ability to query, clone, or restore data as it existed at an earlier point within a retention period (up to 90 days on higher editions); Delta Lake offers an analogous `VERSION AS OF` / `TIMESTAMP AS OF` capability.

Transient table A Snowflake table that persists across sessions but has no Fail-safe period, reducing storage cost for reproducible staging data; a temporary table lives only for one session.

TrueTime Google's globally synchronized clock (with bounded uncertainty) that lets Cloud Spanner assign commit timestamps and provide external consistency.

TTL (Time to Live) An expiry mechanism that automatically deletes items or documents after a set time (DynamoDB TTL, Cosmos DB TTL, MongoDB TTL index).

Versioning Retaining prior versions of an object or blob so that overwrites and deletes are recoverable.

Virtual warehouse Snowflake's unit of elastic compute, sized XS–6XL and billed in credits, fully decoupled from storage.

Wide-column store A NoSQL data model that stores rows as sparse, dynamically wide sets of column families keyed by a row key (Bigtable, HBase, Cassandra), optimized for high write throughput and range scans.

WiredTiger MongoDB's default storage engine, using a configurable internal cache alongside the operating-system page cache, with journaling and checkpointing.

WORM Write-once-read-many storage that prevents modification or deletion for a retention period; implemented as Object Lock (AWS), immutable blob storage (Azure), or Bucket Lock (GCP).

Z-ordering A Delta Lake optimization that co-locates related information in the same files by multi-dimensional clustering, improving data skipping; applied via `OPTIMIZE ... ZORDER BY` and complemented by `VACUUM` to reclaim obsolete files.

Zero-copy clone Snowflake's instant, storage-free copy of a table, schema, or database that shares underlying micro-partitions until either copy is modified (copy-on-write).

Bibliography

[1] Michael Armbrust, Tathagata Das, Liwen Sun, Burak Yavuz, Shixiong Zhu, Mukul Murthy, Joseph Torres, Herman van Hovell, Adrian Ionescu, Alicja Luszczak, et al. Delta lake: High-performance acid table storage over cloud object stores. *Proceedings of the VLDB Endowment*, 13(12):3411–3424, 2020.

[2] Michael Armbrust, Ali Ghodsi, Reynold Xin, and Matei Zaharia. Lakehouse: A new generation of open platforms that unify data warehousing and advanced analytics. *CIDR*, 2021.

[3] Microsoft. Access tiers for blob data. https://learn.microsoft.com/azure/storage/blobs/access-tiers-overview, 2025.

[4] Microsoft. Active geo-replication for azure sql database. https://learn.microsoft.com/azure/azure-sql/database/active-geo-replication-overview, 2025.

[5] Microsoft. Auto-failover groups overview and best practices. https://learn.microsoft.com/azure/azure-sql/database/auto-failover-group-sql-db, 2025.

[6] Microsoft. Azure api management policies. https://learn.microsoft.com/azure/api-management/api-management-policies, 2025.

[7] Microsoft. Azure cosmos db for nosql python sdk. https://learn.microsoft.com/python/api/overview/azure/cosmos-readme, 2025.

[8] Microsoft. Azure sql database purchasing models. https://learn.microsoft.com/azure/azure-sql/database/purchasing-models, 2025.

[9] Microsoft. Azure storage redundancy. https://learn.microsoft.com/azure/storage/common/storage-redundancy, 2025.

[10] Microsoft. Best practices for azure cache for redis. https://learn.microsoft.com/azure/azure-cache-for-redis/cache-best-practices, 2025.

[11] Microsoft. Change feed in azure cosmos db. https://learn.microsoft.com/azure/cosmos-db/change-feed, 2025.

[12] Microsoft. Configure multi-region writes in azure cosmos db. https://learn.microsoft.com/azure/cosmos-db/how-to-multi-master, 2025.

[13] Microsoft. Consistency levels in azure cosmos db. https://learn.microsoft.com/azure/cosmos-db/consistency-levels, 2025.

[14] Microsoft. Create your azure free account today. https://azure.microsoft.com/free/, 2025.

[15] Microsoft. Data migration assistant. https://learn.microsoft.com/sql/dma/dma-overview, 2025.

[16] Microsoft. Dedicated gateway in azure cosmos db. https://learn.microsoft.com/azure/cosmos-db/dedicated-gateway, 2025.

[17] Microsoft. Define your naming convention. https://learn.microsoft.com/azure/cloud-adoption-framework/ready/azure-best-practices/resource-naming, 2025.

[18] Microsoft. Get started with azcopy. https://learn.microsoft.com/azure/storage/common/storage-use-azcopy-v10, 2025.

[19] Microsoft. Global distribution in azure cosmos db. https://learn.microsoft.com/azure/cosmos-db/distribute-data-globally, 2025.

[20] Microsoft. Health insurance portability and accountability act on microsoft azure. https://learn.microsoft.com/azure/compliance/offerings/offering-hipaa-us, 2025.

[21] Microsoft. High availability in azure database for postgresql flexible server. https://learn.microsoft.com/azure/postgresql/flexible-server/concepts-high-availability, 2025.

[22] Microsoft. Hyperscale service tier for azure sql database. https://learn.microsoft.com/azure/azure-sql/database/service-tier-hyperscale, 2025.

[23] Microsoft. Install azure powershell. https://learn.microsoft.com/powershell/azure/install-azure-powershell, 2025.

[24] Microsoft. Install the azure cli on windows. https://learn.microsoft.com/cli/azure/install-azure-cli-windows, 2025.

[25] Microsoft. Integrated cache in azure cosmos db. https://learn.microsoft.com/azure/cosmos-db/integrated-cache, 2025.

[26] Microsoft. Introduction to azure blob storage. https://learn.microsoft.com/azure/storage/blobs/storage-blobs-introduction, 2025.

[27] Microsoft. Introduction to azure data lake storage gen2. https://learn.microsoft.com/azure/storage/blobs/data-lake-storage-introduction, 2025.

[28] Microsoft. Microsoft azure well-architected framework. https://learn.microsoft.com/azure/well-architected/, 2025.

[29] Microsoft. Microsoft cloud adoption framework for azure. https://learn.microsoft.com/azure/cloud-adoption-framework/, 2025.

[30] Microsoft. Optimize costs by automating azure blob storage access tiers. https://learn.microsoft.com/azure/storage/blobs/lifecycle-management-overview, 2025.

[31] Microsoft. Organize your resources with azure management groups, subscriptions, and resource groups. https://learn.microsoft.com/azure/cloud-adoption-framework/ready/azure-setup-guide/organize-resources, 2025.

[32] Microsoft. Overview of azure cloud shell. https://learn.microsoft.com/azure/cloud-shell/overview, 2025.

[33] Microsoft. Partitioning and horizontal scaling in azure cosmos db. https://learn.microsoft.com/azure/cosmos-db/partitioning-overview, 2025.

[34] Microsoft. Quickstart: Azure cosmos db for nosql. https://learn.microsoft.com/azure/cosmos-db/nosql/quickstart-portal, 2025.

[35] Microsoft. Request units in azure cosmos db. https://learn.microsoft.com/azure/cosmos-db/request-units, 2025.

[36] Microsoft. Serverless compute tier for azure sql database. https://learn.microsoft.com/azure/azure-sql/database/serverless-tier-overview, 2025.

[37] Microsoft. Tutorial: Create and manage azure budgets. https://learn.microsoft.com/azure/cost-management-billing/costs/tutorial-acm-create-budgets, 2025.

[38] Microsoft. Welcome to azure cosmos db. https://learn.microsoft.com/azure/cosmos-db/introduction, 2025.

[39] Microsoft. What is azure api management? https://learn.microsoft.com/azure/api-management/api-management-key-concepts, 2025.

[40] Microsoft. What is azure cache for redis? https://learn.microsoft.com/azure/azure-cache-for-redis/cache-overview, 2025.

[41] Microsoft. What is azure data box? https://learn.microsoft.com/azure/databox/data-box-overview, 2025.

[42] Microsoft. What is azure database for postgresql flexible server? https://learn.microsoft.com/azure/postgresql/flexible-server/overview, 2025.

[43] Microsoft. What is azure database migration service? https://learn.microsoft.com/azure/dms/dms-overview, 2025.

[44] Microsoft. What is azure role-based access control (azure rbac)? https://learn.microsoft.com/azure/role-based-access-control/overview, 2025.

[45] Microsoft. What is azure sql database? https://learn.microsoft.com/azure/azure-sql/database/sql-database-paas-overview, 2025.

[46] Microsoft. What is azure sql managed instance? https://learn.microsoft.com/azure/azure-sql/managed-instance/sql-managed-instance-paas-overview, 2025.

[47] Microsoft. What is microsoft entra id? https://learn.microsoft.com/entra/fundamentals/whatis, 2025.

Index